

别再手动复制粘贴了！B站好物提效篇——选品、内容、工具三维突破的实操

一、引言

一、引言

00后，毕业2年，一个普通的打工族，大学自学了Python，学习半年开始兼职接数据采集的单子。毕业后靠着Python的底子，花了十几天打通影刀RPA。依靠着编程和RPA的基础，在我进入深海圈开始尝试自己剪辑的时候，发现了一系列的难题。

例如：

1. 对标B站视频下面有现成的链接，一个个复制难？（视频下的链接需逐个复制，操作繁琐）
2. 获取到链接了，但是一个个去搜索查询佣金烦？（逐个搜索查询佣金，耗时费力）
3. 获得最终的链接后，获取产品参数还得手动复制？（流程冗余）
4. 终于复制完参数了，PPT图片的产品参数我还得一个个复制，有没有办法提效？（重复手动录入）
5. 产品参数复制完了，PPT图片的商品主图还得一个个找？（主图仍需逐个查找，效率低下）
6. 千辛万苦做完产品PPT了，脚本怎么办？（脚本创作成新难题）
7. 脚本写完了，剪辑怎么办？（视频剪辑环节再次面临效率瓶颈）

~~~~

接下来我会给大家分享我这两个月的提效总结。近两个月成果，打通了0-1。给我的教训是：工作中**不要主次不分，即使写提效代码，也要每天坚持更新视频**，保持内容创作的节奏。

## 二、选品提效

### 2.1 批量获取对标视频的带货链接

这里直接提供工具化解决方案，直接写成了工具：**window系统**可以直接下载使用；**mac电脑**请把代码丢给**豆包**，让豆包教你如何使用，获取专属使用指导

 [使用说明.txt](#)

注意里面的**导出Excel功能**，可以直接导出可以使用的京东或者淘宝的链接，而不是B站压缩的链接格式，便于后续操作。



B站评论提取工具-清泉.exe

41.14MB



B站评论提取工具-清泉

Cookie设置:

保存Cookie

视频BV号:

C3?spm\_id\_from=333.788.recommend\_more\_video~1&vd\_source=81f7c763cda9707266eadce4e245c8b0

爬取模式:

☒ 热门评论

☐ 最新评论

获取页数:

5

页 (每页约20条)

获取评论

导出所有评论(TXT)

导出UP主评论(TXT)

导出Excel

清空

获取成功!

视频标题: 买冲锋衣要注意什么参数? 2025年9月户外冲锋衣推荐|【拓路者】【伯希和】【凯乐石】【骆驼】冲锋衣推荐|UP主: 小洲户外

所有评论

UP主评论

UP主共发布 11 条评论

UP主评论总数: 11

=====

1. 【小洲户外】  
2025年9月冲锋衣推荐  
【软壳冲锋衣】  
性价比: <https://b23.tv/mall-I0oVZ-77Rij>  
性价比: <https://b23.tv/mall-I0oVZ-77RhV>  
性价比: <https://b23.tv/mall-I0oVZ-77RhO>  
大品牌: <https://b23.tv/mall-I0oVZ-76aK4>  
【防水硬壳冲锋衣】  
大牌有一定溢价: <https://b23.tv/mall-I0oVZ-74zPT>  
低海拔性价比: <https://b23.tv/mall-I0oVZ-77Rhu>  
高海拔性价比: <https://b23.tv/mall-I0oVZ-xdLPtjjsBs>  
时尚又有性能: <https://b23.tv/mall-I0oVZ-73ZD5>  
【三合一冲锋衣】

mac版



mac版代码.rar

33.76KB



2.2 批量查询佣金

最简单的方法：点击链接选品，输入待查询链接，识别链接，工具自动识别并输出佣金数据，然后看数据

带货商品 插入小店

我的选品库

商品计划 推荐商品 营销会场

请输入关键词搜索

☐ 自营商品 ☐ 佣金说明

链接选品

华硕MB169CK 1...

联想便携台式机...

爱国者ago15.6...

爱国者ago15.6...

爱国者ago15.6...

爱国者ago15.6...

爱国者14英寸便...

爱国者ago14英...

ARZOPA可桌铂1...

ARZOPA可桌铂...

爱国者16英寸2.5...

爱国者ago16英...

爱国者ago16英...

暂未选择商品  
请从左侧添加

编辑选品文案 取消 确定

带货商品 插入小店

链接添加商品

输入商品链接,如有多条请用“;”(英文逗号)分隔,每次最多支持30条链接

识别链接

支持淘宝联盟、京东联盟、多多进宝、会东购自营、up主小店、工房、课室、数字藏品、个性装扮商品加入支持识别以https://开头的、含商品id的商品链接,暂不支持京东app拼多多app和链接商品品类、禁售商品的不允许在网站带货: 带货商品要求  
带货商品添加遇到任何问题查看帮助文档: 带货商品帮助文档

输入链接,识别链接,看右侧的数据,简单心算一下

有效链接7/7 用于结算的佣金率以供货平台官网为准

TFO 第六代三合一冲锋衣 防水透...

TFO 第四代冲锋衣 情侣同款三合一...

TFO 第四代冲锋衣 防磨户外休闲...

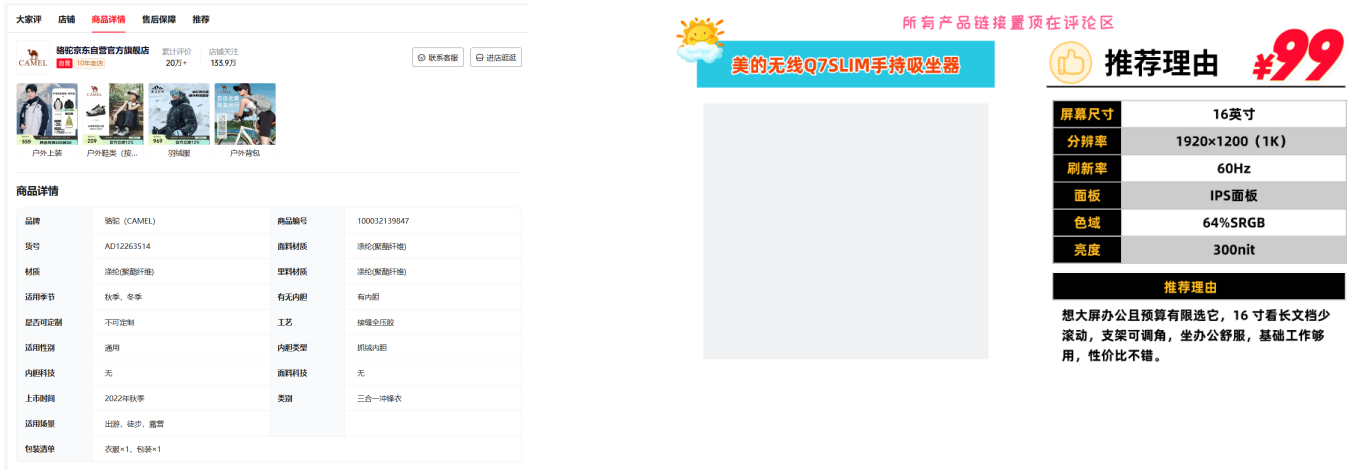
KAILAS凯乐石博格达PLUS冲锋衣...

伯希和 (Pellicot) 【山野经典2.0...

伯希和 (Pellicot) 【山野经典2.0...

2.3 批量获取产品参数

这里手动获取办法是打开链接，查看商品的详细参数，手动复制到PPT中，流程繁琐，如下图所示：



影刀逐个提取产品详情参数截图。

提效：[🔗 提效代码](#)

查看里面的影刀如何实现自动化抓取产品表格

抓取完表格，接下来可以有两种路可以走：

1. AI识别

把这些产品参数表格统统丢给豆包，让豆包识别给出参数

2. 影刀代码

这里说一下大体的逻辑了

a. 在自动化抓取产品表格的基础上，使用影刀离线ocr的功能，把每个产品参数图的数据分别写入到单个txt文档中，txt文件命名就用产品名字。如下所示。

📄 俏依 猪肉酥皮馅饼1.15kg10片 手抓饼煎饼大葱饼 学生早餐健康.txt

b. 把单个txt文档，全部写入到一个总的txt文档，用分隔符区分开，每个分隔。

例如：📄 总参数.txt

=== 文件: 麦西恩原味卷饼270g\_3 8英寸 早餐手抓饼墨西哥卷饼塔可饼皮.txt ===

[ '品牌', '麦西恩', '商品编号', '100136527424', '其他', '产地', '类别', '上海', '国产', '国产/进口', '保质期', '120天', '包装清单', '麦西恩原味卷饼270g\*3' ]

=== 文件结束: 麦西恩原味卷饼270g\_3 8英寸 早餐手抓饼墨西哥卷饼塔可饼皮.txt ===

c. 把这个总的txt文档发给豆包。提示词如下：

豆包提示词

- 1 请将txt文件里面的内容以 json的格式输出，便于我复制。另外，如果输出部分内容不确定，也可参考文件名获取相关信息，文件名就是商品，注意：严格按照我给的格式，引号需要成对出现。注意：每一个文件名都要输出，不能遗漏
- 2 输出格式例如：[
- 3 {
- 4 "文件名": "xxxx",
- 5 "品牌": "博世（BOSCH）",
- 6 "类型": "滚简单洗",
- 7 "能效等级": "一级能效",
- 8 "变频 / 定频": "变频",
- 9 },
- 10 {
- 11 "文件名": "xxxx",
- 12 "品牌": "博世（BOSCH）",
- 13 "类型": "滚简单洗",
- 14 "能效等级": "一级能效",
- 15 "变频 / 定频": "变频",
- 16 },
- 17 ]

会得到一个json文件

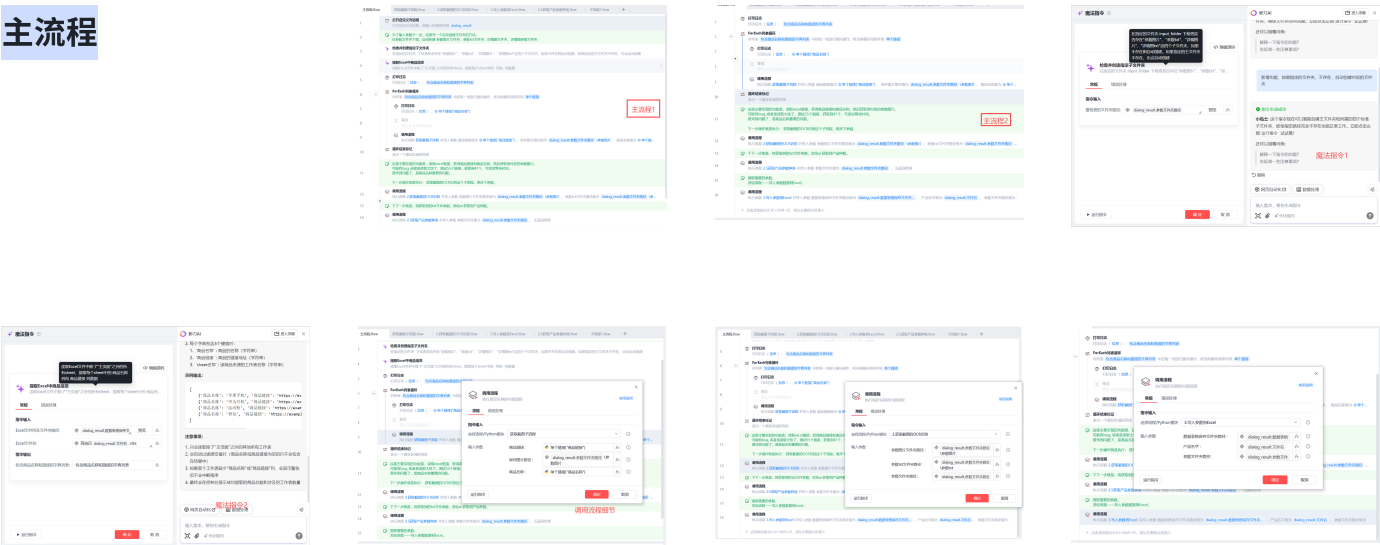
d. 接下来就是编写一个**魔法指令**，让影刀根据json文件提取参数写入到Excel中。

这样就大功告成了===

因为代码编写涉及到许多魔法指令以及图像识别，所以不便分享。

仅供参考，建议还是自己写一个符合自己项目的更快一点。

主流程





## 2.获取截图的ocr识别



## 2.5获取产品参数种类



## 3.写入参数到Excel



## 获取截图子流程



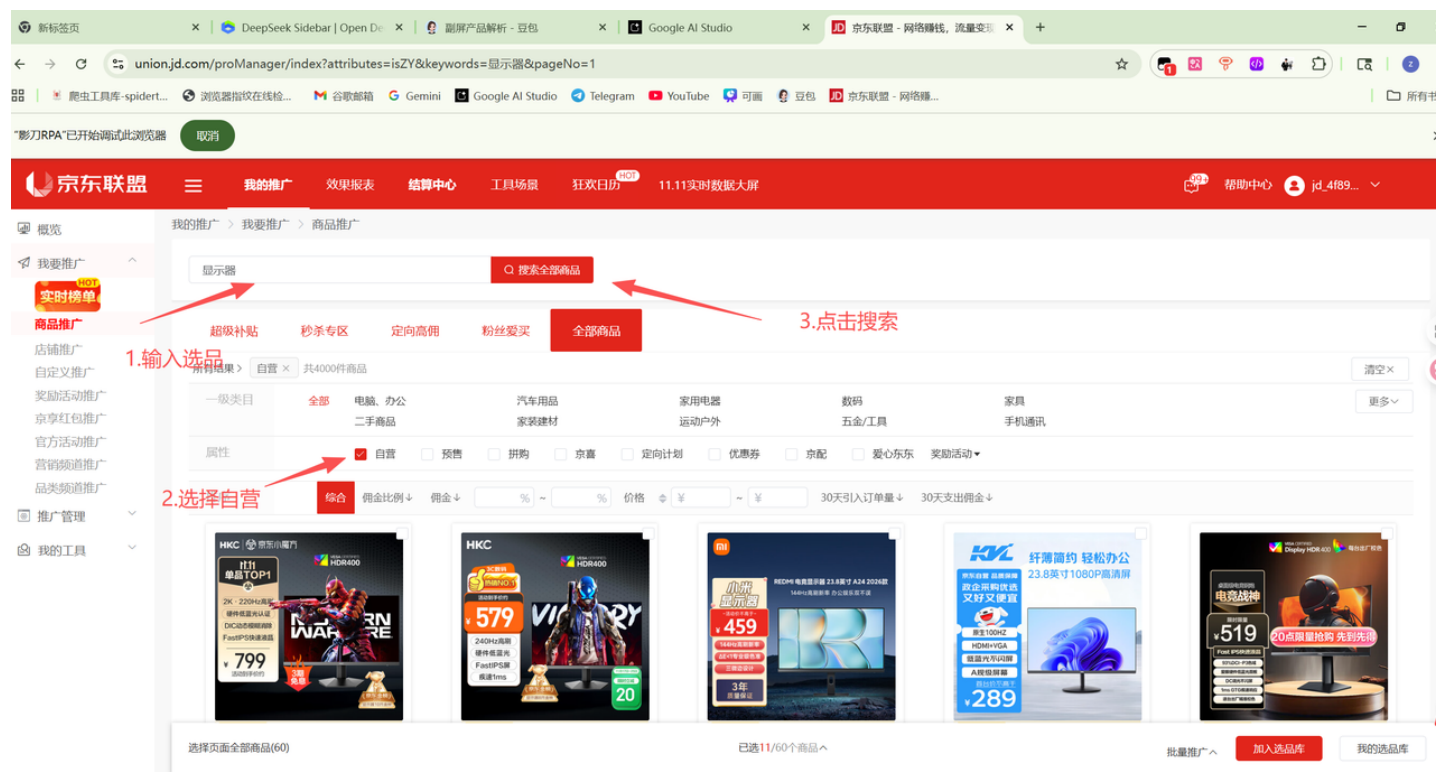
## 2.4 批量获取京东联盟商品信息

自己开发思路：

1. 打开影刀，使用网页自动化功能，使用最上方的批量抓取功能。
2. 使用魔法指令，建立筛选逻辑，导出表格。

使用方法：

Chrome浏览器打开京东联盟，搜索好产品，并选择"自营"。



<https://api.winrobot360.com/redirect/robot/share?inviteKey=1470c7fe6b4e4eb1>

影刀链接如上。用**chrome浏览器**打开这个页面后，点击运行就可以了。输入一下导出路径和导出表格名称。

## 界面预览

## 工具

导出保存文件夹路径

E:\B站\选品\外设-显示器-25-0930

表格名称

外设-显示器-25-0930

这个筛选逻辑是学习自圈友王小小的精华帖，并优化了一下价格区间。

**显示结果：**

WPS Office

找租壳模板

S 内搭-冲锋衣.xlsx

S 筛选后内搭-冲锋衣.xlsx

文件

自动保存

开始

插入

页面

公式

数据

审阅

视图

工具

会员专享

效率

智能工具箱

格式刷

粘贴

宋体

11

A<sup>T</sup>

=

常规

转换

B

I

U

A

田

背景色

前景色

合并

条件格式

| N7 | A         | B         | C      | D      | E        | F         | G         | H          | I    | J    | K | L | M |
|----|-----------|-----------|--------|--------|----------|-----------|-----------|------------|------|------|---|---|---|
| 1  | 商品名称      | 商品链接      | 到手价    | 佣金     | 佣金比例     | 好评数       | 佣金打分      | 金比例打       | 好评打分 | 相加   |   |   |   |
| 2  | TFO 第六件   | https://i | 566.37 | 141.59 | 25.00%   | 10        | 583       | 1081       | 1    | 1665 |   |   |   |
| 3  | TFO 第四件   | https://i | 463.42 | 115.86 | 25.00%   | 3         | 477       | 1081       | 1    | 1559 |   |   |   |
| 4  | TFO 第四件   | https://i | 463.42 | 115.86 | 25.00%   | 6         | 477       | 1081       | 1    | 1559 |   |   |   |
| 5  | KAILAS凯   | https://i | 2520   | 262.08 | 10.40%   | 200       | 1081      | 424        | 12   | 1517 |   |   |   |
| 6  | 伯希和       | https://i | 499    | 29.94  | 6.00%    | 20000     | 122       | 226        | 1081 | 1429 |   |   |   |
| 7  | 伯希和       | https://i | 499    | 29.94  | 6.00%    | 20000     | 122       | 226        | 1081 | 1429 |   |   |   |
| 8  | 伯希和       | https://i | 499    | 29.94  | 6.00%    | 20000     | 122       | 226        | 1081 | 1429 |   |   |   |
| 9  | 伯希和       | https://i | 499    | 29.94  | 6.00%    | 20000     | 122       | 226        | 1081 | 1429 |   |   |   |
| 10 | 伯希和       | https://i | 499    | 29.94  | 6.00%    | 20000     | 122       | 226        | 1081 | 1429 |   |   |   |
| 11 | 伯希和       | https://i | 499    | 29.94  | 6.00%    | 20000     | 122       | 226        | 1081 | 1429 |   |   |   |
| 12 | 伯希和       | https://i | 499    | 29.94  | 6.00%    | 20000     | 122       | 226        | 1081 | 1429 |   |   |   |
| 13 | 伯希和       | https://i | 499    | 29.94  | 6.00%    | 20000     | 122       | 226        | 1081 | 1429 |   |   |   |
| 14 | 骆驼        | https://i | 439    | 27.22  | 6.20%    | 20000     | 111       | 235        | 1081 | 1427 |   |   |   |
| 15 | 骆驼        | https://i | 439    | 27.22  | 6.20%    | 20000     | 111       | 235        | 1081 | 1427 |   |   |   |
| 16 | 骆驼        | https://i | 439    | 27.22  | 6.20%    | 20000     | 111       | 235        | 1081 | 1427 |   |   |   |
| 17 | 骆驼        | https://i | 439    | 27.22  | 6.20%    | 20000     | 111       | 235        | 1081 | 1427 |   |   |   |
| 18 | 骆驼户外      | https://i | 398    | 24.68  | 6.20%    | 20000     | 100       | 235        | 1081 | 1416 |   |   |   |
| 19 | 骆驼户外      | https://i | 398    | 24.68  | 6.20%    | 20000     | 100       | 235        | 1081 | 1416 |   |   |   |
| 20 | 骆驼户外      | https://i | 398    | 24.68  | 6.20%    | 20000     | 100       | 235        | 1081 | 1416 |   |   |   |
| 21 | 骆驼户外      | https://i | 398    | 24.68  | 6.20%    | 20000     | 100       | 235        | 1081 | 1416 |   |   |   |
| 22 | 骆驼户外      | https://i | 398    | 24.68  | 6.20%    | 20000     | 100       | 235        | 1081 | 1416 |   |   |   |
| 23 | 骆驼户外      | https://i | 398    | 24.68  | 6.20%    | 20000     | 100       | 235        | 1081 | 1416 |   |   |   |
| 24 | 骆驼户外      | https://i | 398    | 23.88  | 6.00%    | 20000     | 97        | 226        | 1081 | 1404 |   |   |   |
| 25 | KAILAS凯   | https://i | 1680   | 174.72 | 10.40%   | 53        | 720       | 424        | 4    | 1148 |   |   |   |
| 26 | KAILAS FL | https://i | 2600   | 195    | 7.50%    | 18        | 804       | 294        | 2    | 1100 |   |   |   |
| 27 | KAILAS凯   | https://i | 4200   | 218.4  | 5.20%    | 85        | 901       | 190        | 6    | 1097 |   |   |   |
| 28 | KAILAS凯   | https://i | 1440   | 149.76 | 10.40%   | 100       | 617       | 424        | 6    | 1047 |   |   |   |
| 29 | KAILAS凯   | https://i | 3600   | 187.2  | 5.20%    | 100       | 772       | 190        | 6    | 968  |   |   |   |
| 30 | KAILAS FL | https://i | 2100   | 157.5  | 7.50%    | 29        | 649       | 294        | 3    | 946  |   |   |   |
| 31 | KAILAS FL | https://i | 2100   | 157.5  | 7.50%    | 6         | 649       | 294        | 1    | 944  |   |   |   |
| 32 | KAILAS凯   | https://i | 3200   | 166.4  | 5.20%    | 100       | 686       | 190        | 6    | 882  |   |   |   |
| 33 | 骆驼        | https://i | 399    | 24.74  | 6.20%    | 10000     | 101       | 235        | 541  | 877  |   |   |   |
| IK | <         | >         | 主页面    | 0-500  | 500-1000 | 1000-3000 | 3000-5000 | 5000-10000 | +    |      |   |   |   |

应用场景：选择高佣产品。

## 2.5 目前我在用的方法

注意：我的方法是环环相扣的，需要一个个影刀脚本去执行。

1. 影刀脚本：批量填充分类数据
2. 影刀脚本：导出详情图txt-11月30日更新
3. 影刀脚本：填补空缺参数

需要逐个去运行。

现在我使用的方法：

1. 找到一个对标的B站链接，把它放到Excel中。

 分类 - 模板.xlsx

代码块

- 1 使用方法：
- 2 创建完文件夹后，把这个Excel放入其中，重命名为**分类.xlsx**
- 3 写一个分类名称，放入对应的链接
- 4 注意这个文件夹，后续两个脚本使用的文件夹路径就是自己创建的这个。只需要选到这个文件夹即可。

2. 影刀脚本：**批量填充分类数据**。就可以一键提取对标链接到Excel中，并把链接对应的佣金及推广链接导出到Excel中。(进行下一步前 可以手动添加其他的链接补充)

分享提效原理

- 1 1. 读取存放在Excel里面的B站链接
- 2 2. 根据B站链接获取对应的对标视频链接，区分出是京东链接还是淘宝/天猫链接，写入到Excel里。代码参考**2.1**
- 3 3. 分别读取Excel里面商品链接，将商品链接填写到京东联盟-商品推广的输入框中，逐个添加到选品库，最后点击批量推广->一键推广，会导出一个存放着产品佣金信息的csv表格。
- 4 4. 数据处理：将csv表格数据读取出来，填写到Excel中。
- 5 具体流程看**演示视频**。

影刀链接及注意事项

- 1 <https://api.winrobot360.com/redirect/robot/share?inviteKey=39421485e4958c5f>
- 2
- 3 需要电脑有chrome浏览器和edge浏览器。
- 4 需要chrome浏览器的京东联盟处于登录状态
- 5 需要chrome浏览器的在下载之前询问每个文件的保存位置这个功能要打开

- 6
- 7 拿另外一台笔记本，测试通过，可以正常使用。
- 8 注：由于每个人的电脑情况不同。网络速度不同，无法运行是正常情况



演示第一步和第二步.mp4

212.85MB



### 3. 影刀脚本：导出详情图txt。输入路径，就可以一键实现把Excel里面的对标链接的商品图片及详情图都爬取下来。

代码块

- 1 2025年11月30日更新
- 2 说一个情况，使用的这个插件-cube-seller，获取不到京东的产品详情图片了，插件作者也不更新。
- 3 无奈，动用了一下爬虫知识，自己写了一个爬取京东链接的产品详情图的代码。
- 4 更新后的影刀链接，这次是个大更新，摒弃了以往需要点击插件才能采集商品详情的方法。采用独家开发的京东商品采集详情的代码。
- 5 本次更新优点：适配不同电脑，不用准备插件。不涉及个人账号
- 6 本次更新缺点：不支持淘宝链接
- 7 影刀链接：
- 8 <https://api.winrobot360.com/redirect/robot/share?inviteKey=a005f2c75d8c8437>

弃用：只需要准备好Excel就可以，其他的条件不需要了。比如京东

分享提效原理

- 1 1.准备好一个Excel，里面存放好要读取的商品链接。
- 2
- 3 2.准备好一个网页插件，例如 cubeseller-魔方电商助手，要求可以打包下载到本地 或者你有神奇的代码，可以无视京东/淘宝平台，直接爬取商品详情图。
- 4
- 5 3.首先先实现单个商品链接的商品详情图爬取，例如：使用cubeseller,点击插件图标，逐个点击图标，最后使用影刀的下载文件功能，修改保存地址重命名保存。（备注：使用插件下载，无法使用网页元素定位，采用图像识别定位方法点击，由于图像识别存在误判，故影刀代码通用性差。）
- 6
- 7 4.实现单个商品链接的详情图爬取后，批量读取Excel链接，写个循环批量批量爬取。
- 8
- 9 5.批量解压得到详情图后，使用影刀的离线OCR识别，循环读取图片并把识别结果保存到txt里面。（备注：魔法指令可以多用途）

代码块

- 1 <https://api.winrobot360.com/redirect/robot/share?inviteKey=fa826333189330b0>
- 2

- 3 注意事项：（不需要了，最新版的影刀代码不需要这个注意事项了，输入直接用就可以）
- 4 需要edge浏览器的京东处于登录状态
- 5 需要edge浏览器安装扩展 **cubeseller-魔方电商助手**，并固定在工具栏，并把这个扩展的**显示页面悬浮按钮 这个按钮关掉。**
- 6 还有一个要求就是浏览器的在下载之前询问每个文件的保存位置这个功能要打开
- 7
- 8
- 9 拿另外一台笔记本测试，有个图像识别没有通过，修改了一下测试通过。
- 10 注意⚠️：
- 11 1. 因为涉及到图像识别/每个人的电脑不同，有概率报错。
- 12 2. 受限于网速，可能会运行出错，重新运行。
- 13 3. 看到我的演示视频，有思路的同学，可以直接拿影刀手搓一个了



演示3.mp4

418.92MB



#### 4. 影刀脚本：填补空缺参数。输入路径及需要的参数。就可以一键实现把参数提取到Excel中。

##### 分享提效原理

- 1 所有的代码其实是环环相扣的，但是由于影刀执行的不稳定性，选择分成多个脚本执行。
- 2
- 3 **这一步的原理是其实是利用AI，把每个产品对应的详情图发给豆包，再发送提示词给豆包，得到json格式的返回数据，读取json数据写入到excel中更新参数。**
- 4
- 5 **备注：**我这里使用的是影刀操作网页，自动输入参数及提示词得到返回结果。有条件可以采用API的形式得到返回结果。
- 6
- 7 **提示词：**
- 8 注意： 给出的参数要被用在PPT展示，除了推荐理由和文件名这个字段，其他参数字数一律限制在13个字以内。注意：文件名要用txt里面的完整输出。
- 9 注意：输出的第一个文件名，后面不要用简称，txt里面是什么名字就填什么名字。



10 例如：色域控制在“95% DCI-P3, 100% SRGB”，接口控制在“2DP1.4+2HDMI2.0+Audio”。不能超过  
我给出的示例字数

11 推荐理由标准：

12 1. 结构强制要求（缺一不可）

13 必须包含“精准目标人群 + 具体场景痛点 + 参数对应解决方案 + 适配/性价比细节”，逻辑闭环，拒  
绝抽象概括。

14 2. 场景化表达规范

15 人群要“具体到身份”

16 痛点要“落地到动作”

17 参数要“转化到利益”

18 优势要“补充到细节”

19 3. 风格与字数

20 语言口语化，像“朋友推荐”的语气，有画面感；控制在40-50字，一句话讲清核心价值，无冗余修  
饰。

21

22 注意：如果有未明确的参数，去互联网查找相关产品的相关参数，写出来。不要写未明确，未提及。

23 请基于提供的文档内容，提取指定的参数信息并以 JSON 格式输出，需满足「商品介绍场景通用展示」  
需求（如 PPT、详情页、宣传物料等），具体要求如下：

24 参数覆盖：严格按照指定参数维度提取，每个维度均需填写，未明确的信息可结合文件名及文档上下文  
合理补充，确保无遗漏。

25 内容规范：

26 优先提取核心关键信息，避免冗余技术细节（如复杂操作步骤、非核心测试数据），“特性” 字段汇总  
商品核心功能、认证、赠品、售后等优势，“推荐理由” 浓缩为 1-2 句商品核心价值总结（突出差异  
化优势）；

27 数值单位统一（如 Hz、nit、ms 小写，分辨率用 “x” 连接，避免 “\*”），同类参数表述一致（如  
“超频”“原生”“认证” 等关键词用法统一）。

28 注意：txt文件里面的每一个产品都要被提取到，不能有遗漏。txt文件名显示为1-20，说明有20个产  
品，一个也不能少。

29 格式要求：

30 JSON 结构规范，引号成对，无语法错误，每个商品为独立 JSON 对象，整体用数组包裹；

31 不遗漏任何文件名，文件名需与文档中完全一致，品牌信息从文件名或文档内容中准确提取。

32 通用适配：输出内容需轻量化、易读易懂，参数值用 “关键数据 + 核心属性” 简洁表述，可直接用于  
各类展示场景，无需二次大量调整。

33 注意文件名，不要采用简称，采用原始名称

34 输出格式例如：[

35 {

36 "文件名": "xxxx",

37 "商品简称": "xxx",

38 "类型": "滚筒简单洗",

39 "能效等级": "一级能效",

40 "变频 / 定频": "变频",

41 },

42 {

43 "文件名": "xxxx",

44 "商品简称": "xxx",

45 "类型": "滚筒简单洗",

46 "能效等级": "一级能效",



```
47 "变频 / 定频": "变频",
48 },
49 ]
```

#### 代码块

```
1  https://api.winrobot360.com/redirect/robot/share?inviteKey=b67340a416053657
2
3  需要edge/chrome浏览器的豆包处于登录状态
4  需要edge浏览器的京东处于登录状态
5
6  输入参数有两个，第一个很简单，就是自己创建的文件夹。文件夹里面是分类.xlsx
7  第二个这里有个需要注意的地方，就是第一个是商品简称，我建议保留，因为PPT制作时不可能把商品
  名称全部填写上去。注意一下，参数输入是英文逗号，还有就是可以在自己想要的参数后面 加上， 适
  用人群，适用场景，推荐理由等参数。
8  如下：
9  ["商品简称", "面板", "对比度", "类型", "响应时间", "屏幕刷新率", "接口", "特性", "分辨
  率", "屏幕尺寸", "亮度", "支架底座", "色域", "推荐理由"]
10 ["商品简称", "防水指数", "耐磨指数", "透湿性能", "适用人群", "适用场景", "推荐理由"]
11 ["商品简称", "品牌", "桌面长度", "是否可定制", "桌面可调角度", "桌面材质", "控制方式",
  "适用人群", "适用场景", "推荐理由"]
12 注意：
13 自己测试，没有登录豆包，会出现bug,因为无法复制。
14 还有有时候，会出现延迟的情况，会出现 一个弹窗，请手动复制，这个时候点击一下 复制按钮，再点
  击确定即可。
15
```

## 提取冲锋衣参数信息

内容由 AI 生成

"透气性能": "国标 10000 / 日标 24000g/(m<sup>2</sup> · 24h)",

"适用场景": "高海拔攀岩、户外攀冰、重装登山",

"推荐理由": "专业户外玩家必备, GORE-TEX 面料透气, 全衣压胶 + 搜救器, 极限环境适配"

}

]

复制

提示

请手动点击复制按钮

确定



推荐一些性价比高的冲锋衣 →

如何选择适合自己的冲锋衣? →

→ 冲锋衣\_合并\_1-7.txt

询问关于这篇文章的任何问题



深度思考

技能



## 最终效果

完美解决参数问题, 再也不需要手动复制了



演示影刀代码搭建-给想要自己搭建的小伙伴一个借鉴的思路，演示时长半个小时。



录屏\_20251128\_230239.mp4

596.59MB



## 魔法指令

批量填充分类数据-在原文件中创建统一格式的Excel工作表

```
1  # 使用此指令前，请确保安装必要的Python库，例如使用以下命令安装：
2  # pip install pandas openpyxl
3
4  import pandas as pd
5  from openpyxl import load_workbook
6  from openpyxl.styles import Font, Alignment, Border, Side
7
8  from typing import *
9  try:
10     from xbot.app.logging import trace as print
11 except:
12     from xbot import print
13
14 def create_sheets_by_category_with_format(excel_path):
15     """
16     title: 在原文件中创建统一格式的Excel工作表
17     description: 读取Excel文件中 % 分类 % 列的内容，为每个分类在原文件中创建新的工作
18     表，对所有工作表（除"分类"外）设置统一格式，列名使用方正仿宋_GBK加粗，数据行使用等线字体。
19     将Sheet1重命名为"分类"。
20
21     inputs:
22     - excel_path (file): Excel文件路径，eg: "data.xlsx"
23     outputs: None
24     """
25
26     def _read_excel_and_get_categories(file_path):
27         """读取Excel文件并获取分类列的唯一值"""
28         try:
29             df = pd.read_excel(file_path)
30             if '分类' not in df.columns:
31                 raise ValueError("Excel文件中未找到'分类'列")
32             categories = df['分类'].dropna().unique().tolist()
33             return categories
34         except Exception as e:
35             raise Exception(f"读取Excel文件失败: {str(e)}")
```

```
34     def _check_sheet_exists_with_columns(workbook, sheet_name,
required_columns):
35         """检查工作表是否存在且包含指定列名"""
36         if sheet_name not in workbook.sheetnames:
37             return False
38
39         worksheet = workbook[sheet_name]
40         if worksheet.max_row == 0:
41             return False
42
43         # 获取第一行的值作为列名
44         first_row = [cell.value for cell in worksheet[1]]
45         existing_columns = [col for col in first_row if col is not None]
46
47         # 检查是否包含所有必需的列
48         return all(col in existing_columns for col in required_columns)
49
50     def _clean_sheet_name(category_name):
51         """清理工作表名称，确保不包含'sheet'并符合Excel命名规则"""
52         sheet_name = str(category_name).replace('sheet', '').replace('Sheet',
('').replace('SHEET', '')
53         # 移除Excel不允许的字符
54         invalid_chars = ['\\', '/', '*', '[', ']', ':', '?']
55         for char in invalid_chars:
56             sheet_name = sheet_name.replace(char, '_')
57         # 确保名称不为空且不超过31个字符
58         sheet_name = sheet_name.strip()
59         if not sheet_name:
60             sheet_name = "分类表"
61         return sheet_name[:31]
62
63     def _rename_sheet1_to_category(workbook):
64         """将Sheet1重命名为分类"""
65         if 'Sheet1' in workbook.sheetnames:
66             sheet1 = workbook['Sheet1']
67             if '分类' not in workbook.sheetnames:
68                 sheet1.title = '分类'
69             else:
70                 # 如果分类已存在，删除Sheet1
71                 workbook.remove(sheet1)
72
73     def _format_worksheet(worksheet, column_names):
74         """设置工作表统一格式：固定行高、列宽和预设格式，确保后续新增列也能保持统一"""
75         # 定义每列的固定宽度
76         column_widths = {
77             '联盟类型': 12,
78             '商品链接': 20,
```

```

79         '佣金': 10,
80         '佣金比例': 12,
81         '推广链接': 20,
82         '商品名称': 18,
83         '到手价': 10,
84         '好评数': 10
85     }
86
87     # 统一的字体和样式设置
88     header_font = Font(name='方正仿宋_GBK', size=11, bold=True,
color='000000') # 列名字体不变
89     data_font = Font(name='等线', size=10, color='000000') # 数据行字体改为等
线
90     header_alignment = Alignment(horizontal='center', vertical='center')
91     data_alignment = Alignment(horizontal='left', vertical='center')
92     border = Border(
93         left=Side(border_style='thin', color='000000'),
94         right=Side(border_style='thin', color='000000'),
95         top=Side(border_style='thin', color='000000'),
96         bottom=Side(border_style='thin', color='000000')
97     )
98
99     # 清空工作表内容并添加列名
100    worksheet.delete_rows(1, worksheet.max_row)
101
102    # 设置所有列的默认宽度和格式（预设到Z列，确保后续新增列也有统一格式）
103    for col_idx in range(1, 27): # A到Z列
104        column_letter = worksheet.cell(row=1, column=col_idx).column_letter
105        worksheet.column_dimensions[column_letter].width = 15
106        worksheet.column_dimensions[column_letter].auto_size = False
107
108    # 设置指定的列名和其对应的列宽
109    for col_idx, col_name in enumerate(column_names, 1):
110        cell = worksheet.cell(row=1, column=col_idx)
111        cell.value = col_name
112        cell.font = header_font
113        cell.alignment = header_alignment
114        cell.border = border
115
116    # 设置特定列的宽度
117    column_letter = cell.column_letter
118    width = column_widths.get(col_name, 15)
119    worksheet.column_dimensions[column_letter].width = width
120
121    # 为标题行的所有列（A到Z）预设格式，确保后续新增列名也有统一格式
122    for col_idx in range(1, 27): # A到Z列
123        cell = worksheet.cell(row=1, column=col_idx)

```

```
124         if not cell.value: # 如果单元格为空, 不设置值
125             pass
126         cell.font = header_font
127         cell.alignment = header_alignment
128         cell.border = border
129
130     # 设置固定行高 (禁用自适应)
131     # 标题行
132     worksheet.row_dimensions[1].height = 24
133     worksheet.row_dimensions[1].auto_size = False
134
135     # 预设数据行的固定行高和格式 (预设500行, 确保后续新增数据也有统一格式)
136     for row_num in range(2, 501):
137         worksheet.row_dimensions[row_num].height = 20
138         worksheet.row_dimensions[row_num].auto_size = False
139
140     # 为数据行的所有列预设格式 (使用等线字体)
141     for col_idx in range(1, 27): # A到Z列
142         cell = worksheet.cell(row=row_num, column=col_idx)
143         cell.font = data_font # 数据行使用等线字体
144         cell.alignment = data_alignment
145         cell.border = border
146
147     # 冻结首行
148     worksheet.freeze_panes = 'A2'
149
150     def _create_and_format_sheets(file_path, categories):
151         """创建工作表并格式化所有工作表 (除分类外) """
152         column_names = ['联盟类型', '商品链接', '佣金', '佣金比例', '推广链接', '商
商品名称', '到手价', '好评数']
153
154         try:
155             workbook = load_workbook(file_path)
156         except Exception as e:
157             raise Exception(f"无法打开Excel文件: {str(e)}")
158
159         # 将Sheet1重命名为分类
160         _rename_sheet1_to_category(workbook)
161
162         # 创建新的分类工作表
163         for category in categories:
164             sheet_name = _clean_sheet_name(category)
165
166             # 跳过"分类"工作表
167             if sheet_name == '分类':
168                 continue
169
```

```

170         # 检查工作表是否已存在且包含指定列名
171         if _check_sheet_exists_with_columns(workbook, sheet_name,
column_names):
172             continue
173
174         # 创建或清理工作表
175         if sheet_name not in workbook.sheetnames:
176             workbook.create_sheet(title=sheet_name)
177
178         # 格式化所有工作表 (除了"分类")
179         for sheet_name in workbook.sheetnames:
180             if sheet_name != '分类':
181                 worksheet = workbook[sheet_name]
182                 _format_worksheet(worksheet, column_names)
183
184         # 保存文件
185         workbook.save(file_path)
186         workbook.close()
187
188         # 获取分类列表
189         categories = _read_excel_and_get_categories(excel_path)
190
191         if not categories:
192             raise ValueError("分类列中没有找到有效数据")
193
194         # 创建工作表并格式化
195         _create_and_format_sheets(excel_path, categories)
196

```

批量填充分类数据-获取B站视频标题并更新Excel

```

1  # 使用此指令前, 请确保安装必要的Python库, 例如使用以下命令安装:
2  # pip install openpyxl requests beautifulsoup4
3
4  import openpyxl
5  import requests
6  from bs4 import BeautifulSoup
7  import re
8  import time
9
10 from typing import *
11 try:
12     from xbot.app.logging import trace as print
13 except:
14     from xbot import print
15

```



```

16 def get_bilibili_titles_and_update_excel(excel_file):
17     """
18     title: 获取B站视频标题并更新Excel
19     description: 从Excel文件的"分类"sheet中的 % 对标链接 % 列读取B站链接, 获取视频标题后写入 % 对标标题 % 列的相应行中, 直接修改原文件。
20     inputs:
21         - excel_file (file): Excel文件路径, eg: "data.xlsx"
22     outputs: None
23     """
24
25     def _get_bilibili_title(url):
26         """
27         获取B站视频标题
28         """
29         try:
30             # 设置请求头, 模拟浏览器访问
31             headers = {
32                 'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36',
33                 'Accept':
34                 'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8',
35                 'Accept-Language': 'zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-
US;q=0.3,en;q=0.2',
36                 'Accept-Encoding': 'gzip, deflate',
37                 'Connection': 'keep-alive',
38                 'Upgrade-Insecure-Requests': '1'
39             }
40
41             response = requests.get(url, headers=headers, timeout=10)
42             response.raise_for_status()
43
44             soup = BeautifulSoup(response.content, 'html.parser')
45
46             # B站视频标题通常在title标签或特定的meta标签中
47             title_element = soup.find('title')
48             if title_element:
49                 title = title_element.get_text().strip()
50                 # 去掉B站标题后缀
51                 title = re.sub(r'_哔哩哔哩_bilibili$', '', title)
52                 return title
53
54             # 尝试从meta标签获取
55             meta_title = soup.find('meta', property='og:title')
56             if meta_title:
57                 return meta_title.get('content', '').strip()
58
59             return "无法获取标题"

```

```
59
60     except Exception as e:
61         return f"获取失败: {str(e)}"
62
63     def _is_bilibili_url(url):
64         """
65         检查是否为B站链接
66         """
67         if not isinstance(url, str):
68             return False
69         bilibili_patterns = [
70             r'https?://www\.bilibili\.com/video/',
71             r'https?://b23\.tv/',
72             r'https?://bilibili\.com/video/',
73             r'https?://m\.bilibili\.com/video/'
74         ]
75         return any(re.match(pattern, url.strip()) for pattern in
bilibili_patterns)
76
77     try:
78         # 加载Excel文件
79         workbook = openpyxl.load_workbook(excel_file)
80
81         # 检查是否存在"分类"工作表
82         if "分类" not in workbook.sheetnames:
83             raise ValueError("Excel文件中未找到名为'分类'的工作表")
84
85         worksheet = workbook["分类"]
86
87         # 查找"对标链接"和"对标标题"列
88         header_row = 1
89         link_col = None
90         title_col = None
91
92         for col in range(1, worksheet.max_column + 1):
93             cell_value = worksheet.cell(row=header_row, column=col).value
94             if cell_value == "对标链接":
95                 link_col = col
96             elif cell_value == "对标标题":
97                 title_col = col
98
99         if link_col is None:
100             raise ValueError("未找到'对标链接'列")
101         if title_col is None:
102             raise ValueError("未找到'对标标题'列")
103
104         processed_count = 0
```

```

105
106     # 遍历所有行，处理B站链接
107     for row in range(header_row + 1, worksheet.max_row + 1):
108         link_cell = worksheet.cell(row=row, column=link_col)
109         title_cell = worksheet.cell(row=row, column=title_col)
110
111         # 如果链接单元格为空或标题已存在，跳过
112         if not link_cell.value or title_cell.value:
113             continue
114
115         url = str(link_cell.value).strip()
116
117         # 检查是否为B站链接
118         if _is_bilibili_url(url):
119             print(f"正在处理第{row}行的B站链接: {url}")
120
121             # 获取视频标题
122             title = _get_bilibili_title(url)
123
124             # 写入标题
125             title_cell.value = title
126             processed_count += 1
127
128             print(f"获取到标题: {title}")
129
130             # 添加延时，避免请求过于频繁
131             time.sleep(1)
132
133         # 保存修改到原文件
134         workbook.save(excel_file)
135         workbook.close()
136
137         print(f"处理完成！成功处理了{processed_count}个B站链接")
138
139     except Exception as e:
140         raise Exception(f"处理Excel文件时出错: {str(e)}")
141

```

批量填充分类数据-B站评论爬取并按顺序写入Excel

```

1  # 使用此指令前，请确保安装必要的Python库，例如使用以下命令安装：
2  # pip install requests pandas openpyxl tqdm
3
4  import requests
5  import json
6  import re

```

```

7  import hashlib
8  import urllib
9  import time
10 from urllib.parse import quote, urlparse, parse_qs
11 import os
12 import pandas as pd
13 from tqdm import tqdm
14 import openpyxl
15
16 from typing import *
17 try:
18     from xbot.app.logging import trace as print
19 except:
20     from xbot import print
21
22 def crawl_bilibili_comments_and_write_to_category_sheets_ordered(excel_file,
23 cookie_content="", mode=3):
24     """
25     title: B站评论爬取并按顺序写入Excel
26     description: 从Excel文件"分类"sheet的 % 对标链接 % 列读取B站链接, 获取评论区商品
27     链接, 并**按原始评论顺序**写入对应的sheet中, 数据从第二行开始写入。
28     inputs:
29         - excel_file (file): Excel文件路径, eg: "data.xlsx"
30         - cookie_content (str): B站Cookie内容, 用于访问API, eg: "buvid3=xxx;
31         _uuid=xxx"
32         - mode (int): 评论爬取模式, 2为最新评论, 3为热门评论, eg: 3
33     outputs: None
34     """
35
36     def _get_header(cookie_content):
37         """获取B站请求头"""
38         if cookie_content.strip():
39             header = {
40                 "Cookie": cookie_content.strip(),
41                 "User-Agent": 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
42                 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36
43                 Edg/134.0.0.0'
44             }
45         else:
46             header = {
47                 "User-Agent": 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
48                 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36
49                 Edg/134.0.0.0',
50                 "Referer": "https://www.bilibili.com/",
51                 "Accept": "application/json, text/plain, */*",
52                 "Accept-Language": "zh-CN,zh;q=0.9,en;q=0.8",
53                 "Accept-Encoding": "gzip, deflate, br",

```

```

47         "Connection": "keep-alive"
48     }
49     return header
50
51     def _get_video_info(bv, cookie_content):
52         """通过BV号获取视频的oid和标题"""
53         headers = _get_header(cookie_content)
54         resp = requests.get(f"https://www.bilibili.com/video/{bv}/?p=1",
55                             headers=headers, timeout=10)
56
57         if resp.status_code != 200:
58             raise Exception(f"获取视频信息失败, 状态码: {resp.status_code}")
59
60         # 提取视频oid
61         obj = re.compile(f'"aid":(?P<id>.*?), "bvid": "{bv}"')
62         match = obj.search(resp.text)
63         if not match:
64             raise Exception("无法获取视频oid")
65         oid = match.group('id')
66
67         return oid
68
69     def _md5_encrypt(code):
70         """MD5加密"""
71         MD5 = hashlib.md5()
72         MD5.update(code.encode('utf-8'))
73         return MD5.hexdigest()
74
75     def _generate_api_url(oid, mode):
76         """生成B站评论API的URL"""
77         plat = 1
78         type = 1
79         web_location = 1315875
80         wts = int(time.time())
81
82         pagination_str = '{"offset":""}'
83         code = f"mode={mode}&oid={oid}&pagination_str=
84 {urllib.parse.quote(pagination_str)}&plat={plat}&seek_rpid=&type=
85 {type}&web_location={web_location}&wts={wts}" +
86 'ea1db124af3c7062474693fa704f4ff8'
87
88         w_rid = _md5_encrypt(code)
89         url = f"https://api.bilibili.com/x/v2/reply/wbi/main?oid={oid}&type=
90 {type}&mode={mode}&pagination_str={urllib.parse.quote(pagination_str,
91 safe=':')}&plat=1&seek_rpid=&web_location=1315875&w_rid={w_rid}&wts={wts}"
92
93         return url

```

```

88     def _get_comments_data(bv, mode, cookie_content):
89         """获取B站评论数据"""
90         oid = _get_video_info(bv, cookie_content)
91         url = _generate_api_url(oid, mode)
92
93         headers = _get_header(cookie_content)
94         response = requests.get(url, headers=headers, timeout=10)
95
96         if response.status_code != 200:
97             raise Exception(f"获取评论失败, 状态码: {response.status_code}")
98
99         return response.json()
100
101     def _extract_messages_from_data_ordered(data):
102         """从评论数据中按顺序提取所有message内容"""
103         messages = []
104
105         def find_all_messages_ordered(obj, path=""):
106             if isinstance(obj, dict):
107                 # 按键的顺序遍历, 保持原始结构顺序
108                 for key, value in obj.items():
109                     current_path = f"{path}.{key}" if path else key
110                     if key == "message" and isinstance(value, str):
111                         messages.append(value)
112                     else:
113                         find_all_messages_ordered(value, current_path)
114             elif isinstance(obj, list):
115                 # 按索引顺序遍历列表
116                 for i, item in enumerate(obj):
117                     current_path = f"{path}[{i}]"
118                     find_all_messages_ordered(item, current_path)
119
120         find_all_messages_ordered(data)
121         return messages
122
123     def _expand_b23_link(short_url, cookie_content):
124         """展开B站短链接"""
125         try:
126             b23_pattern = r'https://b23\.tv/[a-zA-Z0-9-]+'
127             match = re.search(b23_pattern, str(short_url))
128
129             if not match:
130                 return None
131
132             valid_url = match.group(0)
133             headers = _get_header(cookie_content)

```

```
134         headers['Accept'] =
135         'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8'
136
137         try:
138             response = requests.get(valid_url, headers=headers,
139 allow_redirects=True, timeout=10)
140             if response.status_code == 200 and response.url != valid_url:
141                 return response.url
142         except:
143             pass
144
145         return None
146     except:
147         return None
148
149 def _analyze_link_type(url):
150     """分析链接类型"""
151     if not url:
152         return "未知类型"
153
154     parsed_url = urlparse(url)
155     domain = parsed_url.netloc.lower()
156     path = parsed_url.path.lower()
157     query = parsed_url.query.lower()
158     full_url = url.lower()
159
160     taobao_keywords = [
161         'taobao.com', 'tmall.com', 'alimama.com', 'alibaba.com',
162         'taobao', 'tmall', 'alimama', 'alibaba', 'mall',
163         'item.taobao', 'detail.tmall', 'click.taobao'
164     ]
165
166     jd_keywords = [
167         'jd.com', 'jd.cn', 'jd', 'jingdong', 'jdunion',
168         'item.jd', 'product.jd', 'click.jd'
169     ]
170
171     for keyword in taobao_keywords:
172         if keyword in domain or keyword in path or keyword in query or
173 keyword in full_url:
174             return "淘宝联盟"
175
176     for keyword in jd_keywords:
177         if keyword in domain or keyword in path or keyword in query or
178 keyword in full_url:
179             return "京东联盟"
```

```

177         return "其他链接"
178
179     def _process_links_ordered(messages, cookie_content):
180         """按顺序处理评论中的链接，保持原始评论顺序"""
181         processed_links = []
182         seen_links = set() # 用于去重，但不影响顺序
183
184         print(f"开始按顺序分析 {len(messages)} 条评论中的链接...")
185
186         # 按评论顺序处理每条消息
187         for msg_index, message in enumerate(messages):
188             print(f"处理第 {msg_index + 1} 条评论: {message[:30]}...")
189
190             # 从当前消息中提取所有链接
191             current_links = []
192
193             # 查找所有b23.tv链接
194             b23_links = re.findall(r'https://b23\.tv/[a-zA-Z0-9-]+', message)
195             current_links.extend(b23_links)
196
197             # 查找其他常见链接
198             other_links = re.findall(r'https?:\/\/[^\s]+', message)
199             current_links.extend(other_links)
200
201             # 按在评论中出现的顺序处理链接
202             for link in current_links:
203                 if link not in seen_links: # 去重但保持顺序
204                     seen_links.add(link)
205
206                     print(f"  处理链接: {link[:50]}...")
207
208                     if 'b23.tv' in link:
209                         print(f"    展开B站短链接...")
210                         real_url = _expand_b23_link(link, cookie_content)
211                         if real_url:
212                             print(f"    展开成功: {real_url[:50]}...")
213                         else:
214                             print(f"    展开失败，使用原链接")
215                             real_url = link
216                     else:
217                         real_url = link
218
219                     if real_url:
220                         link_type = _analyze_link_type(real_url)
221                         processed_links.append({
222                             '联盟类型': link_type,
223                             '商品链接': real_url,

```



```

224         '佣金': '',
225         '佣金比例': '',
226         '推广链接': '',
227         '商品名称': '',
228         '到手价': '',
229         '好评数': ''
230     })
231     print(f"    ✅ 添加链接: [{link_type}]
{real_url[:50]}...")
232
233     print(f"✅ 按顺序处理完成, 共找到 {len(processed_links)} 个唯一商品链接")
234     return processed_links
235
236     def _crawl_bilibili_comments_and_analyze_links_ordered(bv_number,
cookie_content, mode):
237         """根据BV号爬取B站评论并按顺序分析链接"""
238         try:
239             print("开始获取B站评论数据...")
240             comment_data = _get_comments_data(bv_number.strip(), mode,
cookie_content)
241
242             print("按顺序提取评论内容...")
243             messages = _extract_messages_from_data_ordered(comment_data)
244
245             if not messages:
246                 print("未找到任何评论内容")
247                 return []
248
249             print(f"找到 {len(messages)} 条评论, 按顺序分析链接...")
250             processed_links = _process_links_ordered(messages, cookie_content)
251
252             print(f"✅ 分析完成, 按顺序找到 {len(processed_links)} 条商品链接")
253             return processed_links
254
255         except Exception as e:
256             print(f"处理失败: {str(e)}")
257             return []
258
259     def _extract_bv_number(url):
260         """从B站链接中提取BV号"""
261         try:
262             bv_pattern = r'(BV[a-zA-Z0-9]{10})'
263             match = re.search(bv_pattern, url)
264             if match:
265                 return match.group(1)
266             return None
267         except:

```

```

268         return None
269
270     def _is_bilibili_url(url):
271         """检查是否为B站链接"""
272         if not isinstance(url, str):
273             return False
274         bilibili_patterns = [
275             r'https?://www\.bilibili\.com/video/',
276             r'https?://b23\.tv/',
277             r'https?://bilibili\.com/video/',
278             r'https?://m\.bilibili\.com/video/'
279         ]
280         return any(re.match(pattern, url.strip()) for pattern in
bilibili_patterns)
281
282     def _check_excel_file_access(excel_file):
283         """检查Excel文件是否可以访问"""
284         try:
285             if not os.path.exists(excel_file):
286                 raise Exception(f"文件不存在: {excel_file}")
287
288             with open(excel_file, 'r+b') as f:
289                 pass
290
291             print(f"✅ Excel文件访问正常: {excel_file}")
292             return True
293
294         except PermissionError:
295             raise Exception(f"文件被占用或权限不足，请关闭Excel后重试:
{excel_file}")
296         except Exception as e:
297             raise Exception(f"文件访问错误: {str(e)}")
298
299     def _write_links_to_category_sheet_from_row2(excel_file, category_name,
processed_links):
300         """将商品链接数据从第二行开始按顺序写入到对应分类的sheet中"""
301         try:
302             if not processed_links:
303                 print(f"分类 '{category_name}' 没有商品链接需要写入")
304                 return False
305
306             print(f"准备从第二行开始按顺序写入 {len(processed_links)} 个商品链接到
'{category_name}' sheet")
307
308             workbook = openpyxl.load_workbook(excel_file)
309
310             if category_name not in workbook.sheetnames:

```

```
311         print(f"❌ 未找到分类 '{category_name}' 对应的sheet")
312         workbook.close()
313         return False
314
315     category_sheet = workbook[category_name]
316     print(f"✅ 找到目标sheet: '{category_name}'")
317
318     # 设置表头
319     headers = ['联盟类型', '商品链接', '佣金', '佣金比例', '推广链接', '商品
名称', '到手价', '好评数']
320
321     print("设置表头 (第一行) ...")
322     for col, header in enumerate(headers, 1):
323         category_sheet.cell(row=1, column=col, value=header)
324
325     # 清除第二行及以后的所有数据
326     print("清除原有数据...")
327     if category_sheet.max_row > 1:
328         for row in range(2, category_sheet.max_row + 1):
329             for col in range(1, 9):
330                 category_sheet.cell(row=row, column=col, value=None)
331
332     # 从第二行开始按顺序写入数据
333     print("从第二行开始按评论顺序写入数据...")
334     for i, link_data in enumerate(processed_links):
335         row_num = 2 + i
336
337         try:
338             category_sheet.cell(row=row_num, column=1).value =
str(link_data['联盟类型'])
339             category_sheet.cell(row=row_num, column=2).value =
str(link_data['商品链接'])
340             category_sheet.cell(row=row_num, column=3).value =
str(link_data['佣金'])
341             category_sheet.cell(row=row_num, column=4).value =
str(link_data['佣金比例'])
342             category_sheet.cell(row=row_num, column=5).value =
str(link_data['推广链接'])
343             category_sheet.cell(row=row_num, column=6).value =
str(link_data['商品名称'])
344             category_sheet.cell(row=row_num, column=7).value =
str(link_data['到手价'])
345             category_sheet.cell(row=row_num, column=8).value =
str(link_data['好评数'])
346
347             print(f"✅ 第{i+1}行写入成功 (第{row_num}行):
[{link_data['联盟类型']} {link_data['商品链接'][:50]}...]")
```

```

348
349         except Exception as write_error:
350             print(f"✗ 第{i+1}行写入失败: {str(write_error)}")
351             continue
352
353     print("正在保存文件...")
354     try:
355         workbook.save(excel_file)
356         workbook.close()
357         print(f"✅ 成功按顺序保存到 '{category_name}' sheet，数据从第2行开
始")
358         return True
359     except Exception as save_error:
360         print(f"✗ 保存文件失败: {str(save_error)}")
361         workbook.close()
362         return False
363
364     except Exception as e:
365         print(f"✗ 写入操作失败: {str(e)}")
366         try:
367             workbook.close()
368         except:
369             pass
370         return False
371
372     try:
373         print("检查Excel文件访问权限...")
374         _check_excel_file_access(excel_file)
375
376         workbook = openpyxl.load_workbook(excel_file)
377         print(f"✅ 成功加载Excel文件: {excel_file}")
378         print(f"可用的sheet: {workbook.sheetnames}")
379
380         if "分类" not in workbook.sheetnames:
381             workbook.close()
382             raise ValueError("Excel文件中未找到名为'分类'的工作表")
383
384         worksheet = workbook["分类"]
385
386         # 查找所需列
387         header_row = 1
388         link_col = None
389         category_col = None
390
391         print("正在查找列标题...")
392         for col in range(1, worksheet.max_column + 1):
393             cell_value = worksheet.cell(row=header_row, column=col).value

```

```
394         print(f"第{col}列: {cell_value}")
395         if cell_value == "对标链接":
396             link_col = col
397         elif cell_value == "分类":
398             category_col = col
399
400     if link_col is None:
401         workbook.close()
402         raise ValueError("未找到'对标链接'列")
403     if category_col is None:
404         workbook.close()
405         raise ValueError("未找到'分类'列")
406
407     print(f"找到列位置 - 对标链接:{link_col}, 分类:{category_col}")
408
409     workbook.close()
410     processed_count = 0
411
412     # 重新打开文件读取数据
413     workbook = openpyxl.load_workbook(excel_file)
414     worksheet = workbook["分类"]
415
416     # 遍历所有行, 处理B站链接
417     for row in range(header_row + 1, worksheet.max_row + 1):
418         link_cell = worksheet.cell(row=row, column=link_col)
419         category_cell = worksheet.cell(row=row, column=category_col)
420
421         if not link_cell.value:
422             continue
423
424         url = str(link_cell.value).strip()
425         category_name = str(category_cell.value).strip() if
category_cell.value else ""
426
427         if _is_bilibili_url(url):
428             print(f"\n=== 处理第{row}行 ===")
429             print(f"B站链接: {url}")
430             print(f"对应分类sheet: {category_name}")
431
432             bv_number = _extract_bv_number(url)
433             if bv_number:
434                 print(f"提取到BV号: {bv_number}")
435
436             workbook.close()
437
438     # 按顺序获取评论区商品链接
```

```

439         processed_links =
_crawl_bilibili_comments_and_analyze_links_ordered(bv_number, cookie_content,
mode)
440
441         if category_name and processed_links:
442             success =
_write_links_to_category_sheet_from_row2(excel_file, category_name,
processed_links)
443
444             if success:
445                 print(f"✅ 成功按顺序写入商品链接到 '{category_name}'
sheet")
446             else:
447                 print(f"❌ 写入商品链接到 '{category_name}' sheet 失
败")
448
449             elif not category_name:
450                 print("⚠️ 未指定分类, 跳过商品链接写入")
451             else:
452                 print(f"ℹ️ 没有商品链接需要写入")
453
454             workbook = openpyxl.load_workbook(excel_file)
455             worksheet = workbook["分类"]
456
457             else:
458                 print(f"❌ 无法提取BV号")
459
460             processed_count += 1
461
462             print("等待2秒...")
463             time.sleep(2)
464
465             workbook.close()
466
467             print(f"\n🎉 处理完成! 成功处理了{processed_count}个B站链接, 所有数据按评论顺
序从第二行开始写入")
468
469         except Exception as e:
470             print(f"❌ 处理Excel文件时出错: {str(e)}")
471             try:
472                 workbook.close()
473             except:
474                 pass
475             raise Exception(f"处理Excel文件时出错: {str(e)}")

```

批量填充分类数据-提取Excel文件中京东联盟商品链接

1 # 使用此指令前, 请确保安装必要的Python库, 例如使用以下命令安装:

```

2  # pip install pandas openpyxl
3
4  import pandas as pd
5  import os
6
7  from typing import *
8  try:
9      from xbot.app.logging import trace as print
10 except:
11     from xbot import print
12
13 def extract_jd_links_from_excel(excel_file_path):
14     """
15     title: 提取Excel文件中京东联盟商品链接
16     description: 从指定的Excel文件中提取除了"分类"sheet外其他所有sheet中联盟类型为"京
17 东联盟"的商品链接，输出时包含% sheet名称 %和对应的% 链接列表 %。
18     inputs:
19         - excel_file_path (file): Excel文件路径，eg: "C:/data/products.xlsx"
20     outputs:
21         - result (list): 包含sheet名称和链接的字典列表，eg: "[{'sheet_name':
22 'Sheet1', 'links': ['http://example1.com', 'http://example2.com']}]"
23     """
24
25     # 1. 检查输入有效性
26     if not os.path.exists(excel_file_path):
27         raise FileNotFoundError(f"Excel文件不存在: {excel_file_path}")
28
29     if not excel_file_path.lower().endswith(('.xlsx', '.xls')):
30         raise ValueError("输入文件必须是Excel格式(.xlsx或.xls)")
31
32     def _extract_links_from_sheet(sheet_data, sheet_name):
33         """
34         从单个sheet中提取京东联盟链接
35         """
36         try:
37             # 检查必要的列是否存在
38             if '联盟类型' not in sheet_data.columns:
39                 print(f"警告: Sheet '{sheet_name}' 中未找到'联盟类型'列，跳过处理")
40                 return []
41
42             if '商品链接' not in sheet_data.columns:
43                 print(f"警告: Sheet '{sheet_name}' 中未找到'商品链接'列，跳过处理")
44                 return []
45
46             # 筛选联盟类型为"京东联盟"的行
47             jd_rows = sheet_data[sheet_data['联盟类型'] == '京东联盟']

```

```

47         # 提取商品链接列，去除空值
48         links = jd_rows['商品链接'].dropna().tolist()
49
50         return links
51
52     except Exception as e:
53         print(f"处理Sheet '{sheet_name}' 时出错: {str(e)}")
54         return []
55
56 # 2. 读取Excel文件并处理
57 try:
58     # 读取所有sheet名称
59     excel_file = pd.ExcelFile(excel_file_path)
60     sheet_names = excel_file.sheet_names
61
62     result = []
63
64     # 遍历所有sheet (除了"分类")
65     for sheet_name in sheet_names:
66         if sheet_name == '分类':
67             continue
68
69         try:
70             # 读取sheet数据
71             sheet_data = pd.read_excel(excel_file_path,
sheet_name=sheet_name)
72
73             # 提取京东联盟链接
74             links = _extract_links_from_sheet(sheet_data, sheet_name)
75
76             # 添加到结果中
77             if links: # 只有当有链接时才添加
78                 result.append({
79                     'sheet_name': sheet_name,
80                     'links': links
81                 })
82
83         except Exception as e:
84             print(f"读取Sheet '{sheet_name}' 时出错: {str(e)}")
85             continue
86
87     excel_file.close()
88
89     if not result:
90         print("未找到任何京东联盟链接")
91
92     return result

```



```
93
94     except Exception as e:
95         raise Exception(f"处理Excel文件时出错: {str(e)}")
96
```

批量填充分类数据-使用CSV模块转换CSV文件为Excel文件

```
1  # 使用此指令前, 请确保安装必要的Python库, 例如使用以下命令安装:
2  # pip install openpyxl
3
4  import csv
5  import os
6  from openpyxl import Workbook
7
8  from typing import *
9  try:
10     from xbot.app.logging import trace as print
11 except:
12     from xbot import print
13
14 def csv_to_excel_with_csv_module(csv_file_path, excel_file_path):
15     """
16     title: 使用CSV模块转换CSV文件为Excel文件
17     description: 使用Python内置的csv模块读取CSV文件 %csv_file_path%, 并转换为Excel
18     文件保存到 %excel_file_path%.
19     inputs:
20         - csv_file_path (file): CSV文件路径, eg: "data.csv"
21         - excel_file_path (str): Excel文件保存路径, eg: "output.xlsx"
22     outputs:
23         - result (str): 转换结果信息, eg: "转换成功"
24     """
25
26     # 检查输入文件是否存在
27     if not os.path.exists(csv_file_path):
28         raise FileNotFoundError(f"CSV文件不存在: {csv_file_path}")
29
30     # 检查输入文件是否为CSV格式
31     if not csv_file_path.lower().endswith('.csv'):
32         raise ValueError("输入文件必须是CSV格式")
33
34     # 检查输出文件路径是否为Excel格式
35     if not excel_file_path.lower().endswith(('.xlsx', '.xls')):
36         raise ValueError("输出文件必须是Excel格式(.xlsx或.xls)")
37
38     # 确保输出目录存在
39     output_dir = os.path.dirname(excel_file_path)
```

```

39     if output_dir and not os.path.exists(output_dir):
40         os.makedirs(output_dir)
41
42     def _detect_delimiter(file_path, encoding):
43         """检测CSV文件分隔符"""
44         try:
45             with open(file_path, 'r', encoding=encoding, newline='') as
csvfile:
46                 sample = csvfile.read(1024)
47                 if not sample.strip():
48                     return ',' # 空文件使用默认逗号
49
50                 sniffer = csv.Sniffer()
51                 try:
52                     delimiter = sniffer.sniff(sample).delimiter
53                     return delimiter
54                 except csv.Error:
55                     # 自动检测失败, 尝试常见分隔符
56                     common_delimiters = [',', ';', '\t', '|']
57                     for delim in common_delimiters:
58                         if delim in sample:
59                             return delim
60                     return ',' # 默认使用逗号
61         except Exception:
62             return ','
63
64     def _read_csv_data(file_path):
65         """读取CSV文件数据"""
66         data = []
67         encodings = ['utf-8', 'gbk', 'utf-8-sig', 'gb2312']
68
69         for encoding in encodings:
70             try:
71                 # 检测分隔符
72                 delimiter = _detect_delimiter(file_path, encoding)
73
74                 with open(file_path, 'r', encoding=encoding, newline='') as
csvfile:
75                     reader = csv.reader(csvfile, delimiter=delimiter)
76                     data = list(reader)
77                     break
78             except UnicodeDecodeError:
79                 continue
80         except Exception as e:
81             if encoding == encodings[-1]: # 最后一个编码也失败
82                 raise ValueError(f"读取CSV文件失败: {str(e)}")
83

```

```

84         if not data:
85             raise ValueError("CSV文件为空或无法读取")
86
87         return data
88
89     def _write_excel_data(data, file_path):
90         """写入Excel文件"""
91         wb = Workbook()
92         ws = wb.active
93         ws.title = "Sheet1"
94
95         # 写入数据
96         for row_index, row_data in enumerate(data, 1):
97             for col_index, cell_value in enumerate(row_data, 1):
98                 ws.cell(row=row_index, column=col_index, value=cell_value)
99
100        wb.save(file_path)
101
102    try:
103        # 读取CSV数据
104        csv_data = _read_csv_data(csv_file_path)
105
106        # 写入Excel文件
107        _write_excel_data(csv_data, excel_file_path)
108
109    except Exception as e:
110        raise ValueError(f"转换过程中发生错误: {str(e)}")
111
112    return f"转换成功! 已将 {os.path.basename(csv_file_path)} 转换为
113    {os.path.basename(excel_file_path)}, 共 {len(csv_data)} 行数据"

```

批量填充分类数据-替换Excel工作表内容

```

1  # 使用此指令前, 请确保安装必要的Python库, 例如使用以下命令安装:
2  # pip install openpyxl
3
4  import openpyxl
5  from openpyxl.utils.dataframe import dataframe_to_rows
6  import pandas as pd
7
8  from typing import *
9  try:
10     from xbot.app.logging import trace as print
11 except:
12     from xbot import print

```

```

13
14 def replace_sheet_content(target_excel_path, sheet_name, source_excel_path):
15     """
16     title: 替换Excel工作表内容
17     description: 读取 % source_excel_path % 文件的内容, 清空 % target_excel_path
% 中指定 % sheet_name % 的数据, 然后将源文件内容写入目标工作表, 保持原有格式不变。
18     inputs:
19         - target_excel_path (file): 目标Excel文件路径, eg: "C:/target.xlsx"
20         - sheet_name (str): 目标工作表名称, eg: "Sheet1"
21         - source_excel_path (file): 待写入的Excel文件路径, eg: "C:/source.xlsx"
22     outputs:
23         - result (str): 操作结果信息, eg: "成功替换工作表内容"
24     """
25
26     # 检查输入有效性
27     if not target_excel_path or not sheet_name or not source_excel_path:
28         raise ValueError("所有参数都不能为空")
29
30     def _read_source_data(source_path):
31         """读取源Excel文件的第一个工作表数据"""
32         try:
33             df = pd.read_excel(source_path, engine='openpyxl')
34             return df
35         except Exception as e:
36             raise ValueError(f"读取源Excel文件失败: {str(e)}")
37
38     def _clear_sheet_content(worksheet):
39         """清空工作表内容但保持格式"""
40         max_row = worksheet.max_row
41         max_col = worksheet.max_column
42
43         # 清空所有单元格的值
44         for row in range(1, max_row + 1):
45             for col in range(1, max_col + 1):
46                 cell = worksheet.cell(row=row, column=col)
47                 cell.value = None
48
49     def _write_data_to_sheet(worksheet, data_df):
50         """将数据写入工作表"""
51         # 写入列头
52         for col_idx, column_name in enumerate(data_df.columns, 1):
53             worksheet.cell(row=1, column=col_idx, value=column_name)
54
55         # 写入数据
56         for row_idx, row_data in enumerate(data_df.values, 2):
57             for col_idx, value in enumerate(row_data, 1):
58                 worksheet.cell(row=row_idx, column=col_idx, value=value)

```

```

59
60     try:
61         # 读取源数据
62         source_data = _read_source_data(source_excel_path)
63
64         # 打开目标Excel文件
65         workbook = openpyxl.load_workbook(target_excel_path)
66
67         # 检查工作表是否存在
68         if sheet_name not in workbook.sheetnames:
69             raise ValueError(f"工作表 '{sheet_name}' 在目标文件中不存在")
70
71         # 获取目标工作表
72         target_sheet = workbook[sheet_name]
73
74         # 清空工作表内容
75         _clear_sheet_content(target_sheet)
76
77         # 写入新数据
78         _write_data_to_sheet(target_sheet, source_data)
79
80         # 保存文件
81         workbook.save(target_excel_path)
82         workbook.close()
83
84         return f"成功替换工作表 '{sheet_name}' 的内容，共写入 {len(source_data)} 行
数据"
85
86     except Exception as e:
87         raise Exception(f"操作失败: {str(e)}")
88

```

导出详情图txt-检查并创建指定子文件夹

```

1  import os
2
3  from typing import *
4  try:
5      from xbot.app.logging import trace as print
6  except:
7      from xbot import print
8
9  def create_required_subfolders(input_folder):
10     """
11     title: 检查并创建指定子文件夹

```

```

12     description: 在指定的文件夹 % input_folder % 下检查是否存在"参数图片"、"参数
13     txt"、"详情图片"、"详情图txt"这四个子文件夹，如果不存在则自动创建。如果指定的主文件夹不存
14     在，也会自动创建。
15     inputs:
16         - input_folder (folder): 要检查的文件夹路径，eg: "C:/Users/Desktop/项目文件
17     夹"
18     outputs:
19         - None
20     """
21
22     # 如果输入文件夹不存在，则创建它
23     if not os.path.exists(input_folder):
24         os.makedirs(input_folder)
25
26     # 检查是否为文件夹类型
27     if not os.path.isdir(input_folder):
28         raise ValueError(f"指定的路径不是文件夹: {input_folder}")
29
30     # 定义需要检查和创建的子文件夹列表
31     required_subfolders = ["参数图片", "参数txt", "详情图片", "详情图txt"]
32
33     def _create_subfolder_if_not_exists(parent_folder, subfolder_name):
34         """
35         检查并创建子文件夹
36         """
37         subfolder_path = os.path.join(parent_folder, subfolder_name)
38         if not os.path.exists(subfolder_path):
39             os.makedirs(subfolder_path)
40
41     # 遍历所有需要的子文件夹并创建
42     for subfolder in required_subfolders:
43         _create_subfolder_if_not_exists(input_folder, subfolder)
44 
```

#### 导出详情图txt-提取Excel中商品信息

```

1  # 使用此指令前，请确保安装必要的Python库，例如使用以下命令安装：
2  # pip install pandas openpyxl
3
4  import pandas as pd
5  import os
6
7  from typing import *
8  try:
9      from xbot.app.logging import trace as print
10 except:

```

```
11     from xbot import print
12
13     def extract_product_info_from_excel(folder_path, file_name,
14     product_name_column, link_column):
15         """
16         title: 提取Excel中商品信息
17         description: 读取Excel文件中除了"分类"之外的所有sheet，提取每个sheet中的 %
18         product_name_column % 列和 % link_column % 列数据，商品名称格式为"sheet名称_商品名
19         称"。
20         inputs:
21             - folder_path (folder): Excel文件所在文件夹路径，eg: "C:/data/"
22             - file_name (str): Excel文件名，eg: "products.xlsx"
23             - product_name_column (str): 商品名称列名，eg: "商品名称"
24             - link_column (str): 联盟推广链接列名，eg: "联盟推广链接"
25         outputs:
26             - product_data (list): 包含商品名称和联盟推广链接的字典列表，eg: "[{'商品名
27             称': 'sheet1_苹果', '联盟推广链接': 'http://example.com'}]"
28         """
29
30         # 拼接完整的Excel文件路径
31         excel_path = os.path.join(folder_path, file_name)
32
33         # 检查文件夹是否存在
34         if not os.path.exists(folder_path):
35             raise FileNotFoundError(f"文件夹不存在: {folder_path}")
36
37         # 检查Excel文件是否存在
38         if not os.path.exists(excel_path):
39             raise FileNotFoundError(f"Excel文件不存在: {excel_path}")
40
41         # 检查列名参数是否为空
42         if not product_name_column or not link_column:
43             raise ValueError("商品名称列名和链接列名不能为空")
44
45     def _read_excel_sheets(excel_path):
46         """
47         读取Excel文件的所有sheet名称
48         """
49         try:
50             excel_file = pd.ExcelFile(excel_path)
51             return excel_file.sheet_names
52         except Exception as e:
53             raise Exception(f"读取Excel文件失败: {str(e)}")
54
55     def _extract_product_data_from_sheet(excel_path, sheet_name, product_col,
56     link_col):
57         """
```

```

53     从指定sheet中提取商品名称和联盟推广链接
54     """
55     try:
56         df = pd.read_excel(excel_path, sheet_name=sheet_name)
57
58         # 检查必需的列是否存在
59         required_columns = [product_col, link_col]
60         missing_columns = [col for col in required_columns if col not in
df.columns]
61
62         if missing_columns:
63             print(f"警告: sheet '{sheet_name}' 中缺少列: {missing_columns}")
64             return []
65
66         # 提取数据, 过滤掉空值
67         sheet_data = []
68         for index, row in df.iterrows():
69             if pd.isna(row[product_col]) and pd.isna(row[link_col]):
70                 # 将商品名称格式化为"sheet名称_商品名称"
71                 formatted_product_name = f"
{sheet_name}_{str(row[product_col]).strip()}"
72                 sheet_data.append({
73                     '商品名称': formatted_product_name,
74                     '联盟推广链接': str(row[link_col]).strip()
75                 })
76
77         return sheet_data
78
79     except Exception as e:
80         print(f"警告: 读取sheet '{sheet_name}' 时出错: {str(e)}")
81         return []
82
83     # 获取所有sheet名称
84     all_sheets = _read_excel_sheets(excel_path)
85
86     # 过滤掉"分类"sheet
87     target_sheets = [sheet for sheet in all_sheets if sheet != "分类"]
88
89     if not target_sheets:
90         raise ValueError("除了'分类'外没有找到其他sheet")
91
92     # 提取所有目标sheet的商品数据
93     all_product_data = []
94     for sheet_name in target_sheets:
95         sheet_data = _extract_product_data_from_sheet(excel_path, sheet_name,
product_name_column, link_column)
96         all_product_data.extend(sheet_data)

```



```

97
98     if not all_product_data:
99         raise ValueError("未找到任何有效的商品数据")
100
101     print(f"成功提取了 {len(all_product_data)} 条商品信息, 来自
102         {len(target_sheets)} 个sheet")
103
104     return all_product_data

```

导出详情图txt-根据ZIP文件过滤商品列表并清理禁用字符

```

1  import os
2  import re
3
4  from typing import *
5  try:
6      from xbot.app.logging import trace as print
7  except:
8      from xbot import print
9
10 def filter_products_by_zip_files(folder_path, product_list):
11     """
12     title: 根据ZIP文件过滤商品列表并清理禁用字符
13     description: 查询指定文件夹 % folder_path % 下的ZIP文件名, 从商品字典列表 %
14     product_list % 中删除商品名称包含这些ZIP文件名的字典, 然后清理剩余商品名称中的Windows文
15     件名禁用字符, 返回处理后的商品列表 % filtered_list %。
16
17     inputs:
18         - folder_path (folder): 要查询的文件夹路径, eg: "C:/Users/Documents"
19         - product_list (list): 包含商品名称和链接的字典列表, eg: [{'商品名称': '学生
20     党显示器_SANC盛色23.8英寸', '联盟推广链接': 'https://u.jd.com/0ruxMM2'}]
21
22     outputs:
23         - filtered_list (list): 过滤并清理后的商品字典列表, eg: [{'商品名称': '其他
24     商品_清理后名称', '联盟推广链接': 'https://example.com'}]
25
26     """
27
28     # 检查文件夹路径是否存在
29     if not os.path.exists(folder_path):
30         raise ValueError(f"文件夹路径不存在: {folder_path}")
31
32     if not os.path.isdir(folder_path):
33         raise ValueError(f"指定路径不是文件夹: {folder_path}")
34
35     if not isinstance(product_list, list):
36         raise ValueError("product_list 必须是列表类型")
37
38

```

```

31     def _get_zip_files_without_extension(folder_path):
32         """
33         获取文件夹中所有ZIP文件名（不含后缀）
34         """
35         zip_files = []
36         try:
37             for filename in os.listdir(folder_path):
38                 file_path = os.path.join(folder_path, filename)
39                 if os.path.isfile(file_path) and
filename.lower().endswith('.zip'):
40                     name_without_extension = os.path.splitext(filename)[0]
41                     zip_files.append(name_without_extension)
42         except PermissionError:
43             raise PermissionError(f"没有权限访问文件夹: {folder_path}")
44         except Exception as e:
45             raise Exception(f"遍历文件夹时发生错误: {str(e)}")
46
47         return zip_files
48
49     def _filter_products_by_zip_names(zip_files, product_list):
50         """
51         根据ZIP文件名过滤商品列表
52         """
53         filtered_list = []
54
55         for product in product_list:
56             if not isinstance(product, dict) or '商品名称' not in product:
57                 raise ValueError("商品字典必须包含'商品名称'键")
58
59             product_name = product['商品名称']
60             should_keep = True
61
62             # 检查商品名称是否包含任何ZIP文件名
63             for zip_name in zip_files:
64                 if zip_name and zip_name in product_name:
65                     should_keep = False
66                     break
67
68             # 如果商品名称不包含任何ZIP文件名，则保留该商品
69             if should_keep:
70                 filtered_list.append(product)
71
72         return filtered_list
73
74     def _clean_invalid_characters(product_list):
75         """
76         清理商品名称中的Windows文件名禁用字符

```

```

77         """
78         # Windows文件名禁用字符: / \ : * ? " < > |
79         invalid_chars = r'[/\\:.*?"<>|]'
80         cleaned_list = []
81
82         for product in product_list:
83             # 创建产品副本以避免修改原始数据
84             cleaned_product = product.copy()
85
86             # 清理商品名称中的禁用字符, 替换为下划线
87             original_name = cleaned_product['商品名称']
88             cleaned_name = re.sub(invalid_chars, '_', original_name)
89
90             # 去除多余的下划线 (连续的下划线替换为单个)
91             cleaned_name = re.sub('_+', '_', cleaned_name)
92
93             # 去除首尾的下划线
94             cleaned_name = cleaned_name.strip('_')
95
96             cleaned_product['商品名称'] = cleaned_name
97             cleaned_list.append(cleaned_product)
98
99         return cleaned_list
100
101     # 获取ZIP文件名列表
102     zip_files = _get_zip_files_without_extension(folder_path)
103
104     # 根据ZIP文件名过滤商品列表
105     filtered_list = _filter_products_by_zip_names(zip_files, product_list)
106
107     # 清理商品名称中的禁用字符
108     cleaned_list = _clean_invalid_characters(filtered_list)
109
110     return cleaned_list
111

```

#### 导出详情图txt-批量解压子文件夹中的压缩包

```

1  # 使用此指令前, 请确保安装必要的Python库, 例如使用以下命令安装:
2  # pip install py7zr rarfile
3
4  import os
5  import zipfile
6  import tarfile
7  import gzip
8  import shutil

```

```
9  try:
10      import rarfile
11  except ImportError:
12      rarfile = None
13  try:
14      import py7zr
15  except ImportError:
16      py7zr = None
17
18  from typing import *
19  try:
20      from xbot.app.logging import trace as print
21  except:
22      from xbot import print
23
24  def extract_all_archives_in_folder(parent_folder_path, sub_folder_name):
25      """
26      title: 批量解压子文件夹中的压缩包
27      description: 在父文件夹 `parent_folder_path` 下找到子文件夹
28      `sub_folder_name`, 将其中的所有压缩包解压到各自的子文件夹中, 并删除原压缩包。
29      inputs:
30          - parent_folder_path (folder): 父文件夹路径, eg: "C:/Users/Desktop"
31          - sub_folder_name (str): 子文件夹名称, eg: "Downloads"
32      outputs:
33          - result (str): 处理结果信息, eg: "成功解压3个压缩包"
34      """
35
36      if not os.path.exists(parent_folder_path):
37          raise ValueError(f"父文件夹路径不存在: {parent_folder_path}")
38
39      if not os.path.isdir(parent_folder_path):
40          raise ValueError(f"父路径不是文件夹: {parent_folder_path}")
41
42      # 拼接完整的子文件夹路径
43      target_folder_path = os.path.join(parent_folder_path, sub_folder_name)
44
45      if not os.path.exists(target_folder_path):
46          raise ValueError(f"子文件夹不存在: {target_folder_path}")
47
48      if not os.path.isdir(target_folder_path):
49          raise ValueError(f"子路径不是文件夹: {target_folder_path}")
50
51      def _get_archive_files(folder_path):
52          """获取文件夹中的所有压缩包文件"""
53          archive_extensions = ['.zip', '.rar', '.7z', '.tar', '.tar.gz',
54                                '.tgz', '.gz']
55          archive_files = []
```

```

54
55     for filename in os.listdir(folder_path):
56         file_path = os.path.join(folder_path, filename)
57         if os.path.isfile(file_path):
58             for ext in archive_extensions:
59                 if filename.lower().endswith(ext):
60                     archive_files.append(file_path)
61                     break
62
63     return archive_files
64
65 def _extract_archive(archive_path, extract_folder):
66     """根据文件类型解压压缩包"""
67     filename = os.path.basename(archive_path)
68
69     try:
70         if archive_path.lower().endswith('.zip'):
71             with zipfile.ZipFile(archive_path, 'r') as zip_ref:
72                 zip_ref.extractall(extract_folder)
73
74         elif archive_path.lower().endswith('.rar'):
75             if rarfile is None:
76                 raise ImportError("需要安装rarfile库来处理RAR文件")
77             with rarfile.RarFile(archive_path, 'r') as rar_ref:
78                 rar_ref.extractall(extract_folder)
79
80         elif archive_path.lower().endswith('.7z'):
81             if py7zr is None:
82                 raise ImportError("需要安装py7zr库来处理7Z文件")
83             with py7zr.SevenZipFile(archive_path, mode='r') as z:
84                 z.extractall(extract_folder)
85
86         elif archive_path.lower().endswith(('tar', '.tar.gz', '.tgz')):
87             with tarfile.open(archive_path, 'r:*') as tar_ref:
88                 tar_ref.extractall(extract_folder)
89
90         elif archive_path.lower().endswith('.gz') and not
archive_path.lower().endswith('.tar.gz'):
91             # 处理单个.gz文件
92             with gzip.open(archive_path, 'rb') as f_in:
93                 output_filename = os.path.splitext(filename)[0]
94                 output_path = os.path.join(extract_folder, output_filename)
95                 with open(output_path, 'wb') as f_out:
96                     shutil.copyfileobj(f_in, f_out)
97
98     else:
99         raise ValueError(f"不支持的压缩格式: {filename}")

```

```
100
101         except Exception as e:
102             raise Exception(f"解压文件 {filename} 时出错: {str(e)}")
103
104     # 获取目标文件夹中的所有压缩包文件
105     archive_files = _get_archive_files(target_folder_path)
106
107     if not archive_files:
108         return f"子文件夹 '{sub_folder_name}' 中没有找到压缩包文件"
109
110     extracted_count = 0
111     failed_files = []
112
113     # 逐个解压压缩包
114     for archive_path in archive_files:
115         try:
116             filename = os.path.basename(archive_path)
117             # 创建解压目录, 去掉文件扩展名
118             archive_name = os.path.splitext(filename)[0]
119             # 处理.tar.gz等双重扩展名
120             if archive_name.endswith('.tar'):
121                 archive_name = os.path.splitext(archive_name)[0]
122
123             extract_folder = os.path.join(target_folder_path, archive_name)
124
125             # 如果目录已存在, 添加数字后缀
126             counter = 1
127             original_extract_folder = extract_folder
128             while os.path.exists(extract_folder):
129                 extract_folder = f"{original_extract_folder}_{counter}"
130                 counter += 1
131
132             # 创建解压目录
133             os.makedirs(extract_folder, exist_ok=True)
134
135             # 解压文件
136             _extract_archive(archive_path, extract_folder)
137
138             # 删除原压缩包
139             os.remove(archive_path)
140
141             extracted_count += 1
142
143         except Exception as e:
144             failed_files.append(f"{filename}: {str(e)}")
145         continue
146
```

```

147     # 生成结果信息
148     result_msg = f"在子文件夹 '{sub_folder_name}' 中成功解压{extracted_count}个压缩包"
149     if failed_files:
150         result_msg += f", 失败{len(failed_files)}个文件:\n" +
151         "\n".join(failed_files)
152     return result_msg
153

```

导出详情图txt-剪切子文件夹中的视频到新文件夹

```

1  import os
2  import shutil
3
4  from typing import *
5  try:
6      from xbot.app.logging import trace as print
7  except:
8      from xbot import print
9
10 def move_videos_from_subfolders(source_folder, target_folder):
11     """
12     title: 剪切子文件夹中的视频到新文件夹
13     description: 遍历指定文件夹 `source_folder` 下的所有子文件夹, 将每个子文件夹中的视频文件剪切到 `target_folder` 中, 并将视频文件重命名为其所在子文件夹的名字。
14     inputs:
15         - source_folder (folder): 源文件夹路径, eg: "C:/Videos"
16         - target_folder (folder): 目标文件夹路径, eg: "C:/CollectedVideos"
17     outputs:
18         - result (str): 操作结果统计信息, eg: "成功移动了5个视频文件"
19     """
20
21     # 检查源文件夹是否存在
22     if not os.path.exists(source_folder):
23         raise ValueError(f"源文件夹不存在: {source_folder}")
24
25     # 创建目标文件夹 (如果不存在)
26     os.makedirs(target_folder, exist_ok=True)
27
28     def _is_video_file(filename):
29         """判断文件是否为视频文件"""
30         video_extensions = {'.mp4', '.avi', '.mov', '.wmv', '.flv', '.mkv',
31                               '.webm', '.m4v', '.3gp', '.f4v'}
32         return os.path.splitext(filename.lower())[1] in video_extensions

```

```

33     def _get_unique_filename(target_path, base_name, extension):
34         """生成唯一的文件名，避免冲突"""
35         counter = 1
36         new_name = f"{base_name}{extension}"
37         while os.path.exists(os.path.join(target_path, new_name)):
38             new_name = f"{base_name}_{counter}{extension}"
39             counter += 1
40         return new_name
41
42     moved_count = 0
43
44     # 遍历源文件夹下的所有项目
45     for item in os.listdir(source_folder):
46         item_path = os.path.join(source_folder, item)
47
48         # 检查是否为子文件夹
49         if os.path.isdir(item_path):
50             subfolder_name = item
51
52             # 遍历子文件夹中的文件
53             for file in os.listdir(item_path):
54                 file_path = os.path.join(item_path, file)
55
56                 # 检查是否为视频文件
57                 if os.path.isfile(file_path) and _is_video_file(file):
58                     # 获取文件扩展名
59                     _, extension = os.path.splitext(file)
60
61                     # 生成新的文件名（使用子文件夹名字）
62                     new_filename = _get_unique_filename(target_folder,
subfolder_name, extension)
63                     target_file_path = os.path.join(target_folder,
new_filename)
64
65                     # 移动文件
66                     try:
67                         shutil.move(file_path, target_file_path)
68                         moved_count += 1
69                         print(f"已移动: {file} -> {new_filename}")
70                     except Exception as e:
71                         print(f"移动文件失败 {file}: {str(e)}")
72
73     result = f"成功移动了{moved_count}个视频文件"
74     return result
75

```



导出详情图txt 查找子文件夹中包含“详情”和“主图”的路径

```
1
2
3 from typing import *
4 try:
5     from xbot.app.logging import trace as print
6 except:
7     from xbot import print
8
9 def find_detail_and_main_image_paths_in_subfolders(folder_path):
10     """
11     title: 查找子文件夹中包含"详情"和"主图"的路径
12     description: 读取指定文件夹 % folder_path % 下的所有子文件夹，然后在每个子文件夹中
13     查找名字包含"详情"和"主图"的文件或文件夹的绝对路径，输出子文件夹名字和对应的路径列表。
14     inputs:
15         - folder_path (folder): 指定的文件夹路径，eg: "C:/Users/Desktop/主文件夹"
16     outputs:
17         - result (dict): 包含子文件夹名字和对应详情、主图路径的字典，eg: '{"子文件夹
18     1': ['C:/Users/Desktop/主文件夹/子文件夹1/产品详情.txt', 'C:/Users/Desktop/主文件
19     夹/子文件夹1/主图.jpg'], '子文件夹2': []}"
20     """
21
22     # 检查输入路径是否存在且为文件夹
23     if not os.path.exists(folder_path):
24         raise ValueError(f"指定路径不存在: {folder_path}")
25
26     if not os.path.isdir(folder_path):
27         raise ValueError(f"指定路径不是文件夹: {folder_path}")
28
29     def _get_subfolders(path):
30         """获取指定路径下的所有子文件夹"""
31         subfolders = []
32         try:
33             for item in os.listdir(path):
34                 item_path = os.path.join(path, item)
35                 if os.path.isdir(item_path):
36                     subfolders.append(item_path)
37         except PermissionError:
38             print(f"权限不足，无法访问: {path}")
39         return subfolders
40
41     def _find_target_items(subfolder_path):
42         """在指定文件夹中查找名字包含'详情'或'主图'的文件或文件夹"""
43         target_paths = []
44         try:
45             for item in os.listdir(subfolder_path):
46                 if "详情" in item or "主图" in item:
```

```

44     target_paths.append(os.path.abspath(os.path.join(subfolder_path, item)))
45     except PermissionError:
46         print(f"权限不足, 无法访问: {subfolder_path}")
47     return target_paths
48
49     # 获取所有子文件夹
50     subfolders = _get_subfolders(folder_path)
51
52     if not subfolders:
53         print(f"在路径 {folder_path} 下未找到任何子文件夹")
54         return {}
55
56     # 处理每个子文件夹
57     result = {}
58     for subfolder in subfolders:
59         subfolder_path = os.path.join(folder_path, subfolder)
60         target_paths = _find_target_items(subfolder_path)
61         result[subfolder] = target_paths
62
63     # 打印结果
64     if target_paths:
65         print(f"子文件夹 '{subfolder}' 中包含'详情'或'主图'的路径:")
66         for path in target_paths:
67             print(f"    - {path}")
68     else:
69         print(f"子文件夹 '{subfolder}' 中未找到包含'详情'或'主图'的文件或文件夹")
70
71     return result
72

```

导出详情图txt-将文本内容写入txt文件

```

1  import os
2
3  from typing import *
4  try:
5      from xbot.app.logging import trace as print
6  except:
7      from xbot import print
8
9
10 def write_text_to_file(text_content, parent_folder_path, sub_folder_path,
11     file_name):
12     """
13     title: 将文本内容写入txt文件

```

```
13     description: 将获取到的文本内容 % text_content % 写入到父路径 %
parent_folder_path % 和子路径 % sub_folder_path % 组成的文件夹下的 % file_name %
文件中，自动添加.txt后缀。
14     inputs:
15         - text_content (str): 要写入的文本内容，eg: "这是要写入文件的文本内容"
16         - parent_folder_path (folder): 父文件夹路径，eg: "C:/Documents"
17         - sub_folder_path (str): 子文件夹路径，eg: "MyProject/TextFiles"
18         - file_name (str): 文件名（不含扩展名），eg: "output"
19     outputs: None
20     """
21
22     # 检查输入有效性
23     if not isinstance(text_content, str):
24         raise TypeError("文本内容必须为字符串类型")
25
26     if not parent_folder_path:
27         raise ValueError("父文件夹路径不能为空")
28
29     if not sub_folder_path:
30         raise ValueError("子文件夹路径不能为空")
31
32     if not file_name:
33         raise ValueError("文件名不能为空")
34
35     def _ensure_folder_exists(folder_path):
36         """
37         确保文件夹存在，如果不存在则创建
38         """
39         if not os.path.exists(folder_path):
40             os.makedirs(folder_path)
41
42     def _write_content_to_file(content, parent_path, sub_path, filename):
43         """
44         将内容写入到指定文件
45         """
46         try:
47             # 组合完整文件夹路径
48             full_folder_path = os.path.join(parent_path, sub_path)
49
50             # 确保文件夹存在
51             _ensure_folder_exists(full_folder_path)
52
53             # 自动添加.txt后缀
54             full_filename = f"{filename}.txt"
55
56             # 组合完整文件路径
57             full_path = os.path.join(full_folder_path, full_filename)
```

```

58
59         with open(full_path, 'w', encoding='utf-8') as file:
60             file.write(content)
61     except IOError as e:
62         raise IOError(f"文件写入失败: {str(e)}")
63     except Exception as e:
64         raise Exception(f"写入过程中发生错误: {str(e)}")
65
66     # 执行文件写入操作
67     _write_content_to_file(text_content, parent_folder_path, sub_folder_path,
68                             file_name)

```

导出详情图txt-合并指定子文件夹中的txt文件

```

1  import os
2
3  from typing import *
4  try:
5      from xbot.app.logging import trace as print
6  except:
7      from xbot import print
8
9
10 def merge_txt_files_from_subfolders(parent_folder, subfolder_name,
    output_folder, output_filename):
11     """
12     title: 合并指定子文件夹中的txt文件
13     description: 在 % parent_folder % 下找到 % subfolder_name % 子文件夹, 将其中的
    txt文件内容合并写入到 % output_folder % 路径下的 % output_filename % 文件中。
14     inputs:
15         - parent_folder (folder): 父文件夹路径, eg: "C:/data/parent_folder"
16         - subfolder_name (str): 子文件夹名称, eg: "images"
17         - output_folder (folder): 输出文件夹路径, eg: "C:/output"
18         - output_filename (str): 输出文件名, eg: "merged_texts"
19     outputs:
20         - result (str): 处理结果信息, eg: "成功合并了3个txt文件"
21     """
22
23     # 检查父文件夹是否存在
24     if not os.path.exists(parent_folder):
25         raise FileNotFoundError(f"父文件夹不存在: {parent_folder}")
26
27     if not os.path.isdir(parent_folder):
28         raise ValueError(f"父路径不是文件夹: {parent_folder}")
29

```

```

30     # 构建完整的输入路径和输出路径
31     input_folder = os.path.join(parent_folder, subfolder_name)
32     if not os.path.exists(input_folder):
33         raise FileNotFoundError(f"子文件夹不存在: {input_folder}")
34
35     # 确保输出文件夹存在
36     if not os.path.exists(output_folder):
37         os.makedirs(output_folder)
38
39     # 自动添加.txt后缀
40     if not output_filename.lower().endswith('.txt'):
41         output_filename += '.txt'
42
43     output_file = os.path.join(output_folder, output_filename)
44
45     def _find_and_merge_txt_files(input_folder, output_file):
46         """
47         查找并合并txt文件
48         """
49         merged_count = 0
50
51         with open(output_file, 'w', encoding='utf-8') as output_f:
52             # 遍历输入文件夹下的所有子文件夹
53             for root, dirs, files in os.walk(input_folder):
54                 # 查找当前文件夹中的txt文件
55                 txt_files = [f for f in files if f.lower().endswith('.txt')]
56
57                 for txt_file in txt_files:
58                     txt_path = os.path.join(root, txt_file)
59
60                     # 获取相对路径作为标识
61                     relative_path = os.path.relpath(txt_path, input_folder)
62                     folder_name = os.path.basename(root) if root !=
input_folder else "根目录"
63                     file_name_without_ext = os.path.splitext(txt_file)[0]
64
65                     try:
66                         with open(txt_path, 'r', encoding='utf-8') as txt_f:
67                             content = txt_f.read()
68
69                             # 写入标识和内容
70                             output_f.write(f"=== {folder_name} -
{file_name_without_ext} ===\n")
71                             output_f.write(f"文件夹路径:
{os.path.dirname(relative_path) if os.path.dirname(relative_path) else '根目
录'}\n")
72                             output_f.write(f"文件路径: {relative_path}\n")

```

```

73         output_f.write("-" * 50 + "\n")
74         output_f.write(content)
75         output_f.write("\n" + "=" * 60 + "\n\n")
76
77         merged_count += 1
78
79     except Exception as e:
80         output_f.write(f"=== 读取失败: {relative_path} ===\n")
81         output_f.write(f"错误信息: {str(e)}\n")
82         output_f.write("=" * 60 + "\n\n")
83
84     return merged_count
85
86 merged_count = _find_and_merge_txt_files(input_folder, output_file)
87
88 if merged_count == 0:
89     return f"在 {subfolder_name} 文件夹中未找到任何txt文件"
90 else:
91     return f"成功合并了{merged_count}个txt文件到 {output_file}"
92

```

填补空缺参数-主流程-插入代码段1

```

1  import os
2  import re
3  from os import listdir, makedirs
4  from os.path import isfile, join, splitext
5  import pandas as pd
6  from fuzzywuzzy import fuzz  # 需要安装: pip install fuzzywuzzy
7  from xbot import print
8  # ----- 请修改以下配置项 -----
9  source_folder = 详情图txt文件夹  # 原TXT文件存放文件夹
10 output_folder = 参数合并文件夹  # 合并文件保存文件夹
11
12 txt_per_batch = 20  # 每个合并文件包含的TXT数量
13 excel_file_path = Excel文件路径  # Excel文件路径
14 product_name_col = "商品名称"  # Excel中商品名称所在列的列名
15 # -----
16
17 # 确保输出文件夹存在
18 if not os.path.exists(output_folder):
19     makedirs(output_folder)
20     print(f"已自动创建输出文件夹: {output_folder}")
21
22 def clean_filename(name):
23     """清理文件名中的特殊字符，用于匹配"""

```

```

24     # 移除文件名中不允许的字符
25     cleaned = re.sub(r'[/\/*?:"<>|]', '', name)
26     # 统一转为小写并去除空格, 提高匹配率
27     return cleaned.lower().replace(" ", "")
28
29     # 1. 读取Excel文件中的所有sheet和商品名称
30     excel_sheets = {}
31     try:
32         # 获取Excel中所有的sheet名称
33         excel_sheet_names = pd.ExcelFile(excel_file_path).sheet_names
34
35         for sheet_name in excel_sheet_names:
36             # 读取每个sheet的数据
37             df = pd.read_excel(excel_file_path, sheet_name=sheet_name)
38
39             # 检查商品名称列是否存在
40             if product_name_col not in df.columns:
41                 print(f"警告: Sheet '{sheet_name}' 中未找到 '{product_name_col}' 列,
已跳过该sheet")
42                 continue
43
44             # 提取商品名称并清理, 同时保留原始名称用于报告
45             product_names = []
46             for name in df[product_name_col].dropna(): # 排除空值
47                 product_names.append({
48                     "original": str(name),
49                     "cleaned": clean_filename(str(name))
50                 })
51
52             if product_names:
53                 excel_sheets[sheet_name] = product_names
54                 print(f"已读取Sheet '{sheet_name}', 包含 {len(product_names)} 个商
品")
55             else:
56                 print(f"警告: Sheet '{sheet_name}' 中未找到有效商品名称, 已跳过该sheet")
57
58     except Exception as e:
59         print(f"错误: 读取Excel文件失败 - {str(e)}")
60         exit()
61
62     if not excel_sheets:
63         print("错误: 未从Excel文件中读取到任何有效数据, 程序退出")
64         exit()
65
66     # 2. 读取并整理TXT文件
67     txt_files = {}

```

```

68 all_txt_files = [f for f in listdir(source_folder) if
69 isfile(join(source_folder, f)) and f.endswith(".txt")]
70
71 if not all_txt_files:
72     print("错误：原文件夹内未找到任何TXT文件，请检查路径！")
73     exit()
74
75 # 按sheet分组TXT文件
76 for txt_file in all_txt_files:
77     # 提取sheet名称（按第一个下划线分割）
78     if "_" in txt_file:
79         sheet_name = txt_file.split("_")[0]
80     else:
81         sheet_name = "未命名sheet"
82
83     # 提取商品名称（去除sheet名前缀和.txt后缀）
84     if "_" in txt_file:
85         product_part = "_".join(txt_file.split("_")[1:])
86     else:
87         product_part = txt_file
88     product_name = splitext(product_part)[0]
89
90     # 初始化sheet组
91     if sheet_name not in txt_files:
92         txt_files[sheet_name] = []
93
94     # 存储TXT文件信息
95     txt_files[sheet_name].append({
96         "filename": txt_file,
97         "original_product_name": product_name,
98         "cleaned_product_name": clean_filename(product_name),
99         "full_path": join(source_folder, txt_file)
100     })
101
102 # 3. 按Excel中的顺序匹配并排序TXT文件
103 matched_files = {} # 存储匹配成功的文件
104 unmatched_excel_products = {} # 存储Excel中未匹配到的商品
105 unmatched_txt_files = {} # 存储未匹配到Excel的TXT文件
106
107 for sheet_name, excel_products in excel_sheets.items():
108     # 初始化当前sheet的存储结构
109     matched_files[sheet_name] = []
110     unmatched_excel_products[sheet_name] = []
111
112     # 检查是否有对应的TXT文件
113     if sheet_name not in txt_files:
114         print(f"警告：Sheet '{sheet_name}' 在TXT文件夹中没有对应的文件")

```



```

114         unmatched_excel_products[sheet_name] = [p["original"] for p in
excel_products]
115         continue
116
117         # 复制一份TXT文件列表用于匹配 (避免重复匹配)
118         available_txts = txt_files[sheet_name].copy()
119
120         # 遍历Excel中的商品, 寻找最佳匹配
121         for excel_product in excel_products:
122             best_match = None
123             best_score = 0
124
125             # 寻找最佳匹配
126             for i, txt_file in enumerate(available_txts):
127                 # 使用模糊匹配算法计算相似度
128                 score = fuzz.ratio(excel_product["cleaned"],
txt_file["cleaned_product_name"])
129
130                 # 找到更优的匹配
131                 if score > best_score and score >= 60: # 相似度阈值, 可调整
132                     best_score = score
133                     best_match = i
134
135             # 处理匹配结果
136             if best_match is not None:
137                 matched_files[sheet_name].append(available_txts.pop(best_match))
138             else:
139
unmatched_excel_products[sheet_name].append(excel_product["original"])
140
141             # 记录未匹配的TXT文件
142             unmatched_txt_files[sheet_name] = [txt["filename"] for txt in
available_txts]
143
144         # 4. 合并文件
145         total_merged_files = 0
146         total_merged_products = 0
147
148         for sheet_name, sorted_txts in matched_files.items():
149             if not sorted_txts:
150                 continue
151
152             # 计算需要分几批
153             batch_count = (len(sorted_txts) + txt_per_batch - 1) // txt_per_batch
154
155             for batch_idx in range(batch_count):
156                 # 计算当前批的范围

```

```

157     start = batch_idx * txt_per_batch
158     end = min(start + txt_per_batch, len(sorted_txts))
159     current_batch = sorted_txts[start:end]
160
161     # 生成合并文件名
162     product_start = start + 1
163     product_end = end
164     merged_filename = f"{sheet_name}_合并_{product_start}-{product_end}.txt"
165     merged_filepath = join(output_folder, merged_filename)
166
167     # 合并文件内容
168     with open(merged_filepath, "w", encoding="utf-8") as merged_f:
169         for txt_file in current_batch:
170             try:
171                 # 尝试用utf-8编码读取
172                 with open(txt_file["full_path"], "r", encoding="utf-8") as
txt_f:
173                     txt_content = txt_f.read()
174             except UnicodeDecodeError:
175                 try:
176                     # 尝试用gbk编码读取
177                     with open(txt_file["full_path"], "r", encoding="gbk")
as txt_f:
178                         txt_content = txt_f.read()
179                 except Exception as e:
180                     print(f"警告：文件{txt_file['filename']}编码异常，已跳过！
错误：{e}")
181                     continue
182                 except Exception as e:
183                     print(f"警告：读取文件{txt_file['filename']}失败，已跳过！错
误：{e}")
184                     continue
185
186     # 写入带标识的内容
187     merged_f.write(f"【{txt_file['original_product_name']} 详情开始】
\n")
188     merged_f.write(txt_content.strip() + "\n")
189     merged_f.write(f"【{txt_file['original_product_name']} 详情结束】
\n\n")
190
191     print(f"✅ 已生成：{merged_filename} (包含{len(current_batch)}个TXT) ")
192     total_merged_files += 1
193     total_merged_products += len(current_batch)
194
195     # 5. 输出结果报告
196     print("\n" + "="*50)
197     print("🎉 合并操作完成！ ")

```

```

198 print(f"📁 合并文件保存路径: {output_folder}")
199 print(f"📊 共处理 {len(matched_files)} 个sheet, 生成 {total_merged_files} 个合并文件, 合并 {total_merged_products} 个商品")
200
201 # 6. 输出未匹配项报告
202 has_unmatched = False
203
204 print("\n" + "="*50)
205 print(f"🔍 匹配状态报告: ")
206
207 for sheet_name, products in unmatched_excel_products.items():
208     if products:
209         has_unmatched = True
210         print(f"\n⚠️ Sheet '{sheet_name}' 中有 {len(products)} 个Excel商品未找到匹配的TXT文件: ")
211         for i, product in enumerate(products[:5]): # 只显示前5个
212             print(f"    - {product}")
213         if len(products) > 5:
214             print(f"    ... 还有 {len(products)-5} 个未显示")
215
216 for sheet_name, files in unmatched_txt_files.items():
217     if files:
218         has_unmatched = True
219         print(f"\n⚠️ Sheet '{sheet_name}' 中有 {len(files)} 个TXT文件未在Excel中找到匹配: ")
220         for i, file in enumerate(files[:5]): # 只显示前5个
221             print(f"    - {file}")
222         if len(files) > 5:
223             print(f"    ... 还有 {len(files)-5} 个未显示")
224
225 if not has_unmatched:
226     print(f"\n✅ 所有商品和TXT文件都已成功匹配!")
227 else:
228     print(f"\n提示: 请检查上述未匹配项, 可能需要调整文件名或Excel中的商品名称以提高匹配率")
229 print("="*50)
230

```

填补空缺参数-主流程-插入代码段2

```

1 import os
2 from xbot import print
3
4 # 指定目标文件夹路径
5 folder_path = 参数合并文件夹 # 请替换为实际路径
6

```

```

7  # 初始化存储txt文件路径的列表
8  txt_paths = []
9
10 # 检查文件夹是否存在且有效
11 if os.path.exists(folder_path) and os.path.isdir(folder_path):
12     # 只获取当前文件夹中的文件（不包含子文件夹）
13     files = os.listdir(folder_path)
14     for file in files:
15         # 拼接完整路径
16         file_path = os.path.join(folder_path, file)
17         # 筛选出txt文件（不区分大小写）且是文件而不是文件夹
18         if os.path.isfile(file_path) and file.lower().endswith(".txt"):
19             txt_paths.append(file_path)
20 else:
21     print(f"错误: '{folder_path}' 不是有效的文件夹路径")
22
23 # 输出结果
24 print(txt_paths)

```

填补空缺参数-主流程-批量新增Excel列名(跳过分类sheet)

```

1  # 使用此指令前，请确保安装必要的Python库，例如使用以下命令安装：
2  # pip install openpyxl
3
4  import openpyxl
5  import os
6  import ast
7
8  from typing import *
9  try:
10     from xbot.app.logging import trace as print
11 except:
12     from xbot import print
13
14 def add_columns_to_excel_sheets(folder_path, file_name, column_names,
15 start_column):
16     """
17     title: 批量新增Excel列名（跳过分类sheet）
18     description: 打开指定文件夹中的Excel文件，在除了名为"分类"的sheet之外的所有sheet中
19     新增 % column_names % 列表中的列名。从第 % start_column % 列开始检查，如果列名已存在
20     则跳过不添加。直接修改原文件。
21
22     inputs:
23     - folder_path (folder): Excel文件所在的文件夹路径，eg: "C:/Documents"
24     - file_name (str): Excel文件名，eg: "data.xlsx"
25     - column_names (list): 要新增的列名列表，eg: ["姓名", "年龄", "部门"]
26     - start_column (int): 开始检查列名的列数，eg: "3"

```

```

23     outputs:
24         - None
25     """
26
27     # 1. 检查输入有效性和类型转换
28     if column_names is None:
29         raise ValueError("列名参数不能为None")
30
31     # 自动处理不同类型的输入
32     if isinstance(column_names, str):
33         # 检查是否是字符串形式的列表
34         column_names_stripped = column_names.strip()
35         if column_names_stripped.startswith '[' and
column_names_stripped.endswith(']'):
36             try:
37                 # 尝试解析字符串形式的列表
38                 column_names = ast.literal_eval(column_names_stripped)
39                 if not isinstance(column_names, list):
40                     column_names = [column_names]
41             except (ValueError, SyntaxError):
42                 # 如果解析失败，当作普通字符串处理
43                 column_names = [column_names]
44         else:
45             # 如果是普通字符串，转换为单元素列表
46             column_names = [column_names]
47     elif not isinstance(column_names, list):
48         # 如果是其他可迭代类型（如元组、集合等），尝试转换为列表
49         try:
50             column_names = list(column_names)
51         except TypeError:
52             raise ValueError("列名参数必须是字符串、列表或其他可迭代类型")
53
54     print(f"处理后的列名列表：{column_names}")
55     print(f"列名数量：{len(column_names)}")
56
57     if not file_name or not isinstance(file_name, str):
58         raise ValueError("文件名不能为空且必须为字符串类型")
59
60     if not isinstance(start_column, int) or start_column < 1:
61         raise ValueError("开始列数必须为正整数")
62
63     def _add_columns_to_sheet(sheet, columns, start_col):
64         """
65         在指定sheet中新增列名
66         """
67         # 如果没有要添加的列名，直接返回
68         if len(columns) == 0:

```

```
69         return
70
71     print(f"\n=== 处理工作表: {sheet.title} ===")
72
73     # 通过扫描第1行来找到最后一个有数据的列
74     last_col_with_data = 0
75     max_check_col = max(sheet.max_column, 200) # 扩大检查范围
76
77     # 先清理可能存在的 Unnamed 列名
78     for col in range(1, max_check_col + 1):
79         cell = sheet.cell(row=1, column=col)
80         cell_value = cell.value
81         if cell_value is not None:
82             cell_str = str(cell_value).strip()
83             # 如果发现 Unnamed 格式的列名, 清空它
84             if cell_str.startswith("Unnamed:"):
85                 print(f"发现并清除无效列名: {cell_str} (位置: 第{col}列)")
86                 cell.value = None
87             elif cell_str != "":
88                 last_col_with_data = col
89
90     print(f"清理后, 检测到的最后一列位置: {last_col_with_data}")
91
92     # 获取从start_col开始的现有列名
93     existing_columns = []
94     all_existing_columns = [] # 用于调试显示
95
96     for col in range(1, last_col_with_data + 1):
97         cell_value = sheet.cell(row=1, column=col).value
98         if cell_value is not None:
99             cell_str = str(cell_value).strip()
100             all_existing_columns.append(f"第{col}列: '{cell_str}'")
101             if col >= start_col and cell_str != "":
102                 existing_columns.append(cell_str)
103
104     print(f"所有现有列名: {all_existing_columns}")
105     print(f"从第{start_col}列开始的现有列名: {existing_columns}")
106
107     # 找出需要添加的新列名 (不在现有列名中的)
108     new_columns = []
109     for col_name in columns:
110         col_name_str = str(col_name).strip()
111         if col_name_str and col_name_str not in existing_columns:
112             new_columns.append(col_name_str)
113         else:
114             print(f"跳过已存在的列名: '{col_name_str}'")
115
```

```

116     print(f"需要添加的新列名: {new_columns}")
117     print(f"需要添加的列数: {len(new_columns)}")
118
119     # 从下一列开始添加新的列名
120     next_col = last_col_with_data + 1
121     for i, col_name in enumerate(new_columns):
122         current_col = next_col + i
123         cell = sheet.cell(row=1, column=current_col)
124
125         print(f"准备在第{current_col}列写入: '{col_name}' (类型:
126 {type(col_name)})")
127
128         # 确保写入的是字符串
129         cell.value = str(col_name)
130
131         # 验证写入结果
132         written_value = sheet.cell(row=1, column=current_col).value
133         print(f"写入后验证 - 第{current_col}列的值: '{written_value}'")
134
135         if str(written_value) != str(col_name):
136             print(f"警告: 写入验证失败! 期望: '{col_name}', 实际:
137 '{written_value}'")
138
139     try:
140         # 2. 组合完整的Excel文件路径
141         excel_path = os.path.join(folder_path, file_name)
142
143         # 3. 检查文件是否存在
144         if not os.path.exists(excel_path):
145             raise FileNotFoundError(f"找不到指定的Excel文件: {excel_path}")
146
147         # 4. 打开Excel文件
148         workbook = openpyxl.load_workbook(excel_path)
149         print(f"成功打开Excel文件, 包含的工作表: {workbook.sheetnames}")
150
151         # 5. 遍历所有sheet并添加列名 (跳过名为"分类"的sheet)
152         for sheet_name in workbook.sheetnames:
153             if sheet_name == "分类":
154                 print(f"跳过工作表: {sheet_name}")
155                 continue
156             sheet = workbook[sheet_name]
157             _add_columns_to_sheet(sheet, column_names, start_column)
158
159         # 6. 保存修改后的文件
160         print("开始保存文件...")
161         workbook.save(excel_path)
162         workbook.close()

```

```

161         print("Excel文件处理完成")
162
163     except FileNotFoundError as e:
164         raise e
165     except Exception as e:
166         raise Exception(f"处理Excel文件时发生错误: {str(e)}")
167

```

填补空缺参数-主流程-合并文件夹中所有txt文件

```

1  import os
2  import json
3
4  from typing import *
5  try:
6      from xbot.app.logging import trace as print
7  except:
8      from xbot import print
9
10
11  def merge_all_txt_files(folder_path, output_file_path):
12      """
13      title: 合并文件夹中所有txt文件
14      description: 将指定文件夹 % folder_path % 下面的所有txt文件合并到一个文件 %
15      output_file_path % 中。每个txt文件包含JSON格式的字典列表，最终合并所有字典到输出文件。
16      inputs:
17          - folder_path (folder): 包含txt文件的文件夹路径，eg: "C:/data"
18          - output_file_path (file): 合并后的输出文件路径，eg:
19      "C:/output/merged_data.txt"
20      outputs:
21          - result_path (str): 合并后的文件路径，eg: "C:/output/merged_data.txt"
22      """
23
24      if not os.path.exists(folder_path):
25          raise FileNotFoundError(f"文件夹路径不存在: {folder_path}")
26
27      # 确保输出文件的目录存在
28      output_dir = os.path.dirname(output_file_path)
29      if output_dir and not os.path.exists(output_dir):
30          os.makedirs(output_dir)
31
32      def _get_all_txt_files():
33          """获取文件夹中所有txt文件"""
34          txt_files = []
35          try:
36              for file_name in os.listdir(folder_path):

```



```

35         if file_name.lower().endswith('.txt'):
36             file_path = os.path.join(folder_path, file_name)
37             if os.path.isfile(file_path):
38                 txt_files.append(file_path)
39         return sorted(txt_files) # 按名称排序确保处理顺序一致
40     except Exception as e:
41         raise Exception(f"读取文件夹失败: {str(e)}")
42
43 def _read_file_content(file_path):
44     """读取并解析txt文件中的JSON列表内容"""
45     try:
46         with open(file_path, 'r', encoding='utf-8') as f:
47             content = f.read().strip()
48             if not content:
49                 return [] # 空文件返回空列表
50             # 使用json.loads解析JSON格式
51             data = json.loads(content)
52             if not isinstance(data, list):
53                 raise ValueError(f"文件 {file_path} 内容格式错误, 应为列表格
式")
54             return data
55     except json.JSONDecodeError as e:
56         raise Exception(f"文件 {file_path} JSON格式错误: {str(e)}")
57     except Exception as e:
58         raise Exception(f"读取文件 {file_path} 失败: {str(e)}")
59
60 def _merge_all_txt_files():
61     """合并所有txt文件中的字典"""
62     txt_files = _get_all_txt_files()
63
64     if not txt_files:
65         raise ValueError(f"文件夹 {folder_path} 中没有找到txt文件")
66
67     all_dictionaries = []
68
69     for file_path in txt_files:
70         file_data = _read_file_content(file_path)
71
72         # 检查列表中的元素是否为字典
73         for item in file_data:
74             if not isinstance(item, dict):
75                 raise ValueError(f"文件 {file_path} 中存在非字典元素:
{type(item)}")
76             all_dictionaries.append(item)
77
78     return all_dictionaries
79

```

```

80     # 合并所有字典
81     merged_data = _merge_all_txt_files()
82
83     # 写入合并后的文件（保持JSON格式）
84     try:
85         with open(output_file_path, 'w', encoding='utf-8') as f:
86             json.dump(merged_data, f, ensure_ascii=False, indent=2)
87     except Exception as e:
88         raise Exception(f"写入合并文件失败: {str(e)}")
89
90     return output_file_path
91

```

填补空缺参数-主流程-根据txt文件数据更新Excel商品信息(保持原格式)

```

1  # 使用此指令前，请确保安装必要的Python库，例如使用以下命令安装：
2  # pip install pandas openpyxl
3
4  import pandas as pd
5  import ast
6  import os
7  import re
8  from openpyxl import load_workbook
9
10 from typing import *
11 try:
12     from xbot.app.logging import trace as print
13 except:
14     from xbot import print
15
16
17 def update_excel_with_txt_data(excel_folder_path, excel_filename, txt_path,
18                               output_folder_path, output_filename):
19     """
20     title: 根据txt文件数据更新Excel商品信息（保持原格式）
21     description: 读取 % txt_path % 文件中的字典列表，直接使用商品名称在所有sheet中匹
22     配"商品名称"列，将匹配的数据填入对应行列，保存到指定输出文件夹，完全保持原Excel格式。详细
23     显示每个字段的更新情况。
24     inputs:
25         - excel_folder_path (folder): Excel文件所在文件夹路径，eg: "C:/data"
26         - excel_filename (str): Excel文件名，eg: "products.xlsx"
27         - txt_path (file): txt文件路径，eg: "C:/data/product_data.txt"
28         - output_folder_path (folder): 输出文件夹路径，eg: "C:/output"
29         - output_filename (str): 输出Excel文件名，eg: "updated_products.xlsx"
30     outputs: None
31     """

```

```

29
30 def _build_file_path(folder_path, filename):
31     """构建完整文件路径"""
32     return os.path.join(folder_path, filename)
33
34 def _read_txt_data(file_path):
35     """读取并解析txt文件中的字典列表"""
36     try:
37         with open(file_path, 'r', encoding='utf-8') as f:
38             content = f.read().strip()
39             return ast.literal_eval(content)
40     except Exception as e:
41         raise ValueError(f"txt文件格式错误, 无法解析: {e}")
42
43 def _normalize_string(s):
44     """标准化字符串: 统一分隔符、去除多余空格、转换为小写"""
45     if isinstance(s, str):
46         # 去除首尾空格和制表符
47         normalized = s.strip().rstrip('\t')
48
49         # 统一各种分隔符: 将 _ / \ | - 等都替换为空格, 然后统一处理
50         normalized = re.sub(r'[_/\\|\-]', ' ', normalized)
51
52         # 将多个连续空格替换为单个空格
53         normalized = re.sub(r'\s+', ' ', normalized)
54
55         # 转换为小写
56         return normalized.lower()
57     return str(s).lower() if s is not None else ''
58
59 def _clean_string(s):
60     """清理字符串, 去除首尾空格和制表符"""
61     if isinstance(s, str):
62         return s.strip().rstrip('\t')
63     return s
64
65 def _calculate_similarity(str1, str2):
66     """计算两个字符串的相似度 (基于标准化后的字符串) """
67     norm_str1 = _normalize_string(str1)
68     norm_str2 = _normalize_string(str2)
69
70     # 完全匹配
71     if norm_str1 == norm_str2:
72         return 1.0
73
74     # 包含匹配 (双向检查)
75     if norm_str1 in norm_str2:

```

```
76         return len(norm_str1) / len(norm_str2)
77     elif norm_str2 in norm_str1:
78         return len(norm_str2) / len(norm_str1)
79
80     # 计算共同字符占比
81     set1 = set(norm_str1)
82     set2 = set(norm_str2)
83     intersection = set1.intersection(set2)
84     union = set1.union(set2)
85
86     if len(union) == 0:
87         return 0.0
88
89     return len(intersection) / len(union)
90
91 def _find_matching_rows_in_sheet(sheet_df, product_name, sheet_name):
92     """在指定sheet中寻找匹配的行"""
93     matches = []
94
95     if '商品名称' not in sheet_df.columns:
96         return [], "无商品名称列"
97
98     # 创建标准化的商品名称
99     norm_target_name = _normalize_string(product_name)
100
101     print(f"  在sheet '{sheet_name}' 中查找商品名称: '{product_name}'")
102     print(f"  标准化后: '{norm_target_name}'")
103
104     # 遍历所有行, 计算相似度
105     for idx, row_product_name in sheet_df['商品名称'].items():
106         if pd.isna(row_product_name):
107             continue
108
109         # 清理Excel中的商品名称 (去除制表符等)
110         cleaned_row_name = _clean_string(str(row_product_name))
111         similarity = _calculate_similarity(cleaned_row_name, product_name)
112
113         if similarity > 0.4: # 降低阈值, 显示更多候选项
114             matches.append((idx, similarity, cleaned_row_name))
115             norm_row_name = _normalize_string(cleaned_row_name)
116             print(f"    候选匹配 [{idx}]: '{cleaned_row_name}' (相似度:
117 {similarity:.3f})")
118
119     if not matches:
120         return [], "无匹配"
121
122     # 按相似度排序, 选择最佳匹配
```

```
122     matches.sort(key=lambda x: x[1], reverse=True)
123     best_similarity = matches[0][1]
124
125     # 只选择相似度最高的匹配项
126     best_matches = [match for match in matches if match[1] ==
best_similarity]
127
128     match_indices = [match[0] for match in best_matches]
129
130     if best_similarity >= 0.95:
131         return match_indices, "完全匹配"
132     elif best_similarity >= 0.8:
133         return match_indices, f"高相似度匹配({best_similarity:.3f})"
134     elif best_similarity >= 0.6:
135         return match_indices, f"中相似度匹配({best_similarity:.3f})"
136     elif best_similarity >= 0.4:
137         return match_indices, f"低相似度匹配({best_similarity:.3f})"
138     else:
139         return [], "相似度过低, 跳过匹配"
140
141     def _get_column_index_by_name(ws, column_name):
142         """根据列名获取列索引"""
143         for cell in ws[1]: # 假设第一行是标题行
144             if cell.value == column_name:
145                 return cell.column
146         return None
147
148     def _update_sheet_data_preserve_format(ws, sheet_df, sheet_name,
target_data_list):
149         """更新指定sheet的数据, 保持原格式"""
150         updated_count = 0
151
152         if '商品名称' not in sheet_df.columns:
153             print(f"警告: sheet '{sheet_name}' 中没有'商品名称'列, 跳过处理")
154             return updated_count
155
156         print(f"\n开始处理sheet '{sheet_name}'...")
157         print(f"该sheet中共有 {len(sheet_df)} 行数据")
158
159         # 建立列名到列索引的映射
160         column_mapping = {}
161         for cell in ws[1]: # 第一行是标题行
162             if cell.value:
163                 column_mapping[cell.value] = cell.column
164
165         print(f"Excel中可用的列名: {list(column_mapping.keys())}")
166
```

```

167         for data_item in target_data_list:
168             try:
169                 # 直接使用文件名作为商品名称进行匹配
170                 product_name = data_item.get('文件名', '')
171                 if not product_name:
172                     print("跳过：数据项中没有'文件名'字段")
173                     continue
174
175                 print(f"\n处理商品：{product_name}")
176                 print(f"数据项包含的字段：{list(data_item.keys())}")
177
178                 # 在当前sheet中查找匹配的行
179                 matching_rows, match_type =
_find_matching_rows_in_sheet(sheet_df, product_name, sheet_name)
180
181                 if len(matching_rows) > 0:
182                     print(f"找到 {len(matching_rows)} 个匹配行（匹配类型：
{match_type}）")
183
184                     for row_idx in matching_rows:
185                         excel_product_name =
_clean_string(str(sheet_df.loc[row_idx, '商品名称']))
186                         print(f" 匹配行 {row_idx}: '{excel_product_name}'")
187
188                     # 统计更新情况
189                     updated_fields = []
190                     skipped_fields = []
191
192                     # 更新匹配行的数据，直接写入openpyxl的工作表
193                     for key, value in data_item.items():
194                         if key == '文件名':
195                             continue # 跳过文件名字段
196
197                         if key in column_mapping:
198                             col_idx = column_mapping[key]
199                             for row_idx in matching_rows:
200                                 # row_idx是pandas的索引，需要转换为Excel的行号
201                                 # (+2, 因为Excel从1开始，且有标题行)
202                                 excel_row = row_idx + 2
203                                 ws.cell(row=excel_row, column=col_idx,
value=value)
204
205                                 updated_fields.append(key)
206                             else:
207                                 skipped_fields.append(key)
208
209                     updated_count += len(matching_rows)
210                     print(f"已更新 {len(matching_rows)} 行数据")

```

```

208             print(f"  成功更新的字段 ({len(updated_fields)}个):
{updated_fields}")
209             if skipped_fields:
210                 print(f"  跳过的字段 ({len(skipped_fields)}个, Excel中无对
应列): {skipped_fields}")
211             else:
212                 print(f"在sheet '{sheet_name}' 中未找到商品 '{product_name}'
的匹配")
213
214             except Exception as e:
215                 print(f"处理数据项时出错: {e}")
216                 continue
217
218             return updated_count
219
220     # 构建完整文件路径
221     excel_path = _build_file_path(excel_folder_path, excel_filename)
222     output_path = _build_file_path(output_folder_path, output_filename)
223
224     # 检查输入文件是否存在
225     if not os.path.exists(excel_path):
226         raise FileNotFoundError(f"Excel文件不存在: {excel_path}")
227
228     if not os.path.exists(txt_path):
229         raise FileNotFoundError(f"txt文件不存在: {txt_path}")
230
231     # 确保输出文件夹存在
232     os.makedirs(output_folder_path, exist_ok=True)
233
234     # 读取txt文件数据
235     try:
236         product_data = _read_txt_data(txt_path)
237         if not isinstance(product_data, list):
238             raise ValueError("txt文件内容应该是一个列表")
239         print(f"成功读取txt文件, 共有 {len(product_data)} 条数据")
240     except Exception as e:
241         raise ValueError(f"读取txt文件失败: {e}")
242
243     # 用pandas读取Excel文件的所有sheet (用于数据分析)
244     try:
245         excel_data = pd.read_excel(excel_path, sheet_name=None)
246         print(f"成功读取Excel文件, 共有 {len(excel_data)} 个sheet:
{list(excel_data.keys())}")
247     except Exception as e:
248         raise ValueError(f"读取Excel文件失败: {e}")
249
250     # 用openpyxl加载Excel文件 (用于保持格式)

```

```

251     try:
252         workbook = load_workbook(excel_path)
253         print("成功加载Excel工作簿，保持原有格式")
254     except Exception as e:
255         raise ValueError(f"加载Excel工作簿失败：{e}")
256
257     total_updated = 0
258
259     # 处理每个sheet
260     for sheet_name in excel_data.keys():
261         if sheet_name in workbook.sheetnames:
262             try:
263                 sheet_df = excel_data[sheet_name]
264                 ws = workbook[sheet_name]
265
266                 updated_count = _update_sheet_data_preserve_format(ws,
sheet_df, sheet_name, product_data)
267                 total_updated += updated_count
268                 print(f"\nsheet '{sheet_name}' 处理完成，更新了 {updated_count}
行数据")
269             except Exception as e:
270                 raise RuntimeError(f"处理sheet '{sheet_name}' 时发生错误：{e}")
271         else:
272             print(f"警告：sheet '{sheet_name}' 在workbook中未找到")
273
274     # 保存更新后的Excel文件，保持所有原有格式
275     try:
276         workbook.save(output_path)
277         print(f"成功保存Excel文件，保持原有格式：{output_path}")
278     except Exception as e:
279         raise RuntimeError(f"保存Excel文件失败：{e}")
280
281     print(f"\n处理完成！成功更新{total_updated}个商品信息，已保存到：{output_path}")
282

```

#### 填补空缺参数-子流程2-将txt文件复制到剪贴板

```

1  # 使用此指令前，请确保安装必要的Python库，例如使用以下命令安装：
2  # pip install pywin32
3
4  import os
5  import win32clipboard
6  import win32con
7
8  from typing import *
9  try:

```



```

10     from xbot.app.logging import trace as print
11 except:
12     from xbot import print
13
14 def copy_txt_file_to_clipboard(file_path):
15     """
16     title: 将txt文件复制到剪贴板
17     description: 根据指定的txt文件路径 %file_path%, 将txt文件本身复制到剪贴板, 以便粘
18     贴到其他位置 (如文件管理器)。
19     inputs:
20         - file_path (file): txt文件的完整路径, eg: "C:/documents/example.txt"
21     outputs:
22         - None
23     """
24     # 检查文件路径是否存在
25     if not os.path.exists(file_path):
26         raise FileNotFoundError(f"指定的文件路径不存在: {file_path}")
27
28     # 检查是否为文件 (而不是文件夹)
29     if not os.path.isfile(file_path):
30         raise ValueError(f"指定的路径不是文件: {file_path}")
31
32     # 检查文件扩展名是否为txt
33     if not file_path.lower().endswith('.txt'):
34         raise ValueError(f"文件必须是txt格式: {file_path}")
35
36 def _copy_file_to_clipboard(file_path):
37     """将文件复制到剪贴板"""
38     try:
39         # 将路径转换为绝对路径并规范化
40         abs_path = os.path.abspath(file_path).replace('/', '\\')
41
42         # 构造DROPFILES结构
43         # DROPFILES结构: 20字节头部 + 文件路径列表 + 双空字符结尾
44         file_list = abs_path + '\\0\\0' # 文件路径 + 双空字符结尾
45         file_bytes = file_list.encode('utf-16le')
46
47         # DROPFILES结构头部 (20字节)
48         drop_files_header = b'\\x14\\x00\\x00\\x00' # 结构大小(20)
49         drop_files_header += b'\\x00\\x00\\x00\\x00' # 鼠标位置X
50         drop_files_header += b'\\x00\\x00\\x00\\x00' # 鼠标位置Y
51         drop_files_header += b'\\x00\\x00\\x00\\x00' # 是否在NC区域
52         drop_files_header += b'\\x01\\x00\\x00\\x00' # Unicode标志
53
54         # 完整的剪贴板数据
55         clipboard_data = drop_files_header + file_bytes

```

```

56
57         # 设置剪贴板
58         win32clipboard.OpenClipboard()
59         win32clipboard.EmptyClipboard()
60         win32clipboard.SetClipboardData(win32con.CF_HDROP, clipboard_data)
61         win32clipboard.CloseClipboard()
62
63     except Exception as e:
64         try:
65             win32clipboard.CloseClipboard()
66         except:
67             pass
68         raise Exception(f"复制文件到剪贴板时出错: {str(e)}")
69
70     try:
71         _copy_file_to_clipboard(file_path)
72         return None
73     except Exception as e:
74         raise Exception(f"复制txt文件到剪贴板时出错: {str(e)}")
75

```

填补空缺参数-子流程2-插入代码段

```

1  import os
2
3  from xbot import print
4
5  文件名 = os.path.basename(txt文件路径)
6  print(文件名)

```

填补空缺参数-子流程2-将剪贴板内容写入txt文件

```

1  # 使用此指令前, 请确保安装必要的Python库, 例如使用以下命令安装:
2  # pip install pyperclip
3
4  import pyperclip
5  import os
6
7  from typing import *
8  try:
9      from xbot.app.logging import trace as print
10 except:
11     from xbot import print
12
13
14 def write_clipboard_to_txt(folder_path, file_name):

```

```
15     """
16     title: 将剪贴板内容写入txt文件
17     description: 获取剪贴板中的内容，并将其写入到指定文件夹路径下的txt文件中。需要输入
% folder_path % 和 % file_name % 两个参数。
18     inputs:
19         - folder_path (str): 文件夹路径, eg: "C:/Users/Desktop"
20         - file_name (str): 文件名 (包含.txt扩展名), eg: "clipboard_content.txt"
21     outputs: None
22     """
23
24     # 检查输入有效性
25     if not folder_path or not file_name:
26         raise ValueError("文件夹路径和文件名不能为空")
27
28     if not file_name.endswith('.txt'):
29         file_name += '.txt'
30
31     def _get_clipboard_content():
32         """获取剪贴板内容"""
33         try:
34             content = pyperclip.paste()
35             return content
36         except Exception as e:
37             raise RuntimeError(f"无法获取剪贴板内容: {str(e)}")
38
39     def _ensure_folder_exists(folder_path):
40         """确保文件夹存在, 如果不存在则创建"""
41         if not os.path.exists(folder_path):
42             os.makedirs(folder_path)
43
44     def _write_to_file(file_path, content):
45         """将内容写入文件"""
46         try:
47             with open(file_path, 'w', encoding='utf-8') as f:
48                 f.write(content)
49         except Exception as e:
50             raise RuntimeError(f"写入文件失败: {str(e)}")
51
52     # 获取剪贴板内容
53     clipboard_content = _get_clipboard_content()
54
55     # 确保文件夹存在
56     _ensure_folder_exists(folder_path)
57
58     # 构建完整文件路径
59     full_file_path = os.path.join(folder_path, file_name)
60
```

```
61 # 写入文件
62 _write_to_file(full_file_path, clipboard_content)
63
```

## 三、PPT提效

基于我上面自己的方法的基础上进一步优化PPT制作效率；  
首先说明，我用的是**稿定设计**做PPT

### 3.1 稿定设计如何快速批量套版

稿定设计批量套版提效--清泉

文件配置

源Excel文件:

E:\B站\选品\显示器\分类.xlsx

浏览

模板Excel文件:

E:\B站\选品\显示器\1-数据上传.xlsx

浏览

输出文件夹:

E:\B站\选品\显示器

浏览

参数配置

提取列名:

商品简称, 到手价, 屏幕尺寸, 分辨率, 屏幕刷新率, 面板, 亮度, 色域, 响应时间, 接口, 推荐理由

处理进度

开始处理

重置

处理日志

✓ 已加载上次保存的配置

[📄 提效代码](#) [具体流程参考此文档](#)

### 3.2 PPT图片的商品主图如何快速找

代码块

```

1  import os
2  import shutil
3  import re
4  import ast
5  import json
6  import numpy as np
7  import pandas as pd
8  from openpyxl import load_workbook
9  from PIL import Image
10
11 def sanitize_filename(name):
12     """去除文件名中不允许的字符"""
13     return re.sub(r'[/\/*?:"<>|]', '', name)
14
15 def clean_text(text):
16     """清洗文本：保留更多字符，避免过度清洗影响关键词匹配"""
17     if not text:
18         return ""
19     # 保留字母、数字、中文、常见符号（如Δ、°等），只移除空格和常见标点
20     cleaned = re.sub(r'[\s\n\r\t,.,! ? ; : ""' () () [] \[\]\{\}]', '', text)
21     return cleaned.lower()
22
23 def parse_ocr_text(ocr_list_str):
24     """解析图片对应的OCR列表字符串，提取有效文本（减少过度过滤）"""
25     try:
26         ocr_list = ast.literal_eval(ocr_list_str.strip())
27         valid_text = [
28             str(item).strip() for item in ocr_list
29             if item and str(item).strip() # 移除长度限制，保留更多文本
30         ]
31         # 不去重，保持原始顺序，避免丢失重要信息
32         return " ".join(valid_text)
33     except (SyntaxError, ValueError):
34         return re.sub(r'\s+', ' ', ocr_list_str.strip())
35
36 def parse_ocr_collection(ocr_txt_content):
37     """解析TXT中的多图OCR集合，返回{图片名: OCR文本}的字典"""
38     img_ocr_dict = {}
39     img_segments = re.split(r'=== 根目录 - ', ocr_txt_content)
40     for segment in img_segments:
41         if not segment.strip():
42             continue
43         img_name_match = re.match(r'([^\s=]+)\s+===', segment)
44         if not img_name_match:
45             continue
46         img_name = img_name_match.group(1).strip().lower()

```

```

47         ocr_content_match = re.search(r'-----\n(.*?)\n-----\n=====|$', segment,
-----\n(.*?)\n-----\n=====|$', segment,
re.DOTALL)
48         if ocr_content_match:
49             ocr_list_str = ocr_content_match.group(1).strip()
50             img_ocr_dict[img_name] = parse_ocr_text(ocr_list_str)
51         return img_ocr_dict
52
53     def count_valid_text(text):
54         """统计有效文本长度（排除空格、符号，仅算中英文/数字）"""
55         valid_chars = re.findall(r'[\u4e00-\u9fa5a-zA-Z0-9]', text)
56         return len(valid_chars)
57
58     def get_image_dimensions_and_ratio(image_path):
59         """获取图片的宽、高和宽高比，检测750×1499比例"""
60         try:
61             with Image.open(image_path) as img:
62                 width, height = img.size
63                 if height == 0:
64                     return width, height, 0, False
65                 ratio = width / height
66                 is_target_ratio = (abs(width - 750) <= 10) and (abs(height - 1499)
<= 10)
67                 return width, height, ratio, is_target_ratio
68         except Exception as e:
69             print(f"⚠️ 获取图片信息失败 {image_path}: {str(e)}")
70             return 0, 0, 0, False
71
72     # 核心负面关键词（绝对不能出现，即使兜底也要排除）
73     CRITICAL_NEGATIVE_KEYWORDS = [
74         "购前须知", "关于签收", "关于售后", "售后问题", "退换货", "包材",
75         "开箱视频", "通电验机", "屏幕碎裂", "外观破损", "缺少附件",
76         "在线客服", "24小时", "驳回申请", "包装", "保留3个月",
77         "影响退换货", "无理由退货", "质保期", "三包服务", "完整退回",
78         "附件缺失", "全国联保", "服务网点", "售后", "服务时间",
79         "易碎品", "屏幕漏液", "支架配件", "包装清单",
80         "扫码", "关注公众号", "联系客服", "客服", "二维码", "扫描",
81         "直播间", "官网地址", "扫码下单", "公众号",
82         "购买后", "超过15天", "一键申请", "快速处理",
83         "下拉详情", "下拉查看", "详情页查看", "包装附件",
84     ]
85
86     # 次要负面关键词（严格筛选时排除，兜底时可以接受）
87     SECONDARY_NEGATIVE_KEYWORDS = [
88         "中国能效标识", "china energy label", "gb21520",
89         "耗能", "能源效率", "关闭状态功率", "睡眠状态功率",

```

```

90     "生产者名称", "规格型号", "lcd显示器", "暗夜武士", "扬声器",
91     "第一步", "点击此处", "安装指南", "使用前阅读", "操作步骤",
92     "使用说明", "安装步骤",
93     "图片仅供参考", "以实物为准", "颜色差异", "批次差异",
94     "收到的实物为准", "个体差异", "软件版本", "使用条件",
95     "环境因素", "产品信息仅供参考", "请以实际为准",
96     "超频刷新率", "原生刷新率",
97     "Δe", "色彩准确度", "dic技术", "dynamic image", "动态清晰度",
98     "减少拖影", "开启条件", "freesync", "hdr", "实验室数据",
99     "有可能改进", "具体信息参照实物",
100     # 品牌宣传类 (方案1)
101     "品牌后盾", "公司成立于", "总资产", "员工总数", "占地面积",
102     "国家及地区", "均有销售", "智能显示科技园区", "业内不超过",
103     "宣宾智造", "显未见", "盛色智造", "top5", "品牌销量",
104     "万平方米", "亿元", "万台", "年质保", "质保", "保修",
105
106 ]
107
108 # 所有负面关键词 (用于严格筛选)
109 NEGATIVE_KEYWORDS = CRITICAL_NEGATIVE_KEYWORDS + SECONDARY_NEGATIVE_KEYWORDS
110
111 CLEANED_NEGATIVE_KEYWORDS = [clean_text(kw) for kw in NEGATIVE_KEYWORDS]
112
113 def has_negative_keywords(ocr_text, debug=False):
114     cleaned_text = clean_text(ocr_text)
115
116     if debug:
117         print(f"🔍 检查负面关键词, 原始文本: {ocr_text[:150]}...")
118         print(f"🧼 清洗后文本: {cleaned_text[:150]}...")
119
120     for kw in CLEANED_NEGATIVE_KEYWORDS:
121         if kw in cleaned_text:
122             if debug:
123                 print(f"❌ 发现负面关键词: '{kw}' 在文本中")
124             return True, kw
125
126     return False, None
127
128 def is_energy_efficiency_image(ocr_text):
129     energy_keywords = ["中国能效标识", "china energy label", "gb21520"]
130     cleaned_text = clean_text(ocr_text)
131     return any(clean_text(kw) in cleaned_text for kw in energy_keywords)
132
133 def has_critical_negative_keywords(ocr_text):
134     """检查是否包含核心负面关键词 (售后等绝对不能出现的)"""
135     cleaned_text = clean_text(ocr_text)
136     cleaned_critical = [clean_text(kw) for kw in CRITICAL_NEGATIVE_KEYWORDS]

```

```

137     for kw in cleaned_critical:
138         if kw in cleaned_text:
139             return True, kw
140     return False, None
141
142 def select_best_main_image(all_images, img_ocr_dict, folder_path):
143     """选择最佳主图，可以从'主图_'或'详情_'开头的图片中选择"""
144     all_images_lower = [img.lower() for img in all_images]
145     img_map = {img.lower(): img for img in all_images}
146
147     # 优先选择"主图_"开头的图片，如果没有则选择"详情_"开头的图片
148     main_candidates = [img for img in all_images_lower if img.startswith("主图
149 _")]
150     if not main_candidates:
151         main_candidates = [img for img in all_images_lower if
152 img.startswith("详情_")]
153     if not main_candidates:
154         print("✗ 没有找到'主图_'或'详情_'开头的图片")
155         return None
156
157     valid_candidates = []
158     for img in main_candidates:
159         img_path = os.path.join(folder_path, img_map[img])
160         ocr_text = img_ocr_dict.get(img, "")
161
162         # 1. 排除参数说明类图片
163         parameter_keywords = [
164             "技术指标"
165         ]
166         cleaned_ocr = clean_text(ocr_text)
167         has_parameters = any(clean_text(kw) in cleaned_ocr for kw in
168 parameter_keywords)
169         if has_parameters:
170             print(f"✗ 排除主图候选 {img_map[img]}: 含过多技术参数文字")
171             continue
172
173         # 2. 排除色彩单调的图片（基于色彩丰富度）
174         try:
175             with Image.open(img_path) as img_pil:
176                 img_rgb = img_pil.convert("RGB")
177                 histogram = img_rgb.histogram()
178                 color_diversity = sum(1 for val in histogram if val > 0)
179
180                 if color_diversity < 60:
181                     print(f"✗ 排除主图候选 {img_map[img]}: 色彩过于单调（丰富度
182 低）")
183                     continue

```



```

180     except Exception as e:
181         print(f"⚠️ 分析图片 {img_map[img]} 色彩时出错: {e}, 跳过色彩筛选")
182
183     # 3. 排除产品主体占比过低的图片 (如大量留白、主体过小)
184     try:
185         with Image.open(img_path) as img_pil:
186             img_gray = img_pil.convert("L")
187             pixels = list(img_gray.getdata())
188             non_white_pixels = sum(1 for p in pixels if p < 240)
189             total_pixels = len(pixels)
190             if total_pixels > 0:
191                 subject_ratio = non_white_pixels / total_pixels
192                 if subject_ratio < 0.3:
193                     print(f"❌ 排除主图候选 {img_map[img]}: 产品主体占比过低
194                           ({subject_ratio:.2%})")
195                     continue
196             except Exception as e:
197                 print(f"⚠️ 分析图片 {img_map[img]} 主体占比时出错: {e}, 跳过该筛选")
198
199     # 4. 排除色彩分布过于均匀的图片 (如单一角度侧面图)
200     try:
201         with Image.open(img_path) as img_pil:
202             img_rgb = img_pil.convert("RGB")
203             pixels = list(img_rgb.getdata())
204             sample_size = min(10000, len(pixels))
205             sampled_pixels = np.random.choice(pixels, sample_size)
206
207             r_vals = [p[0] for p in sampled_pixels]
208             g_vals = [p[1] for p in sampled_pixels]
209             b_vals = [p[2] for p in sampled_pixels]
210
211             r_std = np.std(r_vals)
212             g_std = np.std(g_vals)
213             b_std = np.std(b_vals)
214             avg_std = (r_std + g_std + b_std) / 3
215
216             if avg_std < 15:
217                 print(f"❌ 排除主图候选 {img_map[img]}: 色彩分布过于均匀 (标准
218                       差: {avg_std:.2f}) ")
219                 continue
220             except Exception as e:
221                 print(f"⚠️ 分析图片 {img_map[img]} 色彩分布时出错: {e}, 跳过该筛选")
222
223     # 5. 检查图片尺寸和比例: 排除极窄高、极宽扁和750×1499比例
224     try:
225         width, height, ratio, is_target_ratio =
226         get_image_dimensions_and_ratio(img_path)

```

```
224         if ratio < 0.5:
225             print(f"❌ 排除主图候选 {img_map[img]}: 极窄高比例 (宽高比:
{ratio:.2f})")
226             continue
227         if ratio > 2.0:
228             print(f"❌ 排除主图候选 {img_map[img]}: 极宽扁比例 (宽高比:
{ratio:.2f})")
229             continue
230         if is_target_ratio:
231             print(f"❌ 排除主图候选 {img_map[img]}: 750×1499比例 ({width}
×{height}) ")
232             continue
233     except Exception as e:
234         print(f"⚠️ 分析图片 {img_map[img]} 比例时出错: {e}, 跳过该筛选")
235
236     # 6. 检查OCR内容: 能效标识和负面关键词
237     if is_energy_efficiency_image(ocr_text):
238         print(f"❌ 排除主图候选 {img_map[img]}: 包含能效标识")
239         continue
240
241     has_negative, matched_kw = has_negative_keywords(ocr_text, debug=True)
242     if has_negative:
243         print(f"❌ 排除主图候选 {img_map[img]}: 包含负面关键词
'{matched_kw}'")
244         continue
245
246     text_length = count_valid_text(ocr_text)
247     if text_length <= 50:
248         valid_candidates.append((img, text_length))
249
250     if valid_candidates:
251         valid_candidates.sort(key=lambda x: x[1])
252         return img_map[valid_candidates[0][0]]
253     else:
254         print(f"⚠️ 无完全符合条件的主图, 使用宽松筛选 (只排除负面关键词和异常比例)")
255         # 宽松筛选: 至少要排除负面关键词、能效标识和异常比例
256         loose_main_candidates = []
257         for img in main_candidates:
258             img_path = os.path.join(folder_path, img_map[img])
259             ocr_text = img_ocr_dict.get(img, "")
260
261             # 检查宽高比
262             try:
263                 width, height, ratio, is_target_ratio =
get_image_dimensions_and_ratio(img_path)
264                 if ratio < 0.5 or ratio > 2.0 or is_target_ratio:
265                     continue
```

```

266         except:
267             pass
268
269         # 检查负面关键词
270         if is_energy_efficiency_image(ocr_text):
271             continue
272         has_negative, _ = has_negative_keywords(ocr_text)
273         if has_negative:
274             continue
275
276         text_length = count_valid_text(ocr_text)
277         loose_main_candidates.append((img, text_length))
278
279     if loose_main_candidates:
280         loose_main_candidates.sort(key=lambda x: x[1])
281         print(f"✅ 宽松筛选选中主图: {img_map[loose_main_candidates[0][0]]}")
282         return img_map[loose_main_candidates[0][0]]
283     else:
284         print("⚠️ 宽松筛选也无结果, 启动兜底机制 (只排除核心负面关键词) ")
285         # 兜底机制: 只排除核心负面关键词和能效标识
286         fallback_candidates = []
287         for img in main_candidates:
288             img_path = os.path.join(folder_path, img_map[img])
289             ocr_text = img_ocr_dict.get(img, "")
290
291             # 只检查核心负面关键词
292             if is_energy_efficiency_image(ocr_text):
293                 continue
294             has_critical, critical_kw =
has_critical_negative_keywords(ocr_text)
295             if has_critical:
296                 print(f"⚠️ 兜底排除 {img_map[img]}: 包含核心负面关键词
'{critical_kw}'")
297                 continue
298
299             text_length = count_valid_text(ocr_text)
300             fallback_candidates.append((img, text_length))
301
302         if fallback_candidates:
303             fallback_candidates.sort(key=lambda x: x[1])
304             print(f"✅ 兜底机制选中主图: {img_map[fallback_candidates[0]
[0]]}")
305             return img_map[fallback_candidates[0][0]]
306         else:
307             print("❌ 兜底机制也无法找到主图 (所有图片都包含核心负面关键词) ")
308             return None
309

```

```

310 def select_best_detail_image(all_images, img_ocr_dict, excluded_main_img,
311                               folder_path):
312     """选择最佳详情图，优先从'详情_'开头的图片中选择，实在没有则从主图候选中选择"""
313     all_images_lower = [img.lower() for img in all_images]
314     img_map = {img.lower(): img for img in all_images}
315     excluded_lower = excluded_main_img.lower() if excluded_main_img else None
316
317     # 优先选择"详情_"开头的图片（排除已选的主图）
318     detail_candidates = [
319         img for img in all_images_lower
320         if img.startswith("详情_") and img != excluded_lower
321     ]
322
323     # 如果没有详情图候选，则从主图候选中选择（排除已选的主图）
324     if not detail_candidates:
325         print("⚠️ 没有找到'详情_'开头的图片，从'主图_'候选中选择详情图")
326         detail_candidates = [
327             img for img in all_images_lower
328             if img.startswith("主图_") and img != excluded_lower
329         ]
330         if not detail_candidates:
331             print("❌ 也没有找到'主图_'开头的图片")
332             return None
333
334     valid_candidates = []
335     for img in detail_candidates:
336         img_full_path = os.path.join(folder_path, img_map[img])
337
338         # 检查图片尺寸和比例：排除750×1499比例、极窄高和极宽扁比例
339         width, height, ratio, is_target_ratio =
340         get_image_dimensions_and_ratio(img_full_path)
341         if is_target_ratio:
342             print(f"❌ 排除详情候选 {img_map[img]}: 750×1499比例 ({width}
343             ×{height}) ")
344             continue
345
346         if ratio < 0.7:
347             print(f"❌ 排除详情候选 {img_map[img]}: 极窄高比例 (宽高比:
348             {ratio:.2f})")
349             continue
350
351         if ratio > 2.0:
352             print(f"❌ 排除详情候选 {img_map[img]}: 极宽扁比例 (宽高比:
353             {ratio:.2f})")
354             continue
355
356         ocr_text = img_ocr_dict.get(img, "")

```

```

352
353     if is_energy_efficiency_image(ocr_text):
354         print(f"❌ 排除详情候选 {img_map[img]}: 包含能效标识")
355         continue
356
357     has_negative, matched_kw = has_negative_keywords(ocr_text, debug=True)
358     if has_negative:
359         print(f"❌ 排除详情候选 {img_map[img]}: 包含负面关键词
360 '{matched_kw}'")
361         continue
362
363     text_length = count_valid_text(ocr_text)
364     if 30 <= text_length <= 100:
365         valid_candidates.append((img, text_length, ratio))
366
367     if not valid_candidates:
368         print("⚠️ 无完全符合条件的详情图，使用宽松筛选（忽略文字长度要求，但仍检查负面关键词和比例）")
369
370     loose_candidates = []
371     for img in detail_candidates:
372         img_full_path = os.path.join(folder_path, img_map[img])
373
374         # 仍然要检查宽高比（宽松阈值0.5-2.0）
375         width, height, ratio, is_target_ratio =
376         get_image_dimensions_and_ratio(img_full_path)
377         if is_target_ratio or ratio < 0.5 or ratio > 2.0:
378             continue
379
380         ocr_text = img_ocr_dict.get(img, "")
381
382         # 仍然要排除能效标识和负面关键词
383         if is_energy_efficiency_image(ocr_text):
384             continue
385         if has_negative_keywords(ocr_text)[0]:
386             continue
387
388         text_length = count_valid_text(ocr_text)
389         loose_candidates.append((img, text_length, ratio))
390
391     if loose_candidates:
392         # 优先选择宽高比更接近1.0的图片（更方正好看），其次考虑文字长度
393         loose_candidates.sort(key=lambda x: (abs(x[2] - 1.0), abs(x[1] -
394 50)))
395
396     print(f"✅ 宽松筛选选中详情图: {img_map[loose_candidates[0][0]]} (宽高
397 比: {loose_candidates[0][2]:.2f})")
398     return img_map[loose_candidates[0][0]]
399 else:

```

```

394         print("⚠️ 宽松筛选也无结果，启动兜底机制（只排除核心负面关键词和极端比
例）")
395         # 兜底机制：只排除核心负面关键词、能效标识和极端比例，允许750×1499
396         fallback_candidates = []
397         for img in detail_candidates:
398             img_full_path = os.path.join(folder_path, img_map[img])
399             ocr_text = img_ocr_dict.get(img, "")
400
401             # 检查极端比例（比例范围放宽到0.5-5.0，允许750×1499）
402             width, height, ratio, is_target_ratio =
get_image_dimensions_and_ratio(img_full_path)
403             if ratio < 0.5 or ratio > 5.0: # 只排除极端比例，允许750×1499
404                 continue
405
406             # 只检查核心负面关键词
407             if is_energy_efficiency_image(ocr_text):
408                 continue
409             has_critical, critical_kw =
has_critical_negative_keywords(ocr_text)
410             if has_critical:
411                 print(f"⚠️ 兜底排除 {img_map[img]}：包含核心负面关键词
'{critical_kw}'")
412                 continue
413
414             text_length = count_valid_text(ocr_text)
415             fallback_candidates.append((img, text_length))
416
417         if fallback_candidates:
418             fallback_candidates.sort(key=lambda x: x[1])
419             print(f"✅ 兜底机制选中详情图：{img_map[fallback_candidates[0]
[0]]}（文字长度：{fallback_candidates[0][1]}）")
420             return img_map[fallback_candidates[0][0]]
421         else:
422             print("⚠️ 详情图候选都不符合要求，尝试从主图中选择")
423             # 最后兜底：从主图中选择（排除已选的主图）
424             main_candidates = [
425                 img for img in all_images_lower
426                 if img.startswith("主图_") and img != excluded_lower
427             ]
428
429             if not main_candidates:
430                 print("❌ 也没有可用的主图")
431                 return None
432
433             # 从主图中选择最佳的作为详情图
434             main_fallback_candidates = []
435             for img in main_candidates:

```

```

436         img_full_path = os.path.join(folder_path, img_map[img])
437         ocr_text = img_ocr_dict.get(img, "")
438
439         # 检查宽高比和核心负面关键词
440         width, height, ratio, is_target_ratio =
get_image_dimensions_and_ratio(img_full_path)
441         if ratio < 0.5 or ratio > 5.0:
442             continue
443
444         if is_energy_efficiency_image(ocr_text):
445             continue
446         has_critical, critical_kw =
has_critical_negative_keywords(ocr_text)
447         if has_critical:
448             continue
449
450         main_fallback_candidates.append((img, ratio))
451
452         if main_fallback_candidates:
453             # 优先选择宽高比接近1.0的主图
454             main_fallback_candidates.sort(key=lambda x: abs(x[1] -
1.0))
455             print(f"✅ 从主图中选择详情图:
{img_map[main_fallback_candidates[0][0]]} (宽高比: {main_fallback_candidates[0]
[1]:.2f})")
456             return img_map[main_fallback_candidates[0][0]]
457         else:
458             print("❌ 所有主图也不符合要求")
459             return None
460
461         # 如果有符合条件的候选, 优先选择宽高比接近1.0的 (更方正好看)
462         if valid_candidates:
463             valid_candidates.sort(key=lambda x: (abs(x[2] - 1.0), abs(x[1] - 50)))
464             print(f"✅ 严格筛选选中详情图: {img_map[valid_candidates[0][0]]} (宽高比:
{valid_candidates[0][2]:.2f})")
465             return img_map[valid_candidates[0][0]]
466         else:
467             print("⚠️ 没有找到符合条件的'详情_ '图片")
468             return None
469
470 def process_image_folders(source_root, ppt_root, excel_path, ocr_txt_root):
471     os.makedirs(ppt_root, exist_ok=True)
472     xls = pd.ExcelFile(excel_path)
473
474     processing_results = {}
475
476     for folder_name in os.listdir(source_root):

```

```
477     folder_path = os.path.join(source_root, folder_name)
478     if not os.path.isdir(folder_path):
479         continue
480
481     print(f"\n{'='*40}\n开始处理文件夹: {folder_name}")
482
483     if '_' not in folder_name:
484         print(f"✗ 文件夹命名错误(缺少下划线): {folder_name}, 跳过")
485         continue
486     sheet_name, product_name = folder_name.split('_', 1)
487
488     if sheet_name not in xls.sheet_names:
489         print(f"✗ Excel中无对应sheet: {sheet_name}, 跳过")
490         continue
491     df = xls.parse(sheet_name)
492     if '商品名称' not in df.columns:
493         print(f"✗ sheet {sheet_name} 中无'商品名称'列, 跳过")
494         continue
495
496     # 先将_替换为/, 再去除特殊符号, 确保匹配一致性
497     normalized_product_name = product_name.replace('_', '/')
498     sanitized_product = sanitize_filename(normalized_product_name)
499
500     matched_row = None
501     for idx, row in df.iterrows():
502         excel_product = sanitize_filename(str(row['商品名称']))
503         # 匹配逻辑: 子串包含即可
504         if sanitized_product in excel_product or excel_product in
sanitized_product:
505             matched_row = idx + 2
506             break
507     if matched_row is None:
508         print(f"✗ 未匹配到商品: {product_name}, 跳过")
509         continue
510     image_number = matched_row - 1
511
512     all_images = [
513         f for f in os.listdir(folder_path)
514         if f.lower().endswith(('.jpg', '.jpeg', '.png', '.gif'))
515     ]
516
517     if len(all_images) < 2:
518         print(f"✗ 图片数量不足2张 (仅{len(all_images)}张) , 跳过")
519         continue
520
521     ocr_txt_filename = f"{folder_name}.txt"
522     ocr_txt_path = os.path.join(ocr_txt_root, ocr_txt_filename)
```



```
523     img_ocr_dict = {}
524     if os.path.exists(ocr_txt_path):
525         with open(ocr_txt_path, 'r', encoding='utf-8') as f:
526             ocr_content = f.read()
527             img_ocr_dict = parse_ocr_collection(ocr_content)
528     else:
529         print(f"⚠️ OCR文件缺失: {ocr_txt_path}, 仅按文件名和比例筛选")
530
531     selected_main = select_best_main_image(all_images, img_ocr_dict,
532     folder_path)
533
534     selected_detail = None
535     if selected_main:
536         selected_detail = select_best_detail_image(all_images,
537         img_ocr_dict, selected_main, folder_path)
538
539     # 检查是否成功选择了主图和详情图
540     if not selected_main or not selected_detail:
541         print(f"⚠️ 跳过文件夹 {folder_name}: 没有找到合适的主图或详情图")
542         continue
543
544     # 记录处理结果
545     processing_results[image_number] = {
546         'folder_name': folder_name,
547         'sheet_name': sheet_name,
548         'all_images': all_images,
549         'img_ocr_dict': img_ocr_dict,
550         'folder_path': folder_path,
551         'selected_main': selected_main,
552         'selected_detail': selected_detail
553     }
554
555     main_target_folder = os.path.join(ppt_root, f"{sheet_name}_图1")
556     detail_target_folder = os.path.join(ppt_root, f"{sheet_name}_图2")
557     os.makedirs(main_target_folder, exist_ok=True)
558     os.makedirs(detail_target_folder, exist_ok=True)
559
560     shutil.copy2(
561         os.path.join(folder_path, selected_main),
562         os.path.join(main_target_folder, f"{image_number}.jpg")
563     )
564     shutil.copy2(
565         os.path.join(folder_path, selected_detail),
566         os.path.join(detail_target_folder, f"{image_number}.jpg")
567     )
```

```

568         print(f"✅ 处理完成：主图={selected_main}（图1文件夹），详情图=
{selected_detail}（图2文件夹），编号={image_number}")
569
570     # 保存编号到源文件夹的映射
571     img_num_to_source_map = {}
572     for folder_name in os.listdir(source_root):
573         folder_path = os.path.join(source_root, folder_name)
574         if not os.path.isdir(folder_path):
575             continue
576
577         if '_' not in folder_name:
578             continue
579
580         sheet_name, product_name = folder_name.split('_', 1)
581
582         try:
583             images = [f for f in os.listdir(folder_path) if
f.lower().endswith(('.jpg', '.jpeg', '.png', '.gif'))]
584             if not images:
585                 continue
586
587             # 从文件夹名称中提取产品名称，然后从 Excel 中查找编号
588             normalized_product_name = product_name.replace('_', '/')
589             sanitized_product = sanitize_filename(normalized_product_name)
590
591             xls_temp = pd.ExcelFile(excel_path)
592             for sheet in xls_temp.sheet_names:
593                 df_temp = xls_temp.parse(sheet)
594                 if '商品名称' not in df_temp.columns:
595                     continue
596
597                 for idx, row in df_temp.iterrows():
598                     excel_product = sanitize_filename(str(row['商品名称']))
599                     if sanitized_product in excel_product or excel_product in
sanitized_product:
600                         img_num = idx + 2 # Excel行号是从2开始（第1行是标题）
601                         img_num_to_source_map[img_num] = source_root
602                         break
603         except:
604             continue
605
606     # 保存映射文件
607     mapping_file = os.path.join(ppt_root, 'img_num_to_source.json')
608     try:
609         with open(mapping_file, 'w', encoding='utf-8') as f:
610             json.dump(img_num_to_source_map, f, ensure_ascii=False, indent=2)
611     except:

```

```
612         pass
613
614     return processing_results
615
616 def reselect_images_interactive(ppt_root, selected_source_folder,
category_excel, output_folder_type="图1"):
617     """交互式重选图片，打开编号对应的产品文件夹让用户手动选择图片"""
618     # 从 PPT 文件夹中提取 sheet_name
619     sheet_name = None
620     for folder_name in os.listdir(ppt_root):
621         if "_图1" in folder_name:
622             sheet_name = folder_name.replace("_图1", "")
623             break
624
625     if not sheet_name:
626         print("✗ 没有找到PPT文件夹中的图片文件夹")
627         return
628
629     # 加载Excel，根据sheet_name加载对应的sheet
630     xls = pd.ExcelFile(category_excel)
631     if sheet_name not in xls.sheet_names:
632         print(f"✗ Excel中没有sheet: {sheet_name}")
633         return
634     df = xls.parse(sheet_name)
635
636     main_target_folder = os.path.join(ppt_root, f"{sheet_name}_图1")
637     detail_target_folder = os.path.join(ppt_root, f"{sheet_name}_图2")
638
639     # 扫描已处理的编号
640     processed_numbers = set()
641     if os.path.exists(main_target_folder):
642         for img_file in os.listdir(main_target_folder):
643             if img_file.lower().endswith(('.jpg', '.jpeg', '.png', '.gif')):
644                 try:
645                     img_num = int(img_file.split('.')[0])
646                     processed_numbers.add(img_num)
647                 except ValueError:
648                     pass
649
650     if not processed_numbers:
651         print("✗ 没有找到之前的处理结果")
652         return
653
654     print("\n" + "="*60)
655     print("📁 已处理的图片编号列表:")
656     for img_num in sorted(processed_numbers):
657         print(f"    编号 {img_num}")
```

```
658
659     while True:
660         user_input = input("\n请输入要重选的图片编号（单个，如 9），或输入 'q' 退出：
661 ").strip()
662         if user_input.lower() == 'q':
663             print("\n✅ 退出重选界面")
664             break
665
666         try:
667             img_num = int(user_input)
668         except ValueError:
669             print("❌ 输入格式错误，请输入数字或 'q'")
670             continue
671
672         if img_num not in processed_numbers:
673             print(f"❌ 编号 {img_num} 不存在")
674             continue
675
676         # 从Excel中查找编号对应的商品名称
677         try:
678             product_name = df.iloc[img_num - 1]['商品名称']
679         except (IndexError, KeyError):
680             print(f"❌ 编号 {img_num} 在 Excel 中不存在")
681             continue
682
683         if not product_name:
684             print(f"❌ 编号 {img_num} 的商品名称为空")
685             continue
686
687         # 先将_替换为/，再去除特殊符号，确保匹配一致性
688         normalized_product_name = product_name.replace('_', '/')
689         sanitized_product = sanitize_filename(normalized_product_name)
690         product_folder = None
691
692         if not os.path.exists(selected_source_folder):
693             print(f"❌ 源文件夹不存在：{selected_source_folder}")
694             continue
695
696         for folder_name in os.listdir(selected_source_folder):
697             folder_path = os.path.join(selected_source_folder, folder_name)
698             if not os.path.isdir(folder_path):
699                 continue
700
701             if '_' not in folder_name:
702                 continue
703
704             # 提取文件夹中的product_name部分
```

```

704         sheet_name_folder, product_name_folder = folder_name.split('_', 1)
705         sanitized_folder_product = sanitize_filename(product_name_folder)
706
707         # 使用原本的匹配逻辑：子串包含即可
708         if sanitized_product in sanitized_folder_product or
sanitized_folder_product in sanitized_product:
709             try:
710                 images = [f for f in os.listdir(folder_path) if
f.lower().endswith(('.jpg', '.jpeg', '.png', '.gif'))]
711                 if images:
712                     product_folder = folder_path
713                     break
714             except:
715                 continue
716
717         if not product_folder:
718             print(f"❌ 编号 {img_num} (商品: {product_name}) 的产品文件夹不存在")
719             continue
720
721         print(f"\n🔄 重选编号 {img_num} 的图片")
722         print(f"📁 正在打开产品文件夹: {product_folder}")
723         print("请选择一个图片文件，然后关闭Windows文件管理器窗口")
724
725         # 打开产品文件夹
726         import subprocess
727         subprocess.Popen(f'explorer "{product_folder}"')
728
729         # 等待用户选择图片
730         selected_file = input("\n请输入选择的图片文件名（不需要路径，仅需要文件名）：
").strip()
731
732         if not selected_file:
733             print(f"❌ 未选择图片")
734             continue
735
736         # 检查文件是否存在
737         selected_file_path = os.path.join(product_folder, selected_file)
738         if not os.path.exists(selected_file_path):
739             print(f"❌ 文件不存在: {selected_file}")
740             continue
741
742         # 覆盖主图和详情图
743         try:
744             # 确保目标文件夹存在
745             os.makedirs(main_target_folder, exist_ok=True)
746             os.makedirs(detail_target_folder, exist_ok=True)
747

```

```

748         # 根据选择只覆盖对应的输出文件夹
749         if output_folder_type == "图1":
750             shutil.copy2(
751                 selected_file_path,
752                 os.path.join(main_target_folder, f"{img_num}.jpg")
753             )
754         else: # 图2
755             shutil.copy2(
756                 selected_file_path,
757                 os.path.join(detail_target_folder, f"{img_num}.jpg")
758             )
759         print(f"\n✅ 已更新编号 {img_num}:")
760         print(f"    图片: {selected_file}")
761         print(f"\n📄 请查看更新后的图片是否满意")
762         print("="*60)
763     except Exception as e:
764         print(f"\n❌ 覆盖图片失败: {str(e)}")
765
766 def write_to_excel(ppt_root, ref_excel_path):
767     for folder_name in os.listdir(ppt_root):
768         folder_path = os.path.join(ppt_root, folder_name)
769         if not os.path.isdir(folder_path) or "_图" not in folder_name:
770             continue
771
772         images = []
773         for f in os.listdir(folder_path):
774             if f.lower().endswith(('.jpg', '.jpeg', '.png', '.gif')):
775                 try:
776                     img_num = int(f.split('.')[0])
777                     images.append(img_num)
778                 except ValueError:
779                     print(f"⚠️ 图片 {f} 命名不符合数字格式, 已跳过")
780
781         images.sort()
782         if not images:
783             print(f"❌ 文件夹 {folder_name} 中无有效图片, 跳过Excel生成")
784             continue
785
786         excel_path = os.path.join(folder_path, f"{folder_name}.xlsx")
787         shutil.copy2(ref_excel_path, excel_path)
788
789         wb = load_workbook(excel_path)
790         ws = wb.active
791
792         if ws.max_column > 2:
793             for col in range(ws.max_column, 2, -1):
794                 ws.delete_cols(col)

```

```

795
796     ws.cell(row=2, column=2).value = "图片_1"
797
798     for i, img_num in enumerate(images):
799         row = i + 3
800         ws.cell(row=row, column=1).value = f"页面{i+1}"
801         ws.cell(row=row, column=2).value = str(img_num)
802
803     wb.save(excel_path)
804     print(f"

```

```

840         excel_folder = f"{sheet_name}{suffix}"
841         excel_path = os.path.join(ppt_root, excel_folder, f"
{excel_folder}.xlsx")
842         if not os.path.exists(excel_path):
843             print(f"✗ 未找到Excel文件: {excel_path}, 跳过")
844             continue
845
846         wb = load_workbook(excel_path)
847         ws = wb.active
848         for row in range(3, ws.max_row + 1):
849             cell_val = ws.cell(row=row, column=2).value
850             if cell_val and str(cell_val) == str(target_number):
851                 ws.cell(row=row, column=2).value = ""
852         wb.save(excel_path)
853         print(f"🔄 更新Excel {excel_folder}.xlsx: 清空编号 {target_number}")
854
855     def load_previous_results(ppt_root, selected_source_folder, category_excel):
856         """从已有的PPT文件夹中加载已处理的编号"""
857         processing_results = {}
858         if not os.path.exists(ppt_root):
859             return processing_results
860
861         # 加载 Excel, 从编号查找商品名称
862         xls = pd.ExcelFile(category_excel)
863         sheet_name = xls.sheet_names[0]
864         df = xls.parse(sheet_name)
865
866         # 优先加载保存的源文件夹映射
867         img_num_to_source_map = {}
868         mapping_file = os.path.join(ppt_root, 'img_num_to_source.json')
869         if os.path.exists(mapping_file):
870             try:
871                 with open(mapping_file, 'r', encoding='utf-8') as f:
872                     img_num_to_source_map = json.load(f)
873                     # 转换为整数键
874                     img_num_to_source_map = {int(k): v for k, v in
img_num_to_source_map.items()}
875             except:
876                 pass
877
878         # 扫描所有可用的源文件夹中的产品文件夹
879         param_root = os.path.join(base_path, "参数")
880         img_num_to_folder_map = {}
881
882         for sub_folder in ['详情图片', '主图图片']:
883             source_folder = os.path.join(param_root, sub_folder)
884             if not os.path.exists(source_folder):

```



```

885         continue
886
887     for folder_name in os.listdir(source_folder):
888         folder_path = os.path.join(source_folder, folder_name)
889         if not os.path.isdir(folder_path):
890             continue
891
892         # 文件夹格式: sheet_name_product_name
893         if '_' not in folder_name:
894             continue
895
896         # 检查是否有图片
897         try:
898             images = [f for f in os.listdir(folder_path) if
899 f.lower().endswith(('.jpg', '.jpeg', '.png', '.gif'))]
900             if images:
901                 img_num_to_folder_map[folder_name] = folder_path
902         except:
903             continue
904
905     # 从 PPT 文件夹中读取已处理的编号
906     for folder_name in os.listdir(ppt_root):
907         if "_图1" in folder_name:
908             sheet_name_ppt = folder_name.replace("_图1", "")
909             # 从文件夹名称提取信息
910             for img_file in os.listdir(os.path.join(ppt_root, folder_name)):
911                 if img_file.lower().endswith(('.jpg', '.jpeg', '.png',
912 '.gif')):
913                     try:
914                         img_num = int(img_file.split('.')[0])
915                         if img_num not in processing_results:
916                             # 从 Excel 中查找编号对应的商品名称
917                             product_name = None
918                             if '商品名称' in df.columns:
919                                 try:
920                                     product_name = df.iloc[img_num - 1]['商品名
921 称']
922                                 except:
923                                     pass
924
925                             # 找到对应的产品文件夹
926                             product_folder = None
927                             if product_name:
928                                 # 洗治商品名称 (移除特殊符号)
929                                 sanitized_product =
930 sanitize_filename(str(product_name))

```

```

927         print(f"\n🔍 [调试] 编号 {img_num}: 商品名称
    ='{product_name}''")
928         print(f"    洗浴后: '{sanitized_product}''")
929         print(f"    扫描的文件夹数量:
    {len(img_num_to_folder_map)}")
930
931         for folder_name_src, folder_path_src in
    img_num_to_folder_map.items():
932             # 洗浴文件夹名称
933             sanitized_folder =
    sanitize_filename(folder_name_src)
934
935             # 检查文件夹名是否匹配产品名称
936             if sanitized_product in sanitized_folder or
    sanitized_folder in sanitized_product:
937                 print(f"    ✅ 匹配成功:
    {folder_name_src}")
938                 product_folder = folder_path_src
939                 break
940
941             if not product_folder:
942                 print(f"    ❌ 没有找到匹配的文件夹")
943                 print(f"    所有可用的文件夹
    ({len(img_num_to_folder_map)} 个):")
944                 for i, folder_name_src in
    enumerate(list(img_num_to_folder_map.keys())[:10]):
945                     print(f"        {i+1}. {folder_name_src}")
946
947                 processing_results[img_num] = {
948                     'sheet_name': sheet_name_ppt,
949                     'folder_name': folder_name,
950                     'all_images': [],
951                     'img_ocr_dict': {},
952                     'folder_path': product_folder,
953                     'selected_main': None,
954                     'selected_detail': None
955                 }
956             except ValueError:
957                 pass
958
959         return processing_results
960
961 if __name__ == "__main__":
962     print("\n" + "="*60)
963     print("📄 选择模式")
964     print("="*60)
965     print("1. 直接运行 - 自动处理所有图片，然后进入重选界面")

```

```

966     print("2. 输入查找 - 选择文件夹, 然后进入重选界面")
967     print("="*60)
968
969     while True:
970         choice = input("\n请选择 (1 或 2): ").strip()
971         if choice in ["1", "2"]:
972             break
973         print("❌ 无效的选择, 请输入 1 或 2")
974
975     # 根据选择设置基本路径, 修改下面这个红色的路径就好了
976     base_path = r"E:\B站\选品\批量\暖冬好物1- 冲锋衣"
977     video_folder = os.path.join(base_path, "视频")
978     ppt_folder = os.path.join(base_path, "PPT图片批量导入")
979     category_excel = os.path.join(base_path, "分类.xlsx")
980     ref_excel = os.path.join(base_path, "1-数据上传.xlsx")
981
982     # 模式 1: 直接运行
983     if choice == "1":
984         source_folder = os.path.join(base_path, "参数", "详情图片")
985         ocr_txt_root = os.path.join(base_path, "参数", "详情图txt")
986         selected_source_folder = source_folder
987
988         print("\n✅ 开始直接运行...")
989         print("="*60)
990         print("开始处理图片...")
991         processing_results = process_image_folders(source_folder, ppt_folder,
category_excel, ocr_txt_root)
992
993         if processing_results:
994             print("\n" + "="*60)
995             print("开始生成Excel...")
996             write_to_excel(ppt_folder, ref_excel)
997
998             # 交互式重选
999             reselect_images_interactive(ppt_folder, source_folder,
category_excel)
1000
1001             # 重新生成Excel (如果有重选)
1002             print("\n" + "="*60)
1003             print("重新生成Excel...")
1004             write_to_excel(ppt_folder, ref_excel)
1005
1006             print("\n" + "="*60)
1007             print("开始处理视频...")
1008             process_videos_and_update_excel(video_folder, ppt_folder,
category_excel)
1009

```

```

1010     print("\n" + "="*60)
1011     print("所有操作完成! ")
1012
1013     elif choice == "2":
1014         print("\n✅ 开始输入查找...")
1015         print("="*60)
1016         print("📁 选择源文件夹")
1017         print("="*60)
1018         print("1. 参数/详情图片")
1019         print("2. 参数/主图图片")
1020         print("="*60)
1021
1022         # 输入文件夹固定为详情图片
1023         selected_source_folder = os.path.join(base_path, "参数", "详情图片")
1024
1025         # 选择输出文件夹类型
1026         print("="*60)
1027         print("📁 选择要重选的输出文件夹")
1028         print("="*60)
1029         print("1. 图1 (主图) ")
1030         print("2. 图2 (详情图片) ")
1031         print("="*60)
1032
1033         while True:
1034             folder_choice = input("\n请选择 (1 或 2): ").strip()
1035             if folder_choice == "1":
1036                 output_folder_type = "图1"
1037                 break
1038             elif folder_choice == "2":
1039                 output_folder_type = "图2"
1040                 break
1041             else:
1042                 print("❌ 无效的选择, 请输入 1 或 2")
1043
1044         print(f"\n✅ 已选择: {output_folder_type}")
1045
1046         # 交互式重选
1047         reselect_images_interactive(ppt_folder, selected_source_folder,
category_excel, output_folder_type)
1048
1049         # 重新生成Excel
1050         print("\n" + "="*60)
1051         print("重新生成Excel...")
1052         write_to_excel(ppt_folder, ref_excel)
1053
1054         print("\n" + "="*60)
1055         print("所有操作完成! ")

```

```
# 根据选择设置基本路径，修改下面这个红色的路径就好了
base_path = r"E:\B站\选品\批量\暖冬好物1- 冲锋衣"
video_folder = os.path.join(base_path, "视频")
ppt_folder = os.path.join(base_path, "PPT图片批量导入")
category_excel = os.path.join(base_path, "分类.xlsx")
ref_excel = os.path.join(base_path, "1-数据上传.xlsx")
```

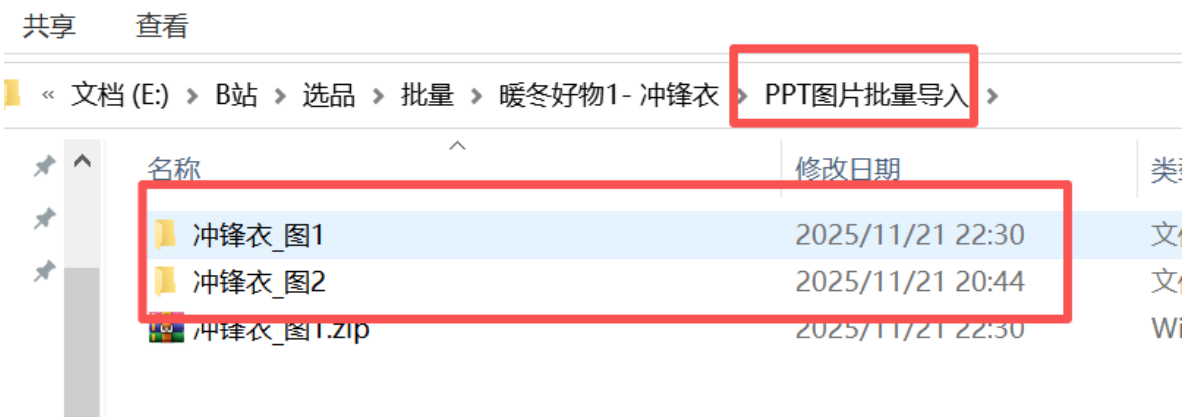
使用方法：

安装完基本的模块后，修改下面的红色路径。

还需要把 1-数据上传.xlsx 1-数据上传.xlsx 放到文件夹下面。

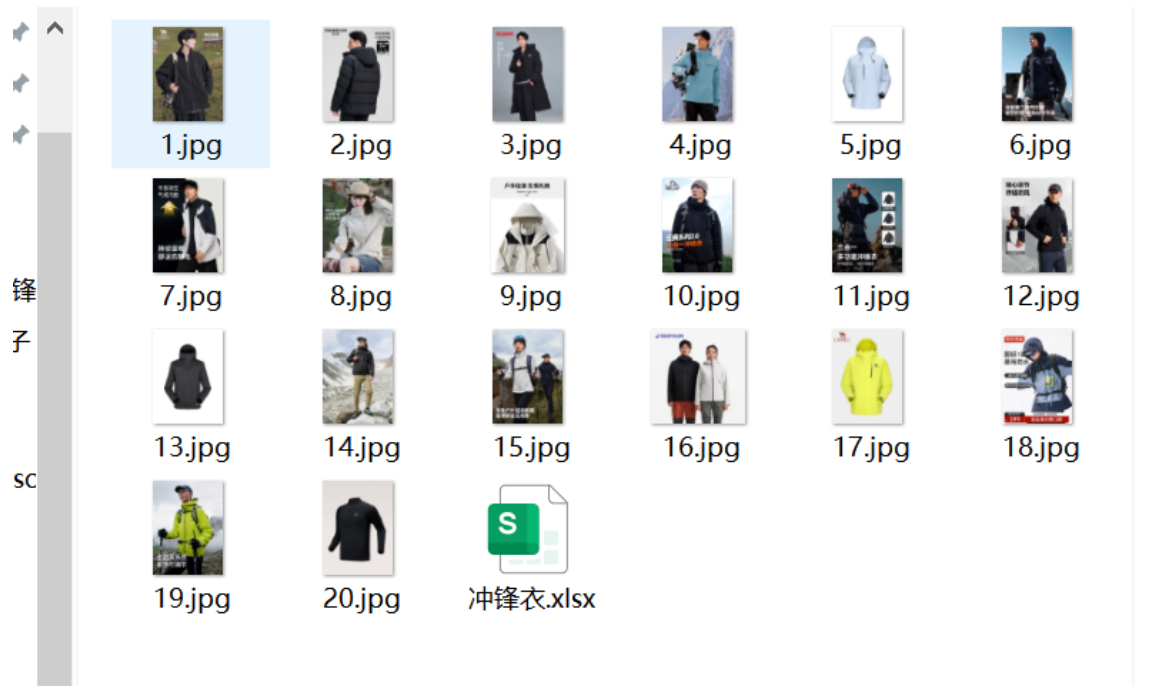
运行代码，会得到如下的结果，图1是主图，图2是详情图片（不用管）

如果主图里面有不符合的，代码里面有切换的功能。



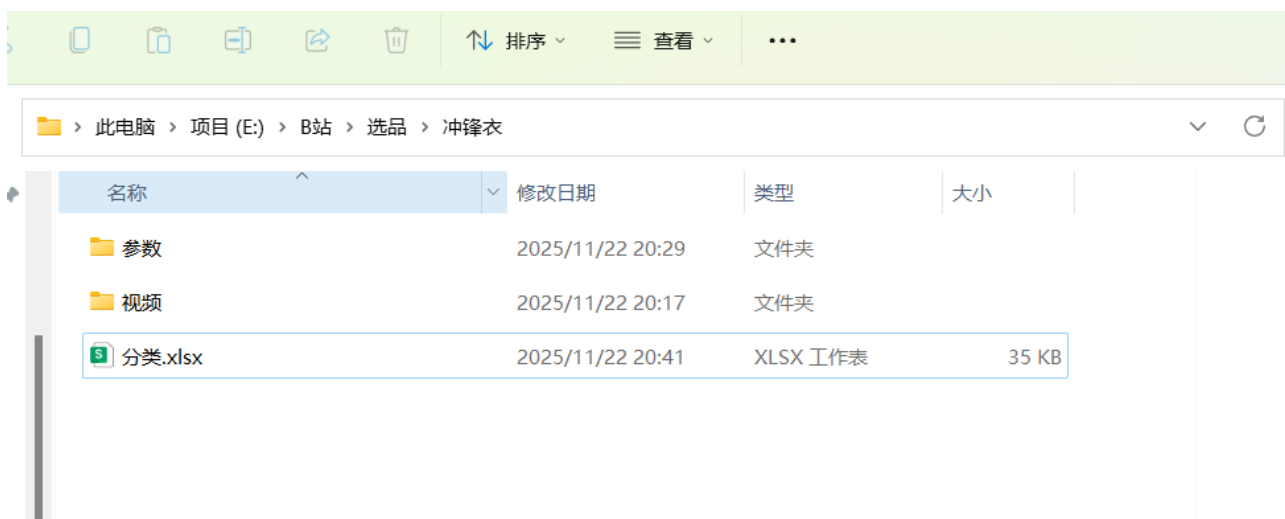
得到主图后，找到3.1批量套版得到的Excel，先把这个Excel放到主图文件夹中，如图中的**冲锋衣\_图1**，注意**主图文件夹**中会有一个代码生成的Excel，可以直接删掉。

然后再编辑Excel，在里面加入 图片\_1这一列。图片\_1这一列下面的数字填写为1,2,3,4等，和文件夹中的图片命名一致即可。接下来就是把 冲锋衣\_图1这个文件夹 压缩成zip文件，导入批量套版即可。



### 3.3 broll 素材提效

还是在我的方法基础上，影刀代码生成的参数文件夹里面，有几个子文件夹，有些为空，因为这是其他影刀代码如2.3的批量获取产品参数会用到的。我现在基本上都只用2.5的影刀代码生成。





写了一个代码，可以在参数文件夹的基础上，对应每一句话得到对应的图片。这样Broll的一部分素材就有了。

代码如下：

拆句子gui代码

```
1 import os
2 import re
3 import json
4 import subprocess
5 from pathlib import Path
6 from typing import List, Dict, Tuple, Optional
7 import tkinter as tk
8 from tkinter import ttk, filedialog, messagebox
9 from PIL import Image, ImageTk
10 import difflib
11 from collections import defaultdict
12 import jieba
13 try:
14     import win32clipboard
15     HAS_WIN32 = True
16 except ImportError:
17     HAS_WIN32 = False
18 import io
19
20 class SentenceImageMatcher:
```

```

21     """句子与图片匹配引擎"""
22
23     def __init__(self, param_folder_path: str):
24         self.param_folder = Path(param_folder_path)
25         self.txt_folder = self.param_folder / "详情图txt"
26         self.img_folder = self.param_folder / "详情图片"
27
28         self.txt_data = {} # {产品名: {图片名: [ocr文字列表]}}
29         self.img_paths = {} # {产品名: {图片名: 图片路径}}
30
31         self._load_data()
32
33     def _load_data(self):
34         """加载txt和图片数据"""
35         if not self.txt_folder.exists():
36             raise ValueError(f"详情图txt文件夹不存在: {self.txt_folder}")
37         if not self.img_folder.exists():
38             raise ValueError(f"详情图片文件夹不存在: {self.img_folder}")
39
40         # 加载txt文件
41         for txt_file in self.txt_folder.glob("*.txt"):
42             product_name = self._extract_product_name(txt_file.name)
43             self.txt_data[product_name] = self._parse_txt_file(txt_file)
44
45         # 加载图片文件夹
46         for img_dir in self.img_folder.iterdir():
47             if img_dir.is_dir():
48                 product_name = self._extract_product_name(img_dir.name)
49                 self.img_paths[product_name] = {}
50                 for img_file in img_dir.glob("*"):
51                     if img_file.suffix.lower() in ['.jpg', '.jpeg', '.png',
52 '.gif', '.bmp']:
53                         img_name = img_file.stem # 不含扩展名
54                         self.img_paths[product_name][img_name] = str(img_file)
55
56     def _extract_product_name(self, filename: str) -> str:
57         """从文件名提取产品名称"""
58         # 格式: 前缀_产品名称
59         # 移除 .txt 扩展名
60         filename = filename.replace('.txt', '')
61
62         # 尝试按 '_' 拆分, 但不是所有文件名都有 '_'
63         # 有些文件名可能是完整的产品名称, 没有前缀
64         if '_' in filename:
65             # 如果有 '_', 返回整个文件名 (包括前缀)
66             # 这样可以正确匹配长文件名
67             return filename

```



```

67         return filename
68
69     def _parse_txt_file(self, txt_file: Path) -> Dict[str, List[str]]:
70         """解析txt文件，提取每张图片的OCR文字"""
71         result = {}
72         with open(txt_file, 'r', encoding='utf-8') as f:
73             content = f.read()
74
75         # 按照 === 根目录 - 图片名.jpg === 分割（支持大写和小写）
76         pattern = r'=== 根目录 - (.+?\.(jpg|JPG|jpeg|JPEG)) ===\n.*?\n.--
+ \n\[ (.+?) \]'
77         matches = re.findall(pattern, content, re.DOTALL)
78
79         for match in matches:
80             img_name = match[0] # 完整的图片名（包含扩展名）
81             ocr_text = match[2] # OCR文字
82             img_key = img_name.rsplit('.', 1)[0] # 去掉扩展名
83             # 解析列表格式的OCR文字
84             try:
85                 ocr_list = eval('[' + ocr_text + ']')
86                 result[img_key] = ocr_list
87             except:
88                 result[img_key] = [ocr_text]
89
90         return result
91
92     def match_sentence_to_images(self, sentence: str, product_name: str,
93 threshold: float = 0.3) -> List[Tuple[str, float, str]]:
94         """
95         匹配句子到图片
96         返回：[(图片名, 相似度, 图片路径), ...]
97         """
98         if product_name not in self.txt_data:
99             return []
100
101         txt_data = self.txt_data[product_name]
102         img_paths = self.img_paths.get(product_name, {})
103
104         results = []
105
106         for img_name, ocr_list in txt_data.items():
107             if img_name not in img_paths:
108                 continue
109
110             # 合并OCR文字
111             ocr_text = ' '.join(str(item) for item in ocr_list)

```

```

112         # 计算相似度
113         similarity = self._calculate_similarity(sentence, ocr_text)
114
115         if similarity >= threshold: # 使用阈值筛选
116             results.append((img_name, similarity, img_paths[img_name]))
117
118         # 按相似度排序
119         results.sort(key=lambda x: x[1], reverse=True)
120         return results
121
122     def match_sentence_to_all_images(self, sentence: str, threshold: float =
0.3) -> List[Tuple[str, float, str, str]]:
123         """
124         从所有产品中匹配句子到图片
125         返回: [(产品名, 图片名, 相似度, 图片路径), ...]
126         """
127         all_results = []
128
129         for product_name in self.txt_data.keys():
130             matches = self.match_sentence_to_images(sentence, product_name,
threshold)
131             for img_name, similarity, img_path in matches:
132                 all_results.append((product_name, img_name, similarity,
img_path))
133
134             # 按相似度排序
135             all_results.sort(key=lambda x: x[2], reverse=True)
136             return all_results
137
138     def _calculate_similarity(self, sentence: str, ocr_text: str) -> float:
139         """计算句子与OCR文字的相似度"""
140         # 使用jieba分词提取关键词
141         sentence_words = list(jieba.cut(sentence))
142         ocr_words = list(jieba.cut(ocr_text))
143
144         # 过滤掉太短的词 (长度<2) 和空白
145         sentence_words = [w for w in sentence_words if len(w) >= 2 and
w.strip()]
146         ocr_words = [w for w in ocr_words if len(w) >= 2 and w.strip()]
147
148         if not sentence_words or not ocr_words:
149             return 0
150
151         # 计算重叠词汇比例
152         overlap = len(set(sentence_words) & set(ocr_words))
153         max_len = max(len(sentence_words), len(ocr_words))
154

```

```

155 keyword_score = overlap / max_len if max_len > 0 else 0
156
157 # 计算字符串相似度
158 string_similarity = difflib.SequenceMatcher(None, sentence,
ocr_text).ratio()
159
160 # 综合评分（关键词权重提高到70%）
161 return keyword_score * 0.7 + string_similarity * 0.3
162
163 def split_sentences(self, text: str) -> List[str]:
164     """将文本拆分成句子（使用高级拆分逻辑）"""
165     # 第一步：清洗文本
166     text = re.sub(r"\s+", " ", text.strip())
167     # 合并连续的句末标点
168     text = re.sub(r"([\u3002\uFF01\uFF1f])\1+", r"\1", text)
169
170     # 第二步：标记不可拆分区域（括号、引号、数字等）
171     non_split_pairs = [
172         ("(", ")"), ("(", ")"), ("【", "】"), ("[" , " ]"),
173         ("“", "”"), ("‘", "’"), ("‘", "’"), ("—", "—"), ("-", "-")
174     ]
175     non_split_map = {}
176     for idx, (left, right) in enumerate(non_split_pairs):
177         left_esc = re.escape(left)
178         right_esc = re.escape(right)
179         pattern = f"({left_esc}.*?{right_esc})"
180         for match in re.finditer(pattern, text):
181             placeholder = f" [NS_{idx}_{match.start()}] "
182             non_split_map[placeholder] = match.group(1)
183             text = text.replace(match.group(1), placeholder, 1)
184
185     # 第三步：按中文句号、分号拆分
186     # 按中文句号、感叹号、问号、分号拆分
187     sentences = re.split(r'[\u3002\uFF01\uFF1f\uFF1b;]', text)
188
189     # 第四步：恢复不可拆分区域
190     sentences = [s.strip() for s in sentences if s.strip()]
191     sentences = [self._restore_non_split_regions(s, non_split_map) for s in
sentences]
192
193     # 第五步：过滤掉太短的句子（低于5个字）
194     sentences = [s for s in sentences if len(s) >= 5]
195
196     return sentences
197
198     def _restore_non_split_regions(self, text: str, non_split_map: Dict[str,
str]) -> str:

```

```

199         """恢复不可拆分区域"""
200         for placeholder, original in non_split_map.items():
201             text = text.replace(placeholder, original)
202         return text
203
204     def get_all_products(self) -> List[str]:
205         """获取所有产品名称"""
206         return list(self.txt_data.keys())
207
208     class SentenceImageMatcherGUI:
209         """GUI界面"""
210
211         def __init__(self, root):
212             self.root = root
213             self.root.title("句子配图工具")
214             self.root.geometry("1200x800")
215             self.root.configure(bg='#f5f5f5')
216
217             # 设置字体
218             self.font_title = ('Segoe UI', 12, 'bold')
219             self.font_normal = ('Segoe UI', 10)
220             self.font_small = ('Segoe UI', 9)
221
222             # 设置颜色
223             self.color_primary = '#2196F3'          # 主色 (蓝色)
224             self.color_bg = '#f5f5f5'               # 背景色
225             self.color_text = '#333333'             # 文字色
226             self.color_border = '#e0e0e0'           # 边框色
227             self.color_hover = '#f0f0f0'            # 悬停色
228
229             self.matcher = None
230             self.sentences = []
231             self.current_sentence_idx = 0
232             self.current_product = None
233             self.current_matches = []
234             self.current_match_idx = 0
235             self.similarity_threshold = 0.09 # 相似度阈值 (默认 0.09 = 9%)
236
237             self._create_widgets()
238
239         def _create_widgets(self):
240             """创建UI组件"""
241             # 顶部：文件夹选择
242             top_frame = ttk.Frame(self.root)
243             top_frame.pack(fill=tk.X, padx=10, pady=10)
244
245             ttk.Label(top_frame, text="参数文件夹:").pack(side=tk.LEFT)

```

```

246         self.folder_var = tk.StringVar()
247         ttk.Entry(top_frame, textvariable=self.folder_var,
width=60).pack(side=tk.LEFT, padx=5)
248         ttk.Button(top_frame, text="浏览",
command=self._select_folder).pack(side=tk.LEFT)
249         ttk.Button(top_frame, text="加载",
command=self._load_folder).pack(side=tk.LEFT, padx=5)
250
251         # 相似度阈值控件 (百分比指示)
252         ttk.Separator(top_frame, orient=tk.VERTICAL).pack(side=tk.LEFT,
fill=tk.Y, padx=10)
253         ttk.Label(top_frame, text="相似度阈值 (%) :").pack(side=tk.LEFT, padx=
(0, 5))
254         self.threshold_var = tk.DoubleVar(value=9) # 改为百分比: 9%
255         self.threshold_scale = ttk.Scale(top_frame, from_=0, to=100,
orient=tk.HORIZONTAL,
256                                     variable=self.threshold_var,
command=self._on_threshold_change,
257                                     length=150)
258         self.threshold_scale.pack(side=tk.LEFT, padx=5)
259         self.threshold_entry_var = tk.StringVar(value="9") # 改为百分比
260         self.threshold_entry = ttk.Entry(top_frame,
textvariable=self.threshold_entry_var, width=5)
261         self.threshold_entry.pack(side=tk.LEFT, padx=5)
262         ttk.Label(top_frame, text="%").pack(side=tk.LEFT, padx=(0, 5))
263         self.threshold_entry.bind('<KeyRelease>',
self._on_threshold_entry_change)
264
265         # 中间: 左侧文本输入, 右侧图片显示
266         main_frame = ttk.Frame(self.root)
267         main_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
268
269         # 左侧: 输入文本和句子列表
270         left_frame = ttk.Frame(main_frame)
271         left_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 10))
272
273         ttk.Label(left_frame, text="输入文本:", font=self.font_title,
foreground=self.color_primary).pack(anchor=tk.W)
274         self.text_input = tk.Text(left_frame, height=8, width=50,
font=self.font_normal,
275                                     bg='white', fg=self.color_text,
insertbackground=self.color_primary,
276                                     relief=tk.FLAT, borderwidth=1)
277         self.text_input.pack(fill=tk.BOTH, expand=True, pady=5)
278
279         button_frame = ttk.Frame(left_frame)
280         button_frame.pack(anchor=tk.W, pady=5)

```

```
281         ttk.Button(button_frame, text="拆分句子",
282                     command=self._split_sentences).pack(side=tk.LEFT, padx=2)
283
284         ttk.Button(button_frame, text="更新列表",
285                     command=self._update_sentence_list).pack(side=tk.LEFT, padx=2)
286
287         ttk.Label(left_frame, text="句子列表:", font=self.font_title,
288                  foreground=self.color_primary).pack(anchor=tk.W, pady=(10, 0))
289
290         # 句子列表框
291         list_frame = ttk.Frame(left_frame)
292         list_frame.pack(fill=tk.BOTH, expand=True)
293
294         scrollbar = ttk.Scrollbar(list_frame)
295         scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
296
297         self.sentence_listbox = tk.Listbox(list_frame,
298                                           yscrollcommand=scrollbar.set, font=self.font_normal,
299                                           bg='white', fg=self.color_text,
300                                           selectbackground=self.color_primary,
301                                           selectforeground='white',
302                                           relief=tk.FLAT, borderwidth=1)
303         self.sentence_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
304         self.sentence_listbox.bind('<<ListboxSelect>>',
305                                   self._on_sentence_select)
306         scrollbar.config(command=self.sentence_listbox.yview)
307
308         # 右侧: 产品选择和图片显示
309         right_frame = ttk.Frame(main_frame)
310         right_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
311
312         ttk.Label(right_frame, text="选择产品:", font=self.font_title,
313                  foreground=self.color_primary).pack(anchor=tk.W)
314         self.product_var = tk.StringVar()
315         self.all_products = [] # 保存所有产品列表
316         self.product_combo = ttk.Combobox(right_frame,
317                                           textvariable=self.product_var, font=self.font_normal)
318         self.product_combo.pack(fill=tk.X, pady=5)
319         self.product_combo.bind('<<ComboboxSelected>>',
320                                self._on_product_select)
321         self.product_combo.bind('<KeyRelease>', self._on_product_input) # 添加
322         模糊匹配
323
324         # 图片显示
325         ttk.Label(right_frame, text="匹配图片:", font=self.font_title,
326                  foreground=self.color_primary).pack(anchor=tk.W, pady=(5, 0))
327
328         # 使用grid布局以不让内容自动调整大小
```

```

316         image_container = tk.Frame(right_frame, bg='#e8e8e8', relief=tk.FLAT,
borderwidth=0, height=500, width=600)
317         image_container.pack(fill=tk.BOTH, expand=False, pady=3)
318         image_container.pack_propagate(False) # 固定大小, 防止移位
319
320         self.image_canvas = tk.Canvas(image_container, bg='#e8e8e8',
highlightthickness=0)
321         self.image_canvas.pack(fill=tk.BOTH, expand=True)
322
323         # 图片信息和导航 (固定高度, 防止移位)
324         info_frame = tk.Frame(right_frame, height=85, bg='white')
325         info_frame.pack(fill=tk.X, pady=3)
326         info_frame.pack_propagate(False) # 不让框根据内容自动调整大小
327
328         self.image_info_label = tk.Label(info_frame, text="", justify=tk.LEFT,
font=self.font_small,
329                                         bg='white', fg=self.color_text)
330         self.image_info_label.pack(anchor=tk.NW, padx=5, pady=5, fill=tk.BOTH,
expand=True)
331
332         # 导航按钮
333         nav_frame = tk.Frame(right_frame, height=40, bg='#f5f5f5')
334         nav_frame.pack(fill=tk.X, pady=3)
335         nav_frame.pack_propagate(False) # 固定高度
336
337         ttk.Button(nav_frame, text="< 上一张",
command=self._prev_image).pack(side=tk.LEFT, padx=2, pady=5)
338         ttk.Button(nav_frame, text="下一张 >",
command=self._next_image).pack(side=tk.LEFT, padx=2, pady=5)
339         ttk.Button(nav_frame, text="复制图片",
command=self._copy_image).pack(side=tk.LEFT, padx=2, pady=5)
340         ttk.Button(nav_frame, text="清空",
command=self._clear_image).pack(side=tk.LEFT, padx=2, pady=5)
341
342         # 底部: 状态栏
343         self.status_var = tk.StringVar(value="就绪")
344         status_bar = ttk.Label(self.root, textvariable=self.status_var,
relief=tk.SUNKEN)
345         status_bar.pack(fill=tk.X, side=tk.BOTTOM)
346
347         def _on_threshold_change(self, value):
348             """阈值滑块变化事件 (百分比) """
349             percentage = float(value)
350             self.similarity_threshold = percentage / 100.0 # 转换为小数
351             self.threshold_entry_var.set(f"{percentage:.0f}")
352             # 重新更新匹配结果
353             self._update_matches()

```

```

354
355 def _on_threshold_entry_change(self, event):
356     """阈值输入框变化事件（百分比）"""
357     try:
358         percentage = float(self.threshold_entry_var.get())
359         # 限制范围在 0 - 100
360         if percentage < 0:
361             percentage = 0
362         elif percentage > 100:
363             percentage = 100
364         self.similarity_threshold = percentage / 100.0 # 转换为小数
365         self.threshold_var.set(percentage)
366         # 重新更新匹配结果
367         self._update_matches()
368     except ValueError:
369         # 如果输入不是数字，忽略
370         pass
371
372 def _select_folder(self):
373     """选择参数文件夹"""
374     folder = filedialog.askdirectory(title="选择参数文件夹")
375     if folder:
376         self.folder_var.set(folder)
377
378 def _load_folder(self):
379     """加载参数文件夹"""
380     folder = self.folder_var.get()
381     if not folder:
382         messagebox.showwarning("提示", "请先选择参数文件夹")
383         return
384
385     try:
386         self.matcher = SentenceImageMatcher(folder)
387         products = self.matcher.get_all_products()
388
389         # 检查是否成功加载了数据
390         if not products:
391             # 详细诊断
392             txt_folder = Path(folder) / "详情图 txt"
393             img_folder = Path(folder) / "详情图片"
394
395             error_msg = "数据加载失败:\n\n"
396
397             if not txt_folder.exists():
398                 error_msg += f"❌ 详情图 txt 文件夹不存在\n    预期位置:
{txt_folder}\n\n"
399             else:

```



```

400         txt_files = list(txt_folder.glob("*.txt"))
401         error_msg += f"✓ 详情图 txt 文件夹存在 ({len(txt_files)} 个
txt 文件)\n\n"
402
403         if not img_folder.exists():
404             error_msg += f"✗ 详情图片 文件夹不存在\n    预期位置:
{img_folder}\n\n"
405         else:
406             img_dirs = [d for d in img_folder.iterdir() if d.is_dir()]
407             error_msg += f"✓ 详情图片 文件夹存在 ({len(img_dirs)} 个产品文
件夹)\n\n"
408
409             error_msg += "解决方案:\n"
410             error_msg += "1. 运行
'\u521d\u59cb\u5316\u53c2\u6570\u6587\u4ef6\u5939.py' 来创\u5efa文件
\u5939\u7ed3\u6784\n"
411             error_msg += "2. 将 txt 文件放入 '\u8be6\u60c5\u56fe txt' 文件
\u5939\n"
412             error_msg += "3. 在 '\u8be6\u60c5\u56fe\u7247' 中创\u5efa产
\u54c1文件\u5939\u56fe\u7247\n"
413
414             messagebox.showerror("错误", error_msg)
415             self.status_var.set("数据加载失败")
416             return
417
418         # 添加"全选"选项
419         self.all_products = ["【全选】所有产品"] + products
420         self.product_combo['values'] = self.all_products
421         if self.all_products:
422             self.product_combo.current(0)
423             self.current_product = "【全选】所有产品"
424             self.status_var.set(f"已加载 {len(products)} 个产品")
425         except ValueError as e:
426             # 文件夹不存在的错误
427             error_msg = f"{str(e)}\n\n解决方案:\n"
428             error_msg += "1. 运行
'\u521d\u59cb\u5316\u53c2\u6570\u6587\u4ef6\u5939.py'\n"
429             error_msg += "2. 或者手动创\u5efa '\u8be6\u60c5\u56fe txt' 和
'\u8be6\u60c5\u56fe\u7247' 文\u4ef6\u5939\n"
430             messagebox.showerror("错误", error_msg)
431             self.status_var.set("加载失败")
432         except Exception as e:
433             messagebox.showerror("错误", f"加载失败: {str(e)}")
434             self.status_var.set("加载失败")
435
436     def _split_sentences(self):
437         """拆分句子，並在输入框中按换行显示"""

```

```
438         if not self.matcher:
439             messagebox.showwarning("提示", "请先加载参数文件夹")
440             return
441
442         text = self.text_input.get("1.0", tk.END).strip()
443         if not text:
444             messagebox.showwarning("提示", "请输入文本")
445             return
446
447         # 拆分句子
448         self.sentences = self.matcher.split_sentences(text)
449
450         # 更新输入框: 每句话一行
451         self.text_input.delete("1.0", tk.END)
452         self.text_input.insert("1.0", "\n".join(self.sentences))
453
454         # 更新句子列表
455         self.sentence_listbox.delete(0, tk.END)
456         for i, sentence in enumerate(self.sentences, 1):
457             self.sentence_listbox.insert(tk.END, f"{i}. {sentence}")
458
459         self.current_sentence_idx = 0
460         self.status_var.set(f"已拆分 {len(self.sentences)} 个句子, 你可以在上方输入
框中调整")
461
462         if self.sentences:
463             self.sentence_listbox.selection_set(0)
464             self._on_sentence_select(None)
465
466     def _update_sentence_list(self):
467         """从输入框中更新句子列表 (按换行拆分) """
468         text = self.text_input.get("1.0", tk.END).strip()
469         if not text:
470             messagebox.showwarning("提示", "输入框为空")
471             return
472
473         # 按换行拆分 (一个换行 = 一句话)
474         self.sentences = [s.strip() for s in text.split('\n') if s.strip()]
475
476         # 更新句子列表
477         self.sentence_listbox.delete(0, tk.END)
478         for i, sentence in enumerate(self.sentences, 1):
479             self.sentence_listbox.insert(tk.END, f"{i}. {sentence}")
480
481         self.current_sentence_idx = 0
482         self.status_var.set(f"已更新 {len(self.sentences)} 个句子")
483
```

```
484         if self.sentences:
485             self.sentence_listbox.selection_set(0)
486             self._on_sentence_select(None)
487
488     def _on_sentence_select(self, event):
489         """句子选择事件"""
490         selection = self.sentence_listbox.curselection()
491         if selection:
492             self.current_sentence_idx = selection[0]
493             self._update_matches()
494
495     def _on_product_input(self, event):
496         """产品输入框模糊匹配事件"""
497         input_text = self.product_var.get()
498
499         # 如果输入为空, 显示所有产品
500         if not input_text:
501             self.product_combo['values'] = self.all_products
502             return
503
504         # 模糊匹配: 不区分大小写, 包含匹配
505         filtered_products = [
506             p for p in self.all_products
507             if input_text.lower() in p.lower()
508         ]
509
510         # 更新下拉框选项
511         self.product_combo['values'] = filtered_products if filtered_products
512     else self.all_products
513
514     def _on_product_select(self, event):
515         """产品选择事件"""
516         self.current_product = self.product_var.get()
517         self._update_matches()
518
519     def _update_matches(self):
520         """更新匹配结果"""
521         if not self.sentences or self.current_sentence_idx >=
522     len(self.sentences):
523             return
524
525         sentence = self.sentences[self.current_sentence_idx]
526
527         if not self.current_product:
528             return
529
530         # 如果选择"全选", 从所有产品中筛选
```

```
529         if self.current_product == "【全选】所有产品":
530             self.current_matches =
self.matcher.match_sentence_to_all_images(sentence, self.similarity_threshold)
531         else:
532             self.current_matches =
self.matcher.match_sentence_to_images(sentence, self.current_product,
self.similarity_threshold)
533
534         self.current_match_idx = 0
535
536         if self.current_matches:
537             self._display_image()
538         else:
539             self._clear_image()
540             self.image_info_label.config(text="未找到匹配的图片")
541
542     def _display_image(self):
543         """显示当前图片"""
544         if not self.current_matches or self.current_match_idx >=
len(self.current_matches):
545             return
546
547         match_data = self.current_matches[self.current_match_idx]
548
549         # 处理两种数据格式：全选模式是4元组，单产品模式是3元组
550         if len(match_data) == 4:
551             product_name, img_name, similarity, img_path = match_data
552             info_prefix = f"产品: {product_name}\n"
553         else:
554             img_name, similarity, img_path = match_data
555             info_prefix = ""
556
557         try:
558             # 检查图片文件是否存在
559             if not os.path.exists(img_path):
560                 raise FileNotFoundError(f"图片文件不存在: {img_path}")
561
562             # 加载并显示图片
563             img = Image.open(img_path)
564
565             # 根据容器大小缩放 (容器600x500px)
566             # 保持完整的宽高比
567             container_width = 590
568             container_height = 490
569
570             # 计算缩放比例, 确保图片完全展示
571             img_ratio = img.width / img.height
```

```
572         container_ratio = container_width / container_height
573
574         if img_ratio > container_ratio:
575             # 图片较宽, 按宽度限制
576             new_width = container_width
577             new_height = int(container_width / img_ratio)
578         else:
579             # 图片较窄, 按高度限制
580             new_height = container_height
581             new_width = int(container_height * img_ratio)
582
583         img = img.resize((new_width, new_height), Image.Resampling.LANCZOS)
584         photo = ImageTk.PhotoImage(img)
585
586         # 使用Canvas显示图片, 并居中显示
587         self.image_canvas.delete('all')
588
589         # 强制更新Canvas以获取正确的宽高
590         self.image_canvas.update_idletasks()
591         canvas_width = self.image_canvas.winfo_width()
592         canvas_height = self.image_canvas.winfo_height()
593
594         # 如果canvas仍未初始化, 使用容器大小
595         if canvas_width <= 1:
596             canvas_width = 590
597         if canvas_height <= 1:
598             canvas_height = 490
599
600         x = canvas_width / 2
601         y = canvas_height / 2
602         self.image_canvas.create_image(x, y, image=photo)
603         # 保持PhotoImage引用, 防止被垃圾回收
604         if not hasattr(self, '_photo_images'):
605             self._photo_images = []
606         self._photo_images.append(photo)
607         # 保留最多10张图片的引用
608         if len(self._photo_images) > 10:
609             self._photo_images.pop(0)
610
611         # 更新信息
612         similarity_pct = similarity * 100
613         current_idx = self.current_match_idx + 1
614         total_count = len(self.current_matches)
615
616         # 构建更清晰的信息文本
617         info_lines = []
618         if info_prefix:
```

```
619         info_lines.append(info_prefix.strip())
620         info_lines.append(f"图片: {img_name}")
621         info_lines.append(f"相似度: {similarity_pct:.1f}%")
622         info_lines.append(f"进度: {current_idx}/{total_count}")
623
624         info_text = "\n".join(info_lines)
625         self.image_info_label.config(text=info_text)
626
627         self.current_image_path = img_path
628     except FileNotFoundError as e:
629         self.image_info_label.config(text=f"文件不存在: {str(e)}")
630         self.image_canvas.delete('all')
631     except Exception as e:
632         self.image_info_label.config(text=f"加载图片失败: {str(e)}")
633         self.image_canvas.delete('all')
634
635     def _prev_image(self):
636         """上一张图片"""
637         if self.current_matches and self.current_match_idx > 0:
638             self.current_match_idx -= 1
639             self._display_image()
640
641     def _next_image(self):
642         """下一张图片"""
643         if self.current_matches and self.current_match_idx <
len(self.current_matches) - 1:
644             self.current_match_idx += 1
645             self._display_image()
646
647     def _copy_image(self):
648         """复制图片到剪贴板"""
649         if not hasattr(self, 'current_image_path') or self.current_image_path
is None:
650             messagebox.showwarning("提示", "没有要复制的图片")
651             return
652
653         if not os.path.exists(self.current_image_path):
654             messagebox.showerror("错误", "图片文件不存在")
655             return
656
657         # 优先级: 1. win32clipboard 2. ctypes 3. PowerShell Set-Clipboard 4.
clip.exe 5. 直接写入剪贴板
658         try:
659             if HAS_WIN32:
660                 self._copy_image_win32()
661             else:
662                 self._copy_image_fallback()
```

```

663         except Exception as e1:
664             try:
665                 self._copy_image_fallback()
666             except Exception as e2:
667                 try:
668                     self._copy_image_powershell()
669                 except Exception as e3:
670                     try:
671                         self._copy_image_via_clipboard_file()
672                     except Exception as e4:
673                         try:
674                             self._copy_image_via_clip_exe()
675                         except Exception as e5:
676                             messagebox.showerror("错误", f"所有复制方案均失
败:\n1. win32: {str(e1)}\n2. ctypes: {str(e2)}\n3. PowerShell: {str(e3)}\n4.
Set-Clipboard: {str(e4)}\n5. clip.exe: {str(e5)}")
677
678     def _copy_image_win32(self):
679         """使用 win32clipboard 复制图片"""
680         try:
681             img = Image.open(self.current_image_path)
682
683             output = io.BytesIO()
684             img.convert('RGB').save(output, 'BMP')
685             bmp_data = output.getvalue()
686             output.close()
687
688             # BMP文件格式: 前14字节是文件头, 需要去掉
689             dib_data = bmp_data[14:]
690
691             win32clipboard.OpenClipboard()
692             win32clipboard.EmptyClipboard()
693             win32clipboard.SetClipboardData(win32clipboard.CF_DIB, dib_data)
694             win32clipboard.CloseClipboard()
695
696             messagebox.showinfo("成功", "图片已复制到剪贴板")
697         except Exception as e:
698             raise Exception(f"win32clipboard 复制失败: {str(e)}")
699
700     def _copy_image_fallback(self):
701         """备用方案: 使用 ctypes 直接操作 Windows 剪贴板"""
702         import ctypes
703
704         try:
705             img = Image.open(self.current_image_path)
706
707             output = io.BytesIO()

```

```
708         img.convert('RGB').save(output, 'BMP')
709         bmp_data = output.getvalue()
710         output.close()
711
712         # 去掉 BMP 文件头 (前14字节)
713         dib_data = bmp_data[14:]
714
715         user32 = ctypes.windll.user32
716         kernel32 = ctypes.windll.kernel32
717
718         # 打开剪贴板
719         if not user32.OpenClipboard(None):
720             raise Exception("打开剪贴板失败")
721
722         # 清空剪贴板
723         user32.EmptyClipboard()
724
725         # 分配全局内存
726         size = len(dib_data)
727         hglobal = kernel32.GlobalAlloc(0x0002, size)
728         if not hglobal:
729             user32.CloseClipboard()
730             raise Exception("分配内存失败")
731
732         # 锁定内存并复制数据
733         ptr = kernel32.GlobalLock(hglobal)
734         if not ptr:
735             kernel32.GlobalFree(hglobal)
736             user32.CloseClipboard()
737             raise Exception("锁定内存失败")
738
739         ctypes.memmove(ptr, dib_data, size)
740         kernel32.GlobalUnlock(hglobal)
741
742         # 设置剪贴板数据 (CF_DIB = 8)
743         if not user32.SetClipboardData(8, hglobal):
744             kernel32.GlobalFree(hglobal)
745             user32.CloseClipboard()
746             raise Exception("设置剪贴板数据失败")
747
748         user32.CloseClipboard()
749         messagebox.showinfo("成功", "图片已复制到剪贴板")
750     except Exception as e:
751         raise Exception(f"ctypes 复制失败: {str(e)}")
752
753     def _copy_image_powershell(self):
754         """第三个备选: 使用 PowerShell 复制图片"""
```



```
755     import subprocess
756
757     try:
758         # 转换路径为 Windows 格式 (使用原始路径)
759         img_path = os.path.abspath(self.current_image_path)
760
761         # PowerShell 脚本: 将图片复制到剪贴板
762         # 使用 Add-Type 加载程集, 比 LoadWithPartialName 更可靠
763         ps_script = f"""
764         Add-Type -AssemblyName System.Windows.Forms
765         Add-Type -AssemblyName System.Drawing
766         $img = [System.Drawing.Image]::FromFile('{img_path}')
767         [System.Windows.Forms.Clipboard]::SetImage($img)
768         $img.Dispose()
769         """
770
771         # 执行 PowerShell 脚本
772         result = subprocess.run(
773             ['powershell', '-NoProfile', '-Command', ps_script],
774             capture_output=True,
775             text=True,
776             timeout=5,
777             encoding='utf-8'
778         )
779
780         if result.returncode != 0:
781             raise Exception(f"PowerShell 错误: {result.stderr}")
782
783         messagebox.showinfo("成功", "图片已复制到剪贴板")
784     except Exception as e:
785         raise Exception(f"PowerShell 复制失败: {str(e)}")
786
787     def _copy_image_via_clipboard_file(self):
788         """第四个备选: 使用 PowerShell 的 Set-Clipboard 复制图片文件 (最稳定) """
789         try:
790             # 最简单的方案: 使用 PowerShell 的 Set-Clipboard 命令复制文件
791             # PowerShell 5.0+ 自带此命令, 不需要加载程集
792
793             img_path = os.path.abspath(self.current_image_path)
794
795             # 使用 PowerShell 的 Set-Clipboard 命令, 传入文件路径
796             # PowerShell 会自动检测文件类型并复制为图片
797             ps_script = f"""
798             $file = Get-Item -LiteralPath '{img_path}'
799             if ($file.PSIsContainer) {{
800                 throw 'Path is a directory, not a file'
801             }}
```

```

802         Set-Clipboard -Path '{img_path}'
803         """
804
805         result = subprocess.run(
806             ['powershell', '-NoProfile', '-Command', ps_script],
807             capture_output=True,
808             text=True,
809             timeout=5,
810             encoding='utf-8'
811         )
812
813         if result.returncode != 0:
814             # 如果 Set-Clipboard 失败, 尝试使用 clip.exe
815             raise Exception(f"Set-Clipboard 失败: {result.stderr}")
816
817         messagebox.showinfo("成功", "图片已复制到剪贴板")
818     except Exception as e:
819         raise Exception(f"剪贴板复制失败: {str(e)}")
820
821     def _copy_image_via_clip_exe(self):
822         """第五个备选: 使用 clip.exe 复制图片文件路径"""
823         try:
824             # clip.exe 是 Windows 内置工具, 会自动检测文件类型
825             img_path = os.path.abspath(self.current_image_path)
826
827             # 使用 PowerShell 调用 clip.exe
828             ps_script = f"""
829             $file = Get-Item -LiteralPath '{img_path}'
830             if ($file.PSIsContainer) {{
831                 throw 'Path is a directory'
832             }}
833             $file.FullName | clip.exe
834             """
835
836             result = subprocess.run(
837                 ['powershell', '-NoProfile', '-Command', ps_script],
838                 capture_output=True,
839                 text=True,
840                 timeout=5,
841                 encoding='utf-8'
842             )
843
844             if result.returncode != 0:
845                 raise Exception(f"clip.exe 失败: {result.stderr}")
846
847             messagebox.showinfo("成功", "图片已复制到剪贴板")
848         except Exception as e:

```

```

849         raise Exception(f"clip.exe 复制失败: {str(e)}")
850
851     def _clear_image(self):
852         """清空图片暗示"""
853         self.image_canvas.delete('all')
854         self.image_canvas.image = None
855         self.image_info_label.config(text="")
856         self.current_image_path = None
857
858     def main():
859         root = tk.Tk()
860         app = SentenceImageMatcherGUI(root)
861         root.mainloop()
862
863     if __name__ == "__main__":
864         main()
865

```



这里的代码是今天才开发的11.22号，也是基于现有系统上，

功能呢，首先需要点击浏览，找到对应的参数文件夹。参数文件夹存放的文件夹看 **2.5我在用的方法，就是你自己创建的文件夹，经过影刀代码的处理，会产生参数文件夹。**

浏览完，点击加载。

如果对于拆分句子的结果不满意，可以在输入文本这里自己换行，然后点击**更新列表**。

选择产品这里，支持下拉框，选择其他的产品图片文件夹。这里我的想法是，如果你想介绍产品的时候，一句话对应一张图片的话，可以把介绍产品的段落放到输入文本里面。然后在**选择产品**这里，下拉选择到对应产品图片文件夹进行图片匹配。**支持输入关键词模糊匹配**

**相似度阈值**这里，可以调整，调的越小，匹配到的图片越多。

**复制图片，windows系统**测试可以用。Mac电脑没有测试

**如何使用？ 问豆包！**

### 3.4 快速获取产品宣传视频

这个也算是Broll提效的一部分。

我这里写的代码是 只能获取京东的宣传视频

多线程爬取京东宣传视频

```

1  import requests
2  import re
3  import json

```

```
4 import os
5 from pathlib import Path
6 from playwright.sync_api import sync_playwright
7 import time
8 from concurrent.futures import ThreadPoolExecutor, as_completed
9 import threading
10
11 class JDVideoCrawlerFast:
12     def __init__(self, save_dir="./jd_video_downloads", max_workers=3):
13         self.save_dir = save_dir
14         Path(save_dir).mkdir(parents=True, exist_ok=True)
15         self.max_workers = max_workers # 并发线程数
16         self.lock = threading.Lock() # 线程锁, 用于打印输出
17
18         self.headers = {
19             "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
20             "Referer": "https://www.jd.com/",
21         }
22
23     def extract_sku_id(self, url):
24         """从URL中提取商品SKU ID"""
25         match = re.search(r'/(\\d+)\\.html', url)
26         if match:
27             return match.group(1)
28         return None
29
30     def extract_video_ids(self, content):
31         """从页面内容中提取视频ID"""
32         video_ids = []
33
34         pattern1 = r'data-vu=["\\'](\\d+)["\\']'
35         matches = re.findall(pattern1, content)
36         video_ids.extend(matches)
37
38         pattern2 = r'mainVideoId["\\']?\\s*[:=]\\s*["\\'](\\d+)["\\']'
39         matches = re.findall(pattern2, content)
40         video_ids.extend(matches)
41
42         pattern3 = r'vid["\\']?\\s*[:=]\\s*["\\'](\\d+)["\\']'
43         matches = re.findall(pattern3, content)
44         video_ids.extend(matches)
45
46         return list(set(video_ids))
47
48     def get_video_from_tencent_api(self, video_id, sku_id):
49         """通过视频ID从腾讯视频API获取视频URL"""
```

```

50         try:
51             api_url = f"https://api.m.jd.com/tencent/video_v3?
callback=jQuery123&vid={video_id}&type=1&from=1&appid=item-
v3&functionId=pc_tencent_video_v3"
52
53             response = requests.get(api_url, headers=self.headers, timeout=10)
54             response.raise_for_status()
55
56             jsonp_match = re.search(r'jQuery\d+\(((.*)\))', response.text)
57             if jsonp_match:
58                 json_str = jsonp_match.group(1)
59                 video_data = json.loads(json_str)
60
61                 if video_data.get('code') == 0 and video_data.get('playUrl'):
62                     duration = video_data.get('duration', 0)
63                     play_url = video_data.get('playUrl')
64                     return play_url, duration
65
66             return None, 0
67         except Exception as e:
68             return None, 0
69
70     def check_video_exists_with_playwright(self, url):
71         """使用Playwright检查页面中是否存在视频"""
72         try:
73             with sync_playwright() as p:
74                 browser = p.chromium.launch(headless=True, args=["--no-
sandbox"])
75
76                 context = browser.new_context(extra_http_headers=self.headers)
77                 page = context.new_page()
78
79                 page.goto(url, timeout=15000, wait_until="domcontentloaded")
80                 time.sleep(3) # 减少等待时间以加快速度
81
82                 content = page.content()
83                 video_ids = self.extract_video_ids(content)
84
85                 browser.close()
86
87                 if video_ids:
88                     return True, video_ids
89                 return False, []
90         except Exception as e:
91             return False, []
92
93     def extract_video_urls(self, url, sku_id):

```

```
94         """提取页面中的所有视频URL"""
95         video_urls = []
96
97         try:
98             with sync_playwright() as p:
99                 browser = p.chromium.launch(headless=True, args=["--no-
sandbox"])
100                 context = browser.new_context(extra_http_headers=self.headers)
101                 page = context.new_page()
102
103                 page.goto(url, timeout=15000, wait_until="domcontentloaded")
104                 time.sleep(3)
105
106                 content = page.content()
107                 video_ids = self.extract_video_ids(content)
108
109                 for video_id in video_ids:
110                     play_url, duration =
self.get_video_from_tencent_api(video_id, sku_id)
111                     if play_url:
112                         video_urls.append({
113                             'url': play_url,
114                             'duration': duration,
115                             'video_id': video_id,
116                             'source': 'tencent_api'
117                         })
118
119                 browser.close()
120
121         except Exception as e:
122             pass
123
124         return video_urls
125
126     def download_video(self, video_url, filename):
127         """下载单个视频"""
128         try:
129             save_path = os.path.join(self.save_dir, filename)
130
131             if os.path.exists(save_path):
132                 with self.lock:
133                     print(f" ⚠️ 文件已存在, 跳过: {filename}")
134                     return True
135
136             with self.lock:
137                 print(f" 🔄 正在下载: {filename}")
138
```

```
139         response = requests.get(video_url, headers=self.headers,
stream=True, timeout=30)
140         response.raise_for_status()
141
142         total_size = int(response.headers.get('content-length', 0))
143         downloaded = 0
144         chunk_size = 1024 * 1024
145
146         with open(save_path, 'wb') as f:
147             for chunk in response.iter_content(chunk_size=chunk_size):
148                 if chunk:
149                     f.write(chunk)
150                     downloaded += len(chunk)
151
152         file_size = os.path.getsize(save_path) / 1024 / 1024
153         with self.lock:
154             print(f" ✅ 下载完成: {filename} ({file_size:.2f} MB)")
155         return True
156
157     except Exception as e:
158         with self.lock:
159             print(f" ❌ 下载失败: {str(e)}")
160         return False
161
162     def process_url(self, url):
163         """处理单个URL"""
164         with self.lock:
165             print(f"\n{'='*70}")
166             print(f"处理URL: {url}")
167             print(f"{'='*70}")
168
169         if 'jd.com' not in url:
170             with self.lock:
171                 print(f"❌ 不是有效的京东URL")
172             return False
173
174         sku_id = self.extract_sku_id(url)
175         if not sku_id:
176             with self.lock:
177                 print(f"❌ 无法从URL中提取商品ID")
178             return False
179
180         with self.lock:
181             print(f"✅ 商品ID: {sku_id}")
182
183         with self.lock:
184             print(f"\n【第一步】检查是否存在视频...")
```

```
185
186     has_video, video_ids = self.check_video_exists_with_playwright(url)
187
188     if not has_video:
189         with self.lock:
190             print(f"❌ 该商品页面不存在视频")
191             return False
192
193     with self.lock:
194         print(f"✅ 检测到视频存在 ({len(video_ids)}个视频ID) ")
195
196     with self.lock:
197         print(f"\n【第二步】提取视频URL...")
198
199     video_urls = self.extract_video_urls(url, sku_id)
200
201     if not video_urls:
202         with self.lock:
203             print(f"❌ 无法提取视频URL")
204             return False
205
206     with self.lock:
207         print(f"✅ 找到 {len(video_urls)} 个视频")
208
209     with self.lock:
210         print(f"\n【第三步】下载视频...")
211
212     success_count = 0
213
214     for i, video_info in enumerate(video_urls, 1):
215         video_url = video_info['url']
216         duration = video_info['duration']
217         filename = f"jd_video_{sku_id}_{i}.mp4"
218
219         with self.lock:
220             print(f"\n视频{i}/{len(video_urls)} (时长: {duration}秒)")
221
222         if self.download_video(video_url, filename):
223             success_count += 1
224
225         time.sleep(0.5)
226
227     with self.lock:
228         print(f"\n✅ 成功下载 {success_count}/{len(video_urls)} 个视频")
229
230     return True
231
```



```

232     def process_urls_batch(self, urls):
233         """批量处理URL（多线程）"""
234         print(f"\n{'='*70}")
235         print(f"开始批量处理 {len(urls)} 个URL（并发数：{self.max_workers}）")
236         print(f"{'='*70}\n")
237
238         success_count = 0
239         failed_count = 0
240
241         with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
242             # 提交所有任务
243             futures = {executor.submit(self.process_url, url): url for url in
urls}
244
245             # 处理完成的任务
246             for future in as_completed(futures):
247                 url = futures[future]
248                 try:
249                     result = future.result()
250                     if result:
251                         success_count += 1
252                     else:
253                         failed_count += 1
254                 except Exception as e:
255                     with self.lock:
256                         print(f"✗ 处理失败 {url}: {str(e)}")
257                         failed_count += 1
258
259             print(f"\n{'='*70}")
260             print(f"【批量处理完成】")
261             print(f"成功：{success_count} 个")
262             print(f"失败：{failed_count} 个")
263             print(f"总计：{len(urls)} 个")
264             print(f"{'='*70}\n")
265
266     def process_urls_from_file(self, file_path):
267         """从文件中批量处理URL"""
268         try:
269             with open(file_path, 'r', encoding='utf-8') as f:
270                 urls = [line.strip() for line in f if line.strip()]
271
272                 self.process_urls_batch(urls)
273
274         except FileNotFoundError:
275             print(f"✗ 文件不存在：{file_path}")
276         except Exception as e:
277             print(f"✗ 错误：{str(e)}")

```

```
278
279 def main():
280     print("=" * 70)
281     print("京东视频批量爬取工具 - 快速版（多线程）")
282     print("=" * 70)
283
284     # 设置并发线程数（根据网络情况调整）
285     # 建议：网络好用3-5，网络一般用2-3
286     crawler = JDVideoCrawlerFast(max_workers=3)
287
288     while True:
289         print("\n请选择操作模式：")
290         print("1. 输入单个URL")
291         print("2. 从文件批量处理URL（多线程）")
292         print("3. 设置并发线程数")
293         print("4. 退出")
294
295         choice = input("\n请输入选择（1/2/3/4）：").strip()
296
297         if choice == '1':
298             url = input("\n请输入京东商品链接：").strip()
299             if url:
300                 crawler.process_url(url)
301
302         elif choice == '2':
303             file_path = input("\n请输入URL文件路径：").strip()
304             if file_path:
305                 crawler.process_urls_from_file(file_path)
306
307         elif choice == '3':
308             try:
309                 workers = int(input("\n请输入并发线程数（建议2-5）：").strip())
310                 if 1 <= workers <= 10:
311                     crawler.max_workers = workers
312                     print(f"✅ 并发线程数已设置为：{workers}")
313                 else:
314                     print("❌ 请输入1-10之间的数字")
315             except ValueError:
316                 print("❌ 请输入有效的数字")
317
318         elif choice == '4':
319             print("\n👋 再见！")
320             break
321
322         else:
323             print("❌ 无效的选择")
324
```

```
325 if __name__ == "__main__":
326     main()
327
```

## 使用指南

```
1  =====
2  =
3  京东视频批量爬取工具 - 使用指南
4  =====
5  =
6  【功能概述】
7  这是一个智能的京东商品视频爬取工具，可以：
8  ✓ 自动检测京东商品页面是否存在视频
9  ✓ 智能提取视频ID和视频URL
10 ✓ 自动下载视频到本地
11 ✓ 支持单个URL或批量处理
12 ✓ 自动去重和断点续传
13 =====
14 =
15 【快速开始】
16 =====
17 =
18 1. 安装依赖
19 pip install requests playwright
20 python -m playwright install chromium
21 2. 运行爬虫
22 python jd_video_crawler_v2.py
23 3. 选择操作模式
24 - 选项1：输入单个URL
25 - 选项2：从文件批量处理URL
26 - 选项3：退出
27
28 =====
29 =
30 【使用示例】
31 =====
32 =
33 【示例1】单个URL爬取
34 -----
```

```
35 1. 运行程序: python jd_video_crawler_v2.py
36 2. 选择选项1
37 3. 输入京东链接: https://item.jd.com/100032139847.html
38 4. 程序自动检测、提取、下载视频
39
```

#### 40 【示例2】批量URL爬取

```
41 -----
```

```
42 1. 创建文本文件 urls.txt, 内容如下:
43     https://item.jd.com/100032139847.html
44     https://item.jd.com/10000000000000.html
45     https://item.jd.com/10000000000001.html
46
47 2. 运行程序: python jd_video_crawler_v2.py
48 3. 选择选项2
49 4. 输入文件路径: urls.txt
50 5. 程序自动批量处理所有URL
51
```

```
52 =====
```

```
53 =
```

#### 53 【工作流程】

```
54 =====
```

```
55 =
```

```
56 输入URL
```

```
57 ↓
```

```
58 验证URL格式 (必须是jd.com)
```

```
59 ↓
```

```
60 提取商品ID (从URL中)
```

```
61 ↓
```

```
62 【检查视频】 使用Playwright加载页面, 提取视频ID
```

```
63 ↓
```

```
64 【提取视频URL】 通过腾讯视频API获取播放地址
```

```
65 ↓
```

```
66 【下载视频】 分块下载到本地
```

```
67 ↓
```

```
68 完成
```

```
69
```

```
70 =====
```

```
71 =
```

#### 71 【输出文件】

```
72 =====
```

```
73 =
```

```
74
```

```
74 所有下载的视频保存在 jd_video_downloads 目录中
```

```
75
```

```
76 文件名格式: jd_video_{商品ID}_{序号}.mp4
```

```
77
```

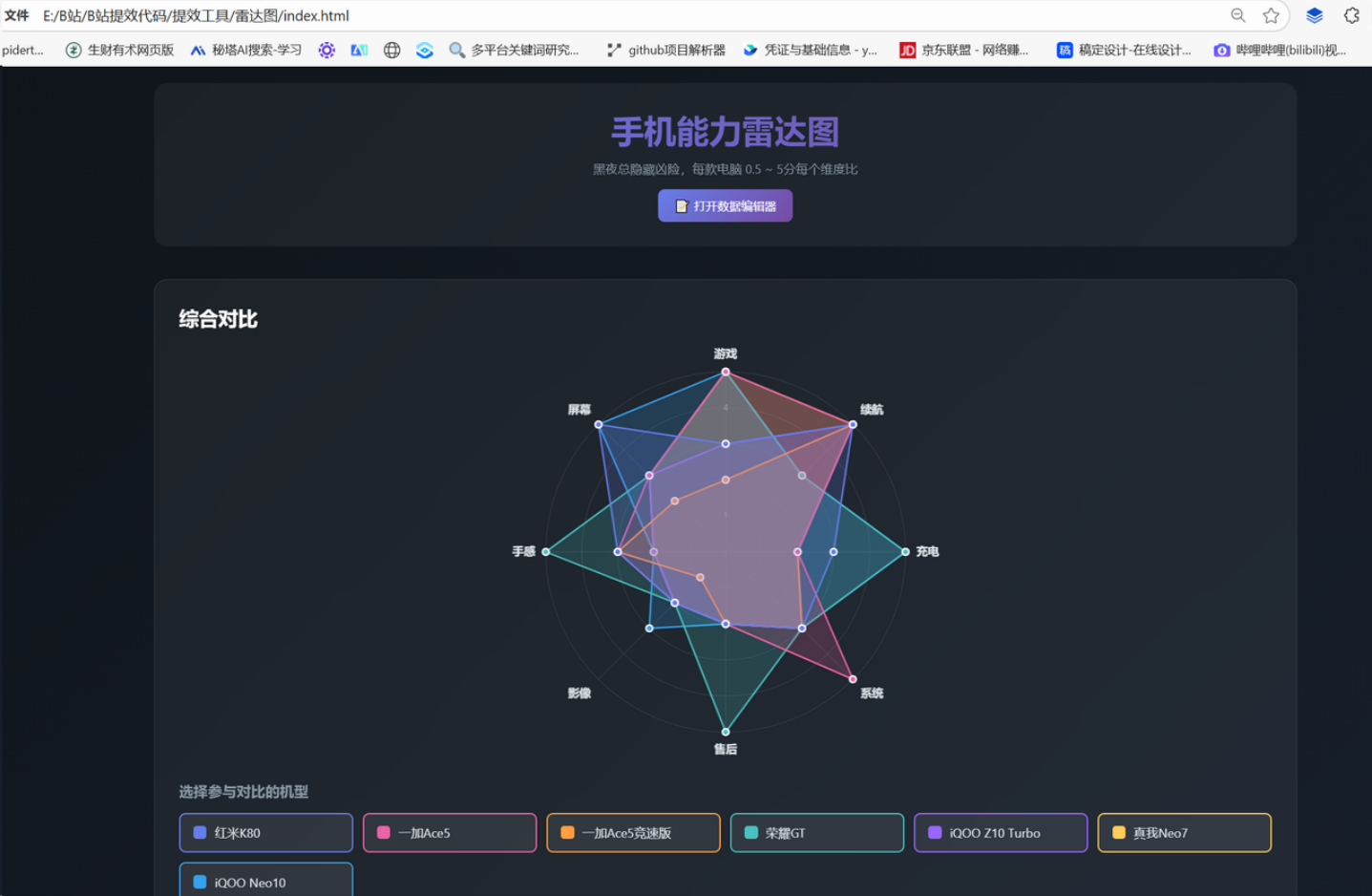
```
78 例如：
79     jd_video_100032139847_1.mp4
80     jd_video_100032139847_2.mp4
81
82 =====
83 =
84 【支持的视频来源】
85 =====
86 =
87 1. 腾讯视频API - 京东通过腾讯视频API加载的视频（主要）
88 2. 直接视频URL - 页面中直接包含的视频链接
89 3. HTML5 Video标签 - 通过<video>标签嵌入的视频
90 4. Source标签 - <video>标签内的<source>子元素
91 =====
92 =
93 【常见问题】
94 =====
95 =
96 Q：为什么有些商品没有视频？
97 A：并非所有京东商品都有视频。工具会自动检测并跳过没有视频的商品。
98
99 Q：下载速度很慢怎么办？
100 A：这取决于网络速度和视频服务器的响应速度。可以尝试：
101     - 检查网络连接
102     - 更换时间段重试
103     - 使用代理加速
104
105 Q：如何处理登录验证？
106 A：如果遇到登录验证，可以：
107     1. 从浏览器F12获取Cookie
108     2. 修改代码中的 HEADERS 字典，添加Cookie
109
110 Q：支持其他视频网站吗？
111 A：目前主要针对京东商品页面。可以根据需要修改代码以支持其他网站。
112
113 Q：如何查看下载的视频？
114 A：打开 jd_video_downloads 文件夹，所有下载的视频都在这里。
115
116 Q：可以同时下载多个视频吗？
117 A：可以。使用批量模式（选项2），在文本文件中输入多个URL，程序会依次处理。
118 =====
119 =
120 【注意事项】
```

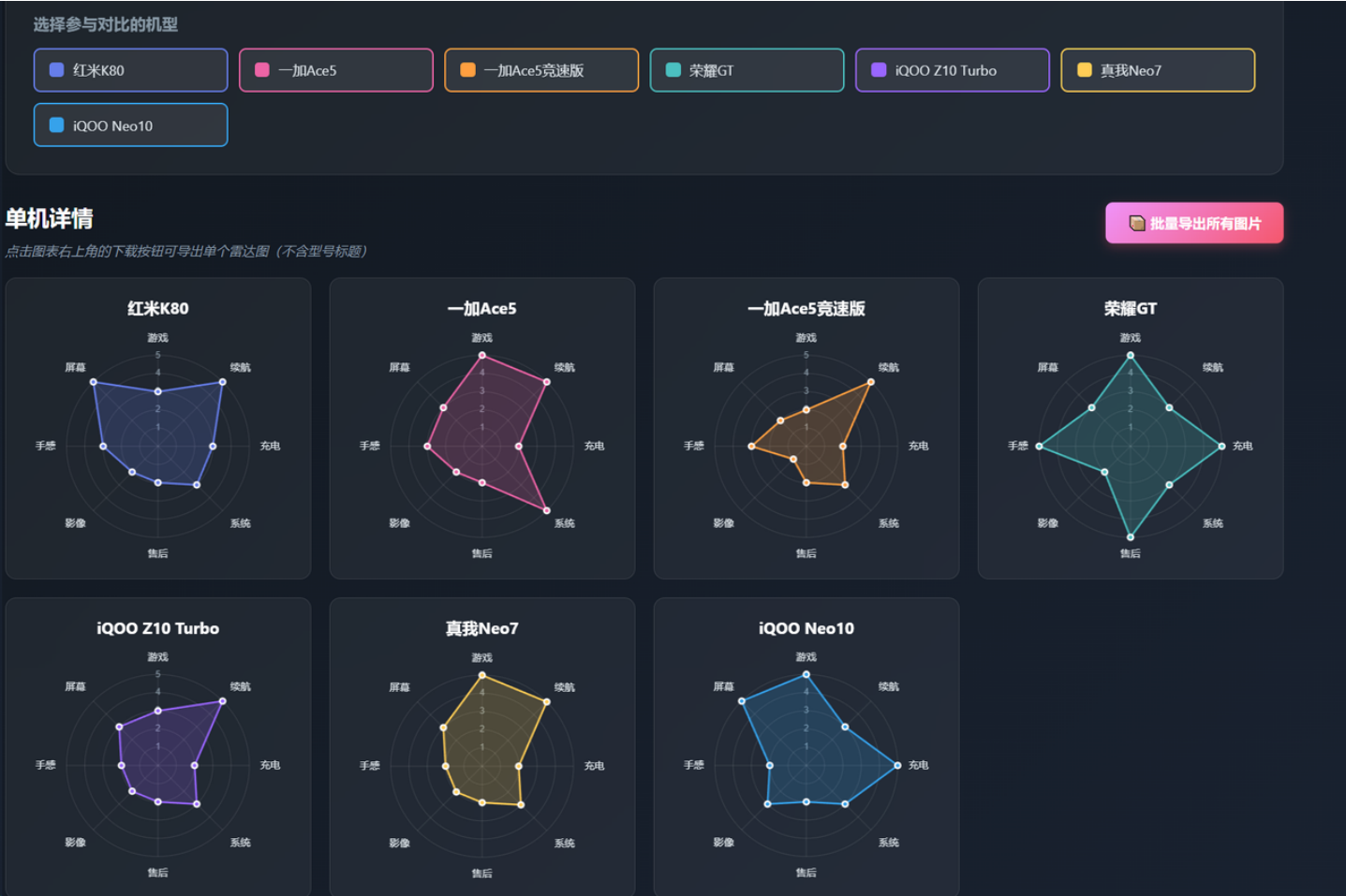
```
120 =====
121 =
122
123 ⚠ 重要提示：
124 - 请遵守京东的服务条款和爬虫协议
125 - 不要过度爬取，控制请求频率
126 - 下载的视频仅供个人学习使用，不得用于商业目的
127 - 尊重视频版权，不得传播或二次发布
128 - 程序会自动添加延迟以避免被反爬
129
130 =====
131 =
132
133 【技术栈】
134
135 =====
136 =
137
138 - requests - HTTP请求库
139 - Playwright - 浏览器自动化工具（支持JavaScript渲染）
140 - re - 正则表达式
141 - json - JSON解析
142
143 =====
144 =
145
146 【文件说明】
147
148 =====
149 =
150
151
152
153 jd_video_crawler_v2.py - 主程序（推荐使用）
154 jd_video_crawler.py - 旧版本（已弃用）
155 test_crawler_v2.py - 功能测试脚本
156 test_urls.txt - 测试URL文件示例
157 README.md - 详细文档
158 使用指南.txt - 本文件
159
160 =====
161 =
162
163 【版本信息】
164
165 =====
166 =
167
168 版本：v2.0
169 发布日期：2025-11-22
170 主要改进：
171 - ✅ 改进视频ID提取逻辑
172 - ✅ 优化腾讯视频API调用
173 - ✅ 增强错误处理
174 - ✅ 提高稳定性
```

```
160
161 =====
162  =
163  【联系与反馈】
164  =====
165  =
166  如有问题或建议，欢迎反馈！
167  =====
168  =
```

如何使用？ 问豆包！

### 3.5 好看的雷达图设计方法





具体设计方法看[目提效代码](#)，文档里面有说明

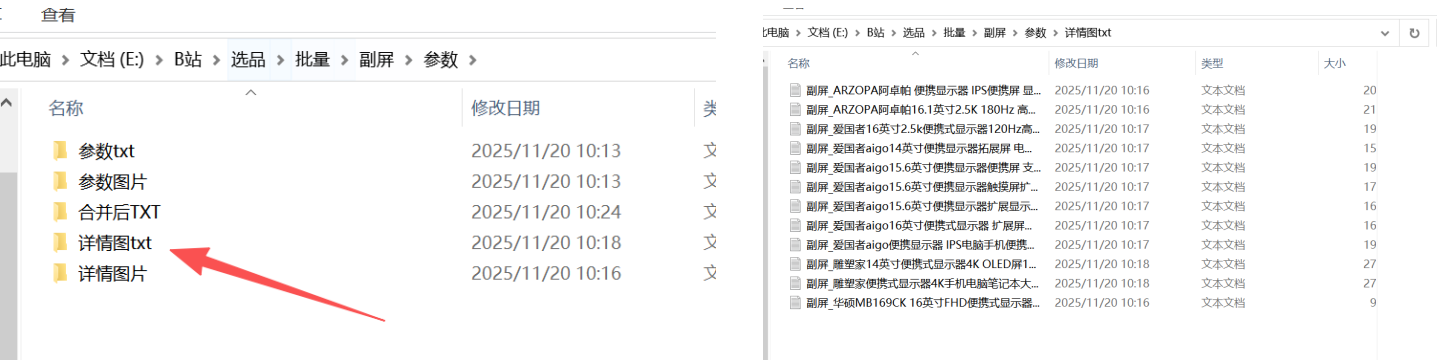
雷达图这里主要用在产品图片PPT中比较多，通过差异化设计突出产品多维度优势。

## 四、脚本提效

拆解对标，先写出来几个自己满意的作为**对标稿子(终极稿子)**

然后再去拆解出对应的AI提示词，在写稿子的时候，投入对应的素材。

比如讲解产品的时候，我的系统里面**参数文件夹**会有一个**详情图txt文件夹**，里面保存的是每个产品的详情图ocr识别结果。我会把所有的txt以及**分类.xlsx**发给AI，让它在这些内容的基础上创作脚本，最大程度上保证了参数的正确性，比如价格。



这样写产品介绍的文案就会很快，提示词采用的是百万哥的帖子里写的



## 提示词-0-仅供参考

- 1 我现在在做B站好物带货项目，我提供给你的文件中：“**终极稿子.txt**”，是我已经做完的，4个反响还不错的视频口播稿，也就是我自己的风格；**12个产品详情内容txt文档是我准备推荐的产品--【副屏】**；
- 2 我需要你：帮我以这**12个产品**为主，先分析我过去口播稿的内容、结构和风格（比如采用了【引入痛点-介绍产品-总结】的结构、使用真实经历增加可信度、每几十秒就有一个吸引观众的“钩子”，增加口播视频完播率，以上只是举例，具体的需要你自己分析），**再参考分类.xlsx里面的sheet=副屏的产品参数，为我创作出属于我自己的【副屏】带货口播稿。**
- 3 口播稿需要满足以下几个要求：
- 4 1. 提到的所有的产品可以使用产品标题中的简称；
- 5 2. 所有产品的参数一定要正确，都要来源于其产品链接中，不可以使用未提及的参数，如果有未提及的参数，请使用【待确认】预留占位；
- 6 3. 总字数在**4000-5000**字；
- 7 4. 口播稿中的up主自称为 **xx** ；
- 8 5. 口播稿要完全按照口播的形式，不需要“发言人 08:55”这种时间轴，也不需要表格，要完全符合口语化的叙述风格；
- 9 6. 款式介绍不要使用一二三这种称呼，要用“第一款”、“第二款”、“第三款”这种称呼；
- 10 7. 口播稿中避免使用“\*\*”、“•”、“/”这些符号，而是使用口语化的表达，比如“\*\*”完全可以去掉，多行产品前的“•”可以改为首先...其次...然后...最后...的口语化表达方式，而"/"要改为“或”，双引号和xx Script备注也需要去掉，以此类推，类似的符号表达尽量都避免，而使用口语化的表达，直接念出来就是口播内容。
- 11 8. 对技术术语和英文缩写不要进行音译处理和口语化替代。好了，要求如上述，给我完整的口播稿内容。
- 12 9. 要求每个产品都要被介绍到，不能重复介绍也不能缺少介绍。

得到初始的稿子后，接下来就是去校正有没有错误，润色文案。

使用方法：

**修改提示词标记部分，上传所有产品的txt文件，上传分类.xlsx, 上传终极稿子.txt**

注意⚠：终极稿子是自己调教好的脚本

使用效果：

A12

fx

做风【Uzi推荐】A4Pro电竞桌国家补贴实木电动升降桌

| A                 | B                           | C       | D       | E   |
|-------------------|-----------------------------|---------|---------|-----|
| 商品名称              | 商品详情页URL                    | 月销      | 单价      | 到手价 |
| 乐歌电动升降桌           | http://item.jd.com/10152100 | 1169    | 1049    | 3.5 |
| 西昊D03B 电竞桌        | http://item.jd.com/10014300 | 1299    | 1175    | 2   |
| 京东京造电动升降桌         | http://item.jd.com/1000343  | 1369    | 1269    | 5   |
| Brateck北弧电竞桌      | http://item.jd.com/1000225  | 1293    | 1293    | 6   |
| andasea安德斯特电竞桌    | http://item.jd.com/10025100 | 2104    | 1704    | 7   |
| 西昊D07电竞桌          | http://item.jd.com/10013500 | 1799    | 1499    | 2   |
| 京东京造电动升降桌         | http://item.jd.com/1000352  | 1789    | 1689    | 5   |
| 乐歌电动智能升降桌         | http://item.jd.com/10151100 | 2619    | 2299    | 3.5 |
| 乐歌碳纤维电竞桌          | http://item.jd.com/1015031  | 3098    | 2698    | 10  |
| Brateck北弧电竞桌      | http://item.jd.com/1001819  | 2699    | 2299    | 5   |
| 做风【Uzi推荐】A4Pro电竞桌 | http://item.jd.com/10011900 | 3049    | 3049    | 2   |
| 骁骑XD1pro电竞桌       | http://item.jd.com/1000621  | 2940    | 2940    | 2   |
| 9am智能电动升降桌        | http://item.jd.com/1000326  | 3675    | 3175    | 2   |
| andasea安德斯特电竞桌    | http://item.jd.com/10009200 | 3699    | 3699    | 7   |
| 西昊D07电竞桌          | http://item.jd.com/10011100 | 5399    | 5058    | 2   |
| Brateck双电机电竞桌     | http://item.jd.com/1001061  | 4749    | 3799    | 5.9 |
| 智芯黑胡桃实木电竞桌        | http://item.jd.com/1011420  | 5605    | 5605    | 2   |
| 9am智能电动升降桌        | http://item.jd.com/1001189  | 5158    | 3858    | 2   |
| 9am智能电动升降桌        | http://item.jd.com/1000669  | 10975   | 9175    | 2   |
| 韦露姿W1实木电竞桌        | http://item.jd.com/1012025  | 2635.31 | 2235.31 | 2   |

豆包

升降桌的核心参数、痛点、分类及适配人群

内容由 AI 生成

高度记忆、遇阻回退和久坐提醒。

流线椭圆桌腿设计，没有棱角，家里有小孩不容易磕碰，可调节脚能应对地面不平。适合全家适用和品质追求者，放在品质居家或专业办公区，ENF 级环保材质加双电机稳定性，既健康又实用，现在补贴后到手价 2299 元，追求品质的直接选。

第十一款 做风 A4Pro 电竞升降桌

核心电竞玩家的终极之选，做风 A4Pro 这款是 Uzi 联合推荐的旗舰款！桌面长度 1.6 米，宽度 80cm，专业配置完全够用。桌面材质是进口南洋榆木芯，E0 级环保标准，高密度不易变形，耐用性超强。

双电机升降系统，霍尔磁环同步控制，毫米级精度，升降时几乎零晃动，承重 250 斤，多屏加重型设备都能支撑。升降速度 35mm/s，比普通双电机快不少，噪音仅 35dB，图书馆级静音，游戏时升降完全不打扰。

控制方式是四档升降记忆加四档遇阻回弹，还有内嵌集成理线箱和 15W 无线闪充，桌面整洁又方便，可选光效系统，电竞氛围拉满。适合核心电竞玩家和办公族，放在专业电竞房或高端办公区，Uzi 推荐的品质真话，现在补贴后到手价 3049 元，核心玩家直接冲。

第十二款 骁骑 XD1pro 电竞桌

年轻电竞玩家和潮人必备，骁骑 XD1pro 这款颜值和性能双在线！桌面长度有 1.4 米、1.6 米、1.8 米可选，空间灵活搭配。桌面材质是 25mm ENF 级加厚一体板，比 E0 级更环保，甲醛释放量极低，安全无异味。

双电机驱动，进口电机动力强劲，升降速度 28mm/s，240 斤，3 维立式结构设计，升降时稳如泰山。范围 72-117cm，坐站切换自如，控制方式是手机 APP 四档记忆，能调节 47 种光效，潮酷感十足。

阿满选品\_Brateck双电机电动升降桌\_北弧电脑桌\_站立办公升降桌\_升降台 M5棕1.6米.txt和其他 24 文件

询问关于这篇文章的任何问题

## 五、其他提效

### 5.1 蓝链

tips1:蓝链输入的时候，采用倒序输入链接的方式，得到的链接顺序会自动校正为正常顺序。

这里指的是链接选品的时候，倒序插入商品链接，点击确定后，得到的产品顺序是正序的

带货商品 插入小店

我的选品车 商品计划 推荐商品 营销会场

请输入关键词搜索

自营商品 佣金说明

链接选品

艾美特 (AIRM... 艾美特 (AIRM... 艾美特 (AIRM... 奥克斯 (AUX) ... 先锋 (SINGFUN... 艾美特 (AIRM...  
¥649.00 佣金: 8.1% ¥549.00 佣金: 7% ¥499.90 佣金: 7% ¥219.00 佣金: 4% ¥639.90 佣金: 2% ¥292.42 佣金: 8%  
格力 (GREE) ... 格力 (GREE) ... 格力 (GREE) ... 格力 (GREE) ... 听科冬季保暖加... 奥克斯 (AUX) ...  
¥629.00 佣金: 5.9% ¥414.00 佣金: 5% ¥329.00 佣金: 6% ¥258.00 佣金: 8% ¥143.00 佣金: 4% ¥119.00 佣金: 4%

暂未选择商品 请从左侧添加

编辑选品文案 取消 确定

带货商品 插入小店

链接添加商品

输入商品链接，如有多条请用“;” (英文逗号) 或换行隔开，每次最多添加30条链接

识别链接

支持淘宝联盟、京东联盟、多多进宝、全淘和白搜、up主小黄、工商、费星、数字藏品、个性化商品链接  
·支持识别以https://开头的、含商品id的商品链接，暂不支持含shopid的多app短链  
·禁售品类、禁售商品将不允许在站带货; 带货商品要求  
·带货商品添加遇到问题请查看帮助文档: 带货商品帮助文档

暂未选择商品, 请从左侧添加

取消 确定

点击链接选品

倒序输入链接，识别链接，点击确定后就是正序的

tips2:想要实现流水线生产视频到发布，这一步是必须要解决的

想要实现自动化输入蓝链的，可以参考下我的研究笔记。 提效代码

<<

子流程设置

主流程设置

结果展示

B站获取视频评论区链接

优化体验版本-2025.9.21

优化版本-2025-10.13

蓝链提效研究

原理研究-2025-10-11

换行测试-成功

自定义文本替换(研究-可不看)

梳理逻辑

第一步

第二步

第三步

B站获取对标视频弹幕

B站批量下载视频

阻止剪映专业版更新

雷达图网页工具

如何高效批量套版--稿定设计

1. 准备模板-2025.11.16

2. 设置批量套版

3. 准备文件

4. 运行

蓝链提效研究

原理研究-2025-10-11

1. 输入框内商品链接是 span 标签，可采用js调用修改前端显示代码。因为不涉及后端服务器的响应，应该没问题

2. 空白输入框与含蓝链的输入框。

空白输入框

1 <div id="editor" class=" active " uia-uid='

2 <!--?Lit\$52525482385\$-->

3 <!--?Lit\$52525482385\$--><bili-

4

5 <bili-comment-pictures-upload>

6 </div>

包含蓝链输入框

1 <div class="brt-editor"

contenteditable="true" style="height:

112px;" uia-uid="0|4"><span

class="goods brt-h1" data-type="goods"

data-goods\_url="https://b23.tv/mall-

GFAbYD3lH-78WYQ" data-

goods\_id="105466118"

contenteditable="false">京东11.11红包

</span> <span class="goods brt-h1"

data-type="goods" data-

goods\_url="https://b23.tv/mall-

GFAbYD3lH-18JiN1oIkfg" data-

goods\_id="951900779352072192"

contenteditable="false" uia-uid="0|3">盖

世小鸡超新星手柄</span> </div>

发现输入框是被多个shadow-root包裹的，需要穿透shadow-root，才能获取到输入框

代码块

1 第一层shadow: bili-comments

2 第二层shadow: bili-comments-header-renderer

3 第三层shadow: bili-comment-box

4 第四层shadow: bili-comment-rich-textarea

## 5.2 数据采集

推荐GitHub上面一个自媒体采集的项目，下面这个是原本的代码



MediaCrawler-main.zip

12.25MB



可以采集知乎，小红书，抖音，B站等平台的内容。比如知乎文章，评论等

使用场景：

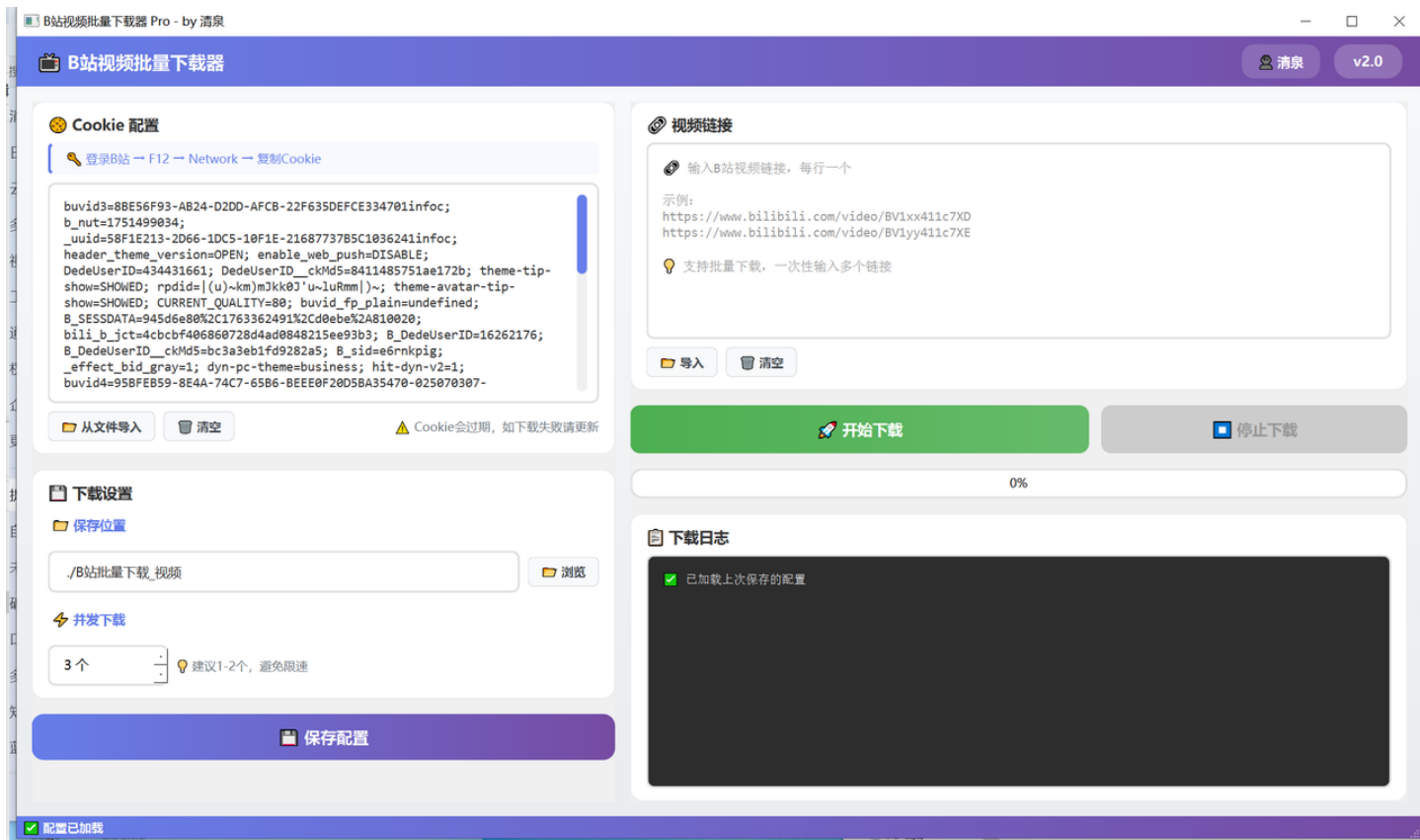
- 1.可以使用这个工具，把B站想要调研的UP的所有视频信息采集到本地，进行AI调研。
- 2.可以使用这个工具，把知乎的测评文章及评论采集到本地，进行脚本撰写的参考。

## 5.3 B站字幕批量下载



具体方法见 [📄 提效代码](#) 文档说明

## 5.4 B站视频批量下载



具体方法见 [国 提效代码](#) 文档说明

## 5.5 去水印

推荐工具链接: <https://www.kaipai.com/video-tool/remove-watermark>

## 5.6 获取B站视频脚本

### 方法1:

下载B站视频<https://snapany.com/zh/bilibili>

通义听悟，视频转文本

### 方法2:

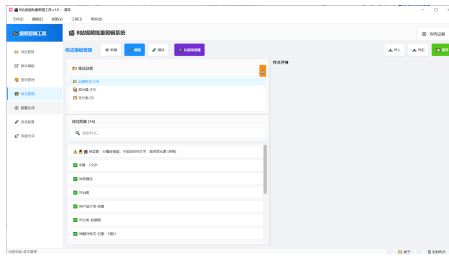
get笔记网页版，输入B站链接。

## 5.7 视频半自动化剪辑系统

开发中，敬请期待下一篇



脚本编辑页面



样式管理页面



配置生成页面

## 预期效果

- 1 🎬 B站批量剪辑工具 - 核心功能介绍
- 2 这是一个专业的视频批量生成系统，专为B站内容创作者设计。以下是它能实现的主要功能：
- 3 🎯 核心功能
- 4 1. 📝 脚本管理
- 5 编写和管理视频脚本
- 6 自动按行分段（每行一个段落）
- 7 实时段落数统计
- 8 2. 🎨 样式配置系统
- 9 从剪映导入样式模板
- 10 为每个段落配置不同的样式
- 11 支持文字、图片、视频、贴纸等多种元素
- 12 支持多元素样式（如三图逐个呈现）
- 13 支持额外样式叠加
- 14 3. 🗣️ 智能配音生成
- 15 Edge TTS（免费）：支持多种中文音色
- 16 MiniMax TTS（高质量）：支持情感控制和克隆音色
- 17 智能增量生成：只重新生成修改的部分
- 18 语速调节：0.5x-2.0x 倍速
- 19 4. 🎬 视频草稿生成
- 20 一键生成剪映草稿文件
- 21 自动应用样式、动画、转场
- 22 支持多元素时间线（按关键词逐个显示元素）
- 23 自动生成字幕
- 24 5. 📁 项目化管理
- 25 独立的项目工作空间
- 26 素材库管理（表情包、视频、音效、产品图）
- 27 配置文件自动保存
- 28 支持项目导入导出
- 29 6. 🔄 增量生成
- 30 智能检测脚本和配置变化
- 31 只重新生成修改的部分
- 32 大幅提高工作效率
- 33
- 34 🚀 工作流程
- 35 编写脚本 → 配置样式 → 生成配音 → 生成视频草稿 → 导入剪映 → 导出最终视频
- 36
- 37 🎯 适用人群

- 38  B站知识分享创作者 - 快速批量制作讲解视频
- 39  产品运营 - 批量生成产品介绍视频
- 40  教育培训 - 快速制作教程类视频
- 41  内容创业者 - 提高视频产出效率
- 42
- 43  实际应用场景
- 44  知识分享视频 - 快速生成讲解类视频
- 45  产品介绍视频 - 批量生成产品展示视频
- 46  教程类视频 - 自动配音和样式应用
- 47  营销视频 - 批量生成营销内容
- 48  短视频创作 - 快速批量制作

## 六、乘风破浪一起去做“提效航海家”

在 B 站好物创作的提效之路上，从选品的精准化、PPT 制作的高效化，到脚本的智能化、工具的集成化，每一个环节的优化都是对内容创作效率与质量的双重提升。这些提效方法与工具，不仅是我个人在两年职场与自学探索中的沉淀，更希望能为同样在 B 站内容电商领域耕耘的从业者提供一份实用的参考。

很幸运能加入**生财 B 站好物|深海圈**和充满活力的航海群，和各位圈友交流碰撞、共享资源、拆解难题——是社群里的思维火花让我在批量查询佣金时想去找到更优解方法，在 PPT 套版环节少走了弯路，找到了自己 B 站好物提效的全链路解决方案。

提效的路还长，有了深海圈和社群的陪伴，我不再是单打独斗的创作者。期待我们继续在这个圈子里互通有无、迭代方法，把提效的势能转化为内容的声量与商业的增量，在 B 站内容电商的领域里，打造出属于自己的内容与商业价值闭环。一起做乘风破浪的“**提效航海家**”！

